

ONBOARDING+

FORMAÇÃO 04 | ASP.NET CORE MVC AVANÇADO



Carlos Freire

MCSD | Microsoft Certified Solution Developer
Software Architecture





Roadmap



4 – ASP.NET CORE MVC ADVANCED

OBJETIVO:

Esta formação tem o objetivo de apresentar conceitos mais avançados do ASP.NET Core MVC. E irá capacitar você a trabalhar em nível mais profissional e conseguir entregar mais valor.

Nível de Complexidade
☆☆☆ Intermédio



Metodologia de Aprendizado:

- **Ciclo de Entrega:** Cada tópico é composto por Teoria + Demo, encerrando uma videoaula.
- **Snapshot de Código:** Uma pasta exclusiva com o código-fonte final para cada aula gravada.
- **Evolução Incremental:** O projeto avança aula a aula, permitindo comparação imediata.



Tópicos da formação:

- **Pré-Requisitos**

- **Módulo: Aprofundar no Essencial**

- Pipeline do ASP.NET
- Criar projeto ASP.NET (Core Empty)
- [DEMO-01] – Criar projeto do Zero (ou quase)
- Ferramentas de Front-end
- [DEMO-02] – Ferramentas de Front-end
- O que são Bundling e Minicaton?
- [DEMO-03] – Bundling e Minicaton
- Trabalhar com mais Layouts
- [DEMO-04] – Utilizar Layout Admin
- Trabalhar com Tag Helper Customizado
- [DEMO-05] – Criar um CRUD e Tag Helper
- Trabalhar com View Component
- [DEMO-06] – Implementar o View Component
- Roteamento Inteligente
- [DEMO-07] – Rotas Inteligentes

- **Módulo: Injeção de Dependência (DI)**

- Como funciona o DI no ASP.NET Core
- Como funciona Ciclos de Vida do DI
- Configurar a DI no ASP.NET Core
- Testar os 3 Tipos DI no ASP.NET Core
- [DEMO] – O Maravilhoso mundo das DIs

- **Módulo: Segurança**

- CSRF (Cross-Site Request Forgery)
- XSS (Cross-Site Scripting)
- HSTS (HTTPS Strict Transport Security)
- [DEMO] – CSRF + XSS + HSTS = Segurança
- Explorar ASP.NET Identity (Authentication)
- [DEMO] – Identity | Authentication
- Explorar o Identity | Authorization | Roles
- Authorization | Policies Required Roles
- Authorization | Policies Required Claims
- Custom Authorization
- [DEMO] – Identity | Authorization

- **Módulo: Configurações Avançadas**

- Organizar a classe program.cs
- Configurar Ambientes
- [DEMO] – Organizar e Configurar
- Ler ficheiro de configuração
- Utilizar User Secrets (secrets.json)
- [DEMO] – Ler ficheiro e User Secrets
- Tratamento de Erros e Middlewares
- [DEMO] – Tratamento de Erros
- Logs para Observabilidade e Monitoria
- [DEMO] – Observabilidade e Monitoria
- Auditoria/Log através de Filtro
- [DEMO] – Monitoria com Filtro

- **Módulo: Globalização da Aplicação**

- Trabalhar com Culturas
- Ajustar validações do front-end
- Criar Data Annotation Customizado
- Adicionar Suporte para mais Culturas
- [DEMO] – WebApp Multi-Linguagem

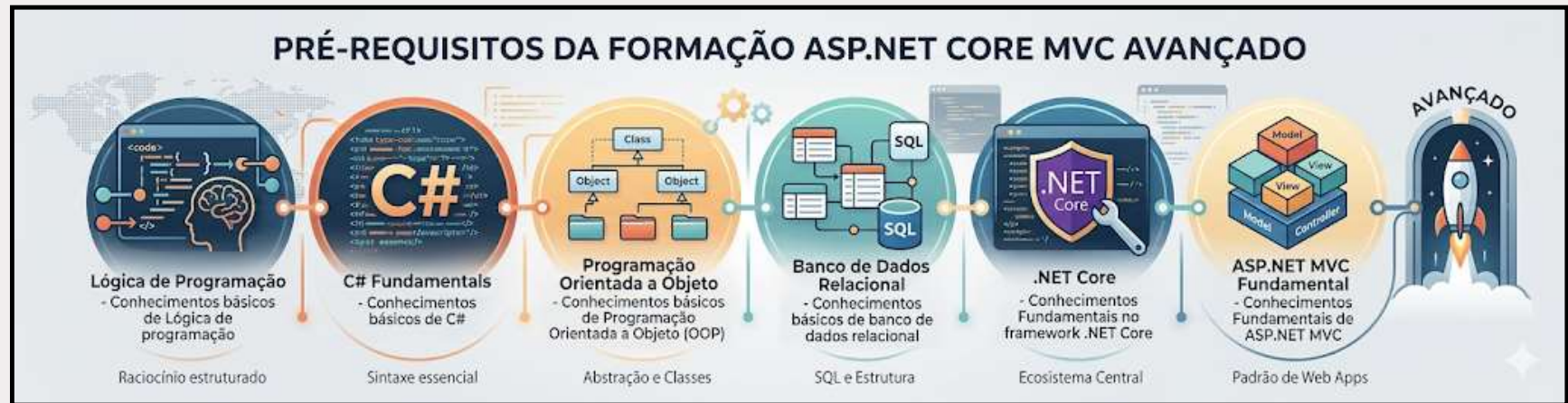
- **Módulo: Conhecimentos Indispensáveis**

- Trabalhar com Ajax e Modal Window
- [DEMO] – Modal Window
- Políticas de Cookies
- [DEMO] – Políticas de Cookie
- Upload de ficheiro (Create)
- Upload de ficheiro (Edit)
- [DEMO] – Upload de ficheiro
- Tarefas em Background Services
- [DEMO] – Adicionar Watermark

- **Encerramento**



Pré-Requisitos

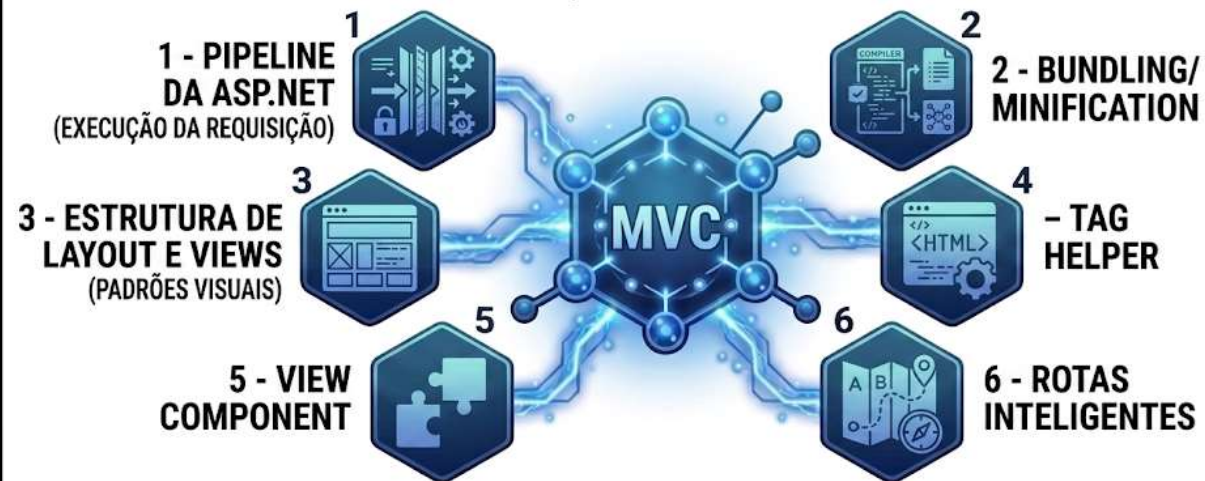




Módulo: Aprofundar o Essencial

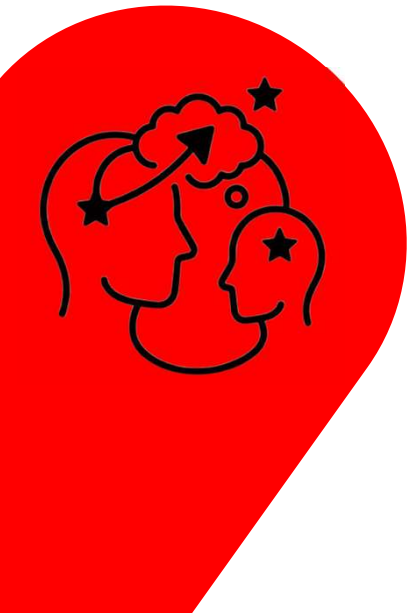
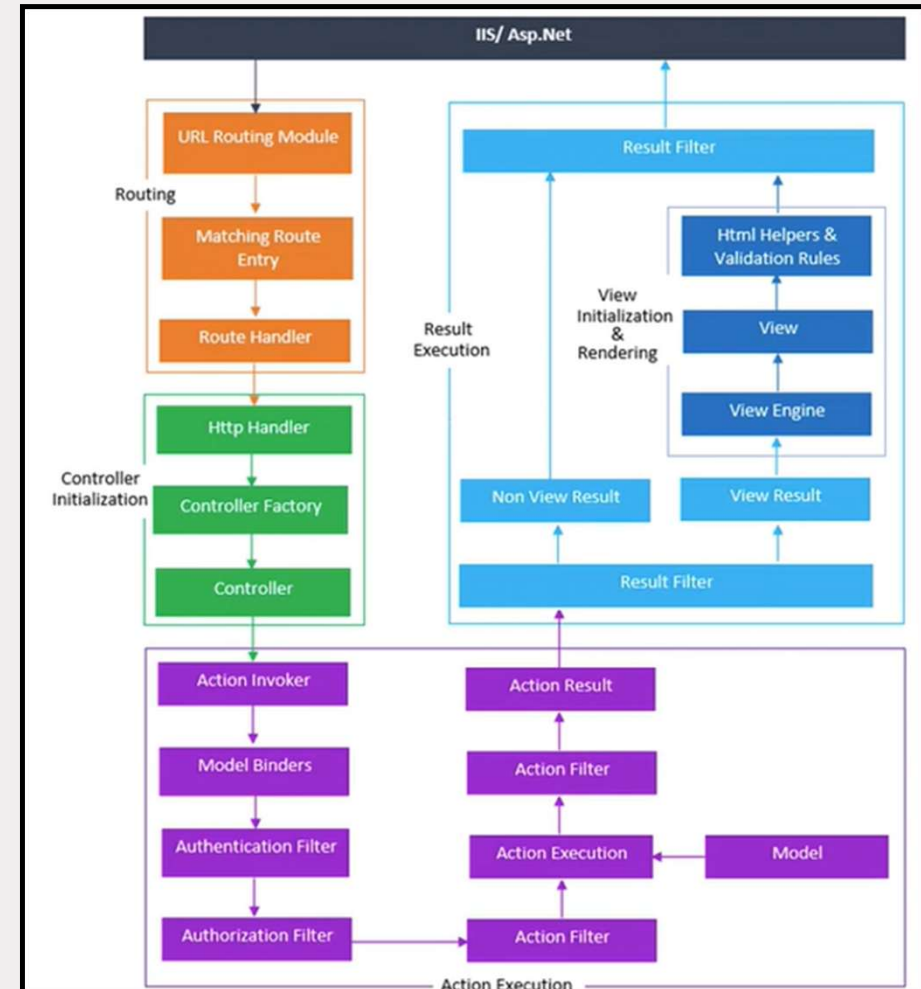
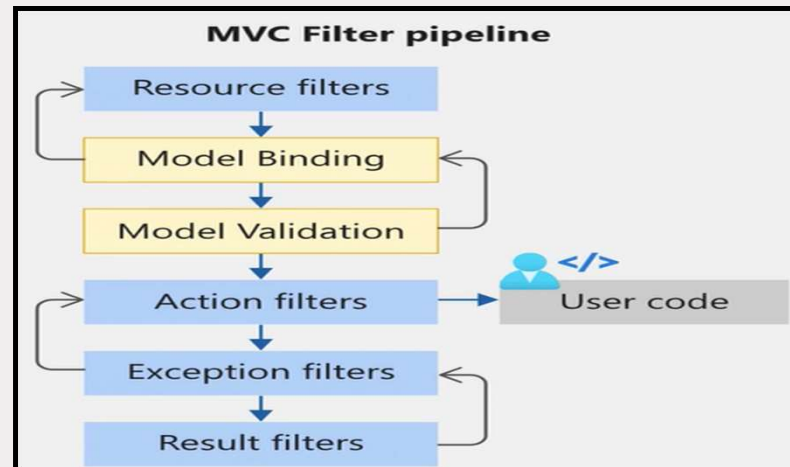
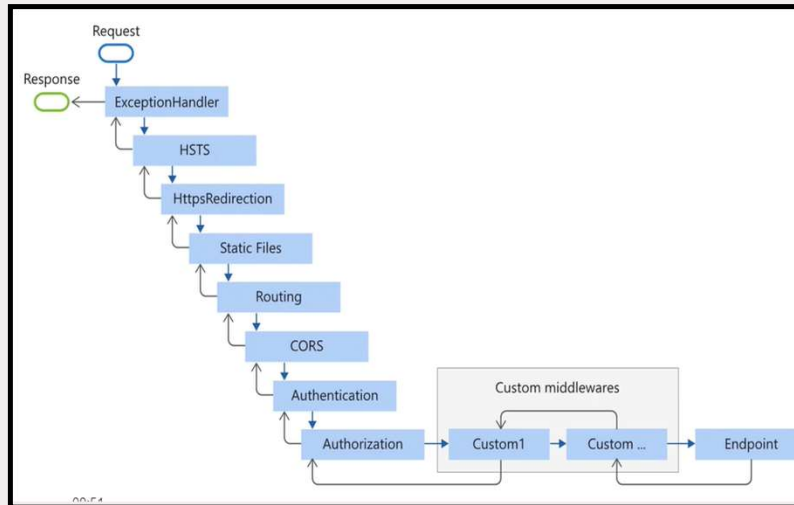
APROFUNDAR O ESSENCIAL

EXPLORANDO O PIPELINE, LAYOUTS E COMPONENTES NO MVC

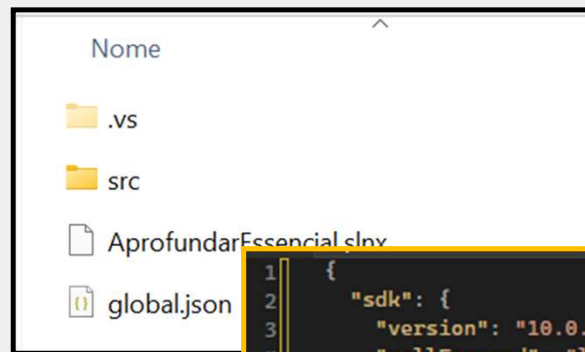
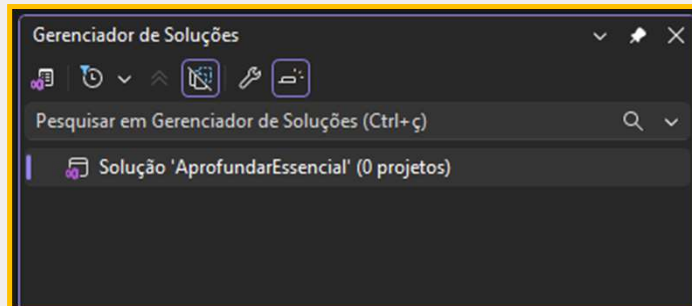


Pipeline do ASP.NET

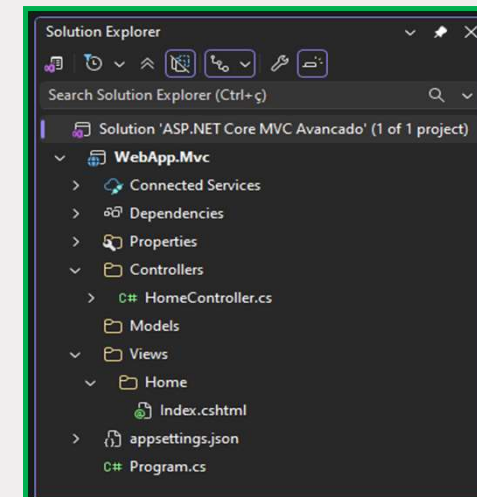
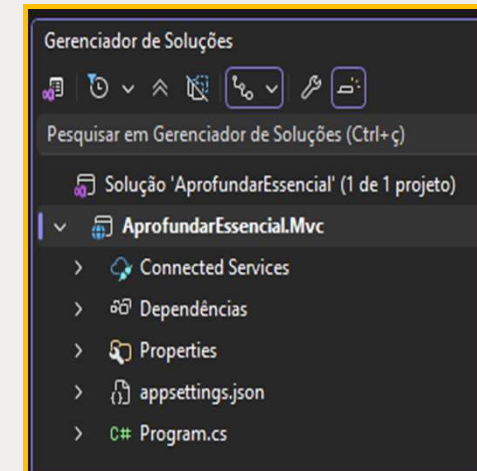
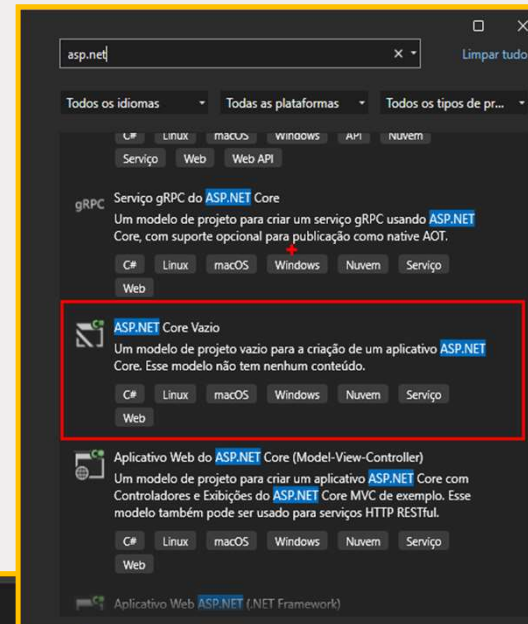
<https://learn.microsoft.com/en-us/aspnet/core/fundamentals/middleware/?view=aspnetcore-10.0>



Criar projeto ASP.NET (Core Empty)



```
1 {
2   "sdk": {
3     "version": "10.0.202",
4     "rollForward": "latestFeature"
5   }
6 }
```



[DEMO] - Criar projeto do Zero (ou quase)



Ferramentas de Front-End

FERRAMENTAS DE FRONT-END ADVANCED

CLI
Client Side Library
GESTÃO DE PACOTES LEVES
(npm/jsDelivr)

libman.json

```

{
  'packages': {
    'bootstrap': 'bootstrap',
    'jquery': 'jquery',
    'font-awesome': 'font-awesome'
  }
}

```

_Layout.cshtml

```

@{
<header>
<main>
    @RenderBody()
</main>
</footer>
...
@}

```

_ViewImports.cshtml

```

1 @using AprofundarEssencial.Mvc
2 @addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers

```

AprofundarEssencial.Mvc

```

1 @{
2     Layout = "_Layout";
3 }

```

_ViewStart.cshtml

```

1 @{
2     Layout = "_Layout";
3 }

```

Program.cs

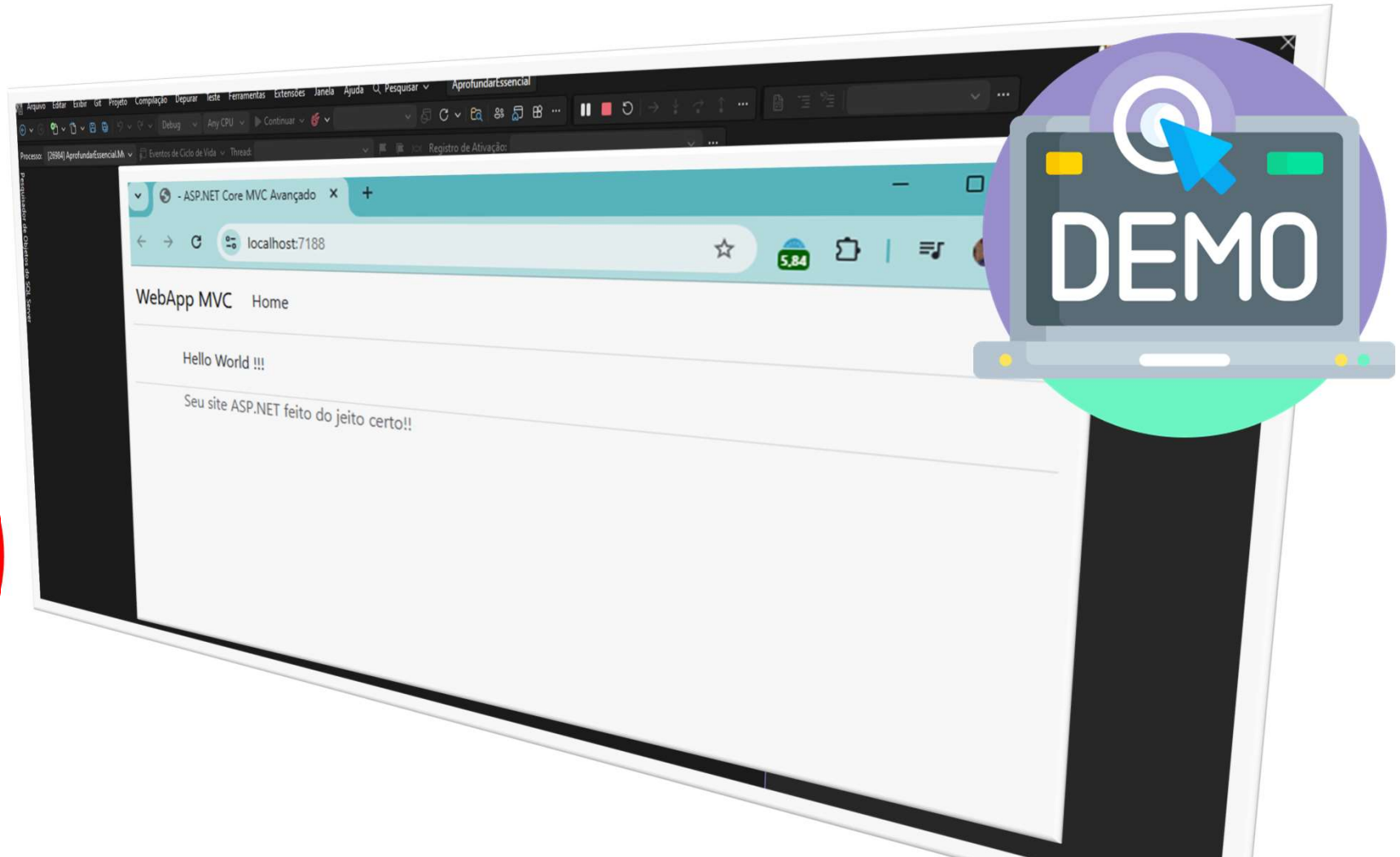
```

1 var builder = WebApplication.CreateBuilder(args);
2
3 // Add services to the container.
4 builder.Services.AddControllers();
5
6 var app = builder.Build();
7
8 app.MapControllerRoute(
9     name: "default",
10    pattern: "{controller}/{action}/{id?}");
11
12 app.Run();

```



[DEMO] - Ferramentas de Front-end



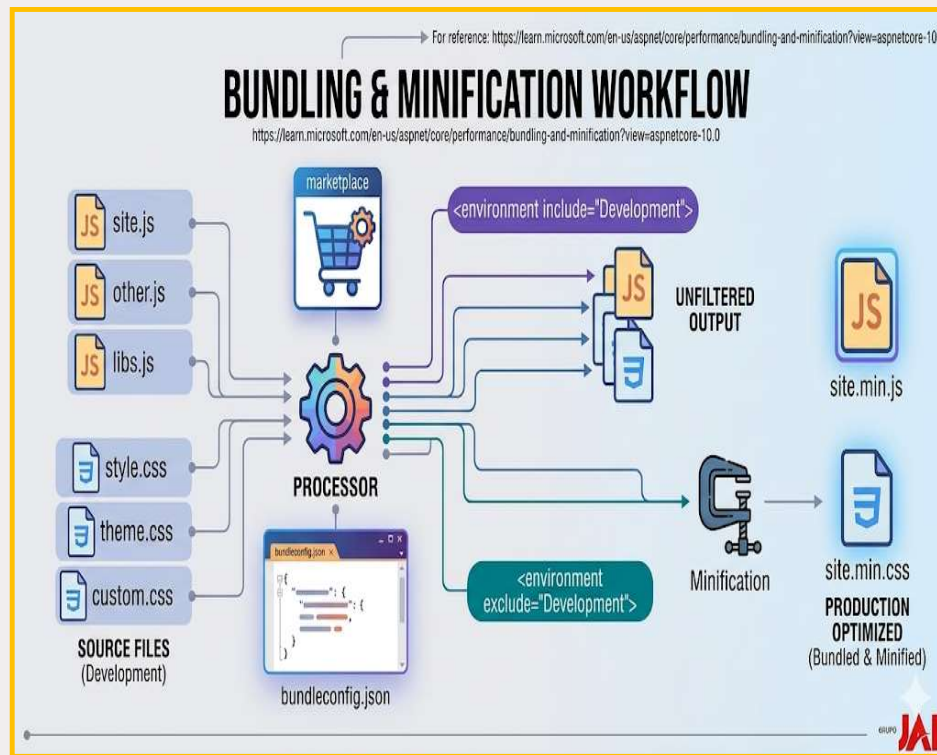
O que são Bundling e Minification?

Visão geral criada por IA

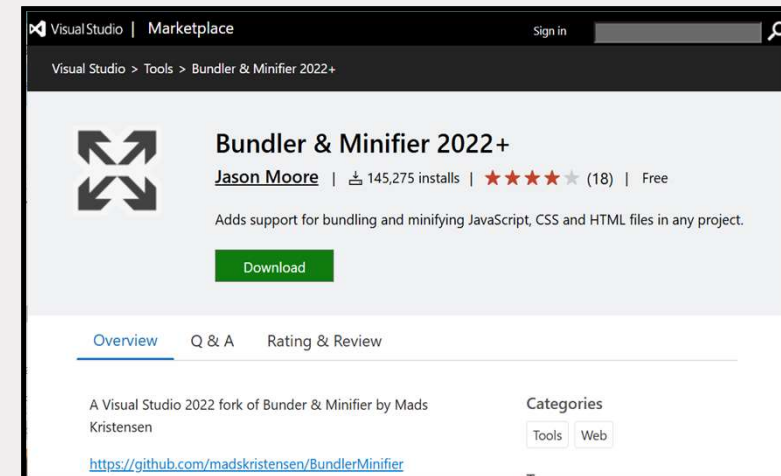
O **Bundling** (agrupamento) no ASP.NET é uma técnica de otimização de desempenho que combina vários arquivos CSS ou JavaScript em um único arquivo. Ele reduz o número de requisições HTTP necessárias para carregar uma página web, acelerando o tempo de carregamento no navegador do usuário. [Microsoft Learn +3](#)

Visão geral criada por IA

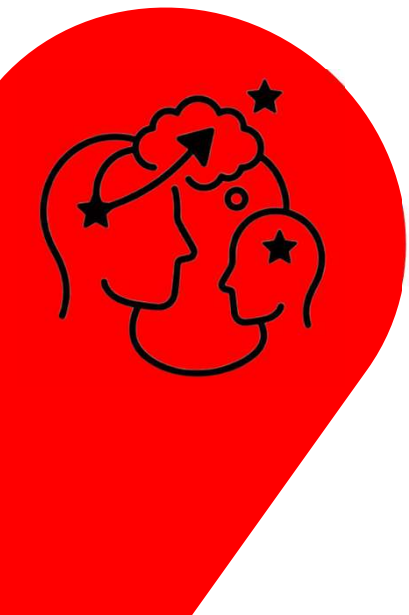
Em ASP.NET, **minification** (minificação) é uma técnica de otimização que reduz o tamanho de arquivos CSS, JavaScript e HTML, removendo caracteres desnecessários como espaços em branco, comentários e quebras de linha, sem alterar a funcionalidade do código. [DevMedia +2](#)



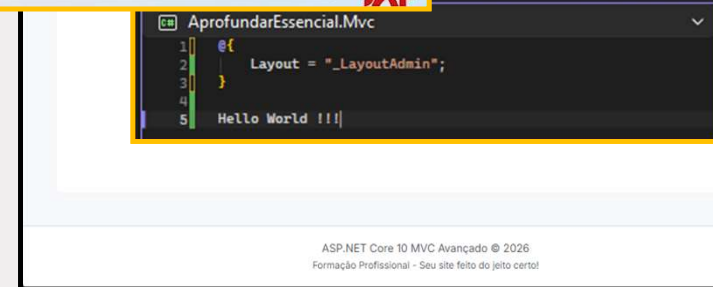
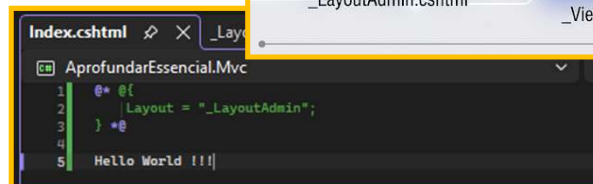
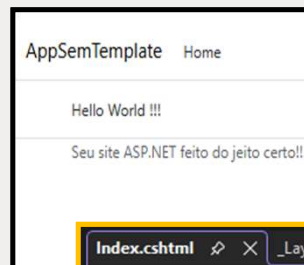
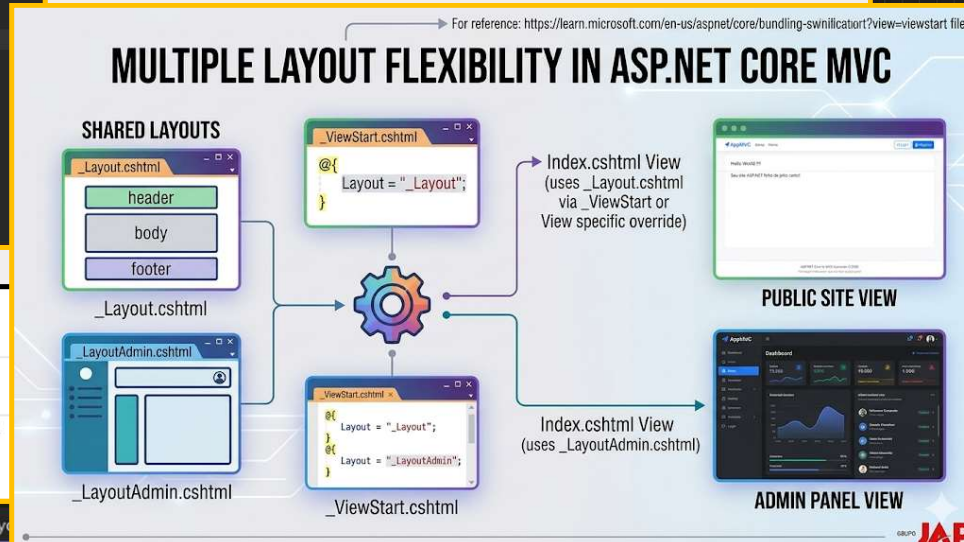
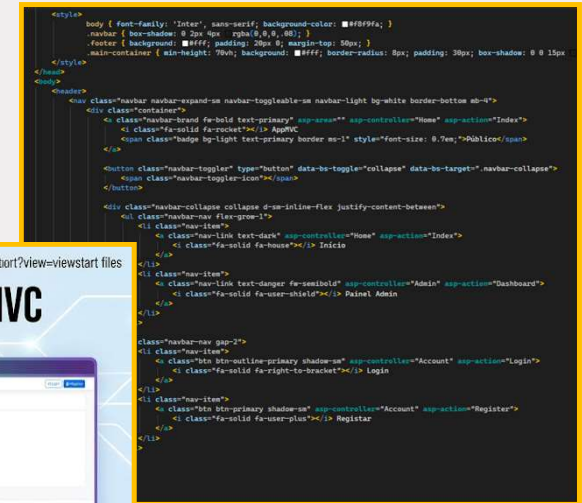
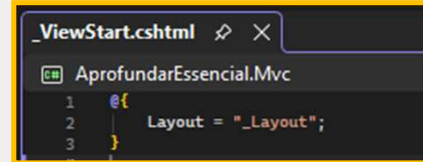
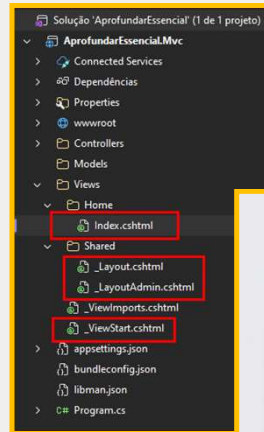
<https://marketplace.visualstudio.com/items?itemName=Failwyn.BundlerMinifier64>



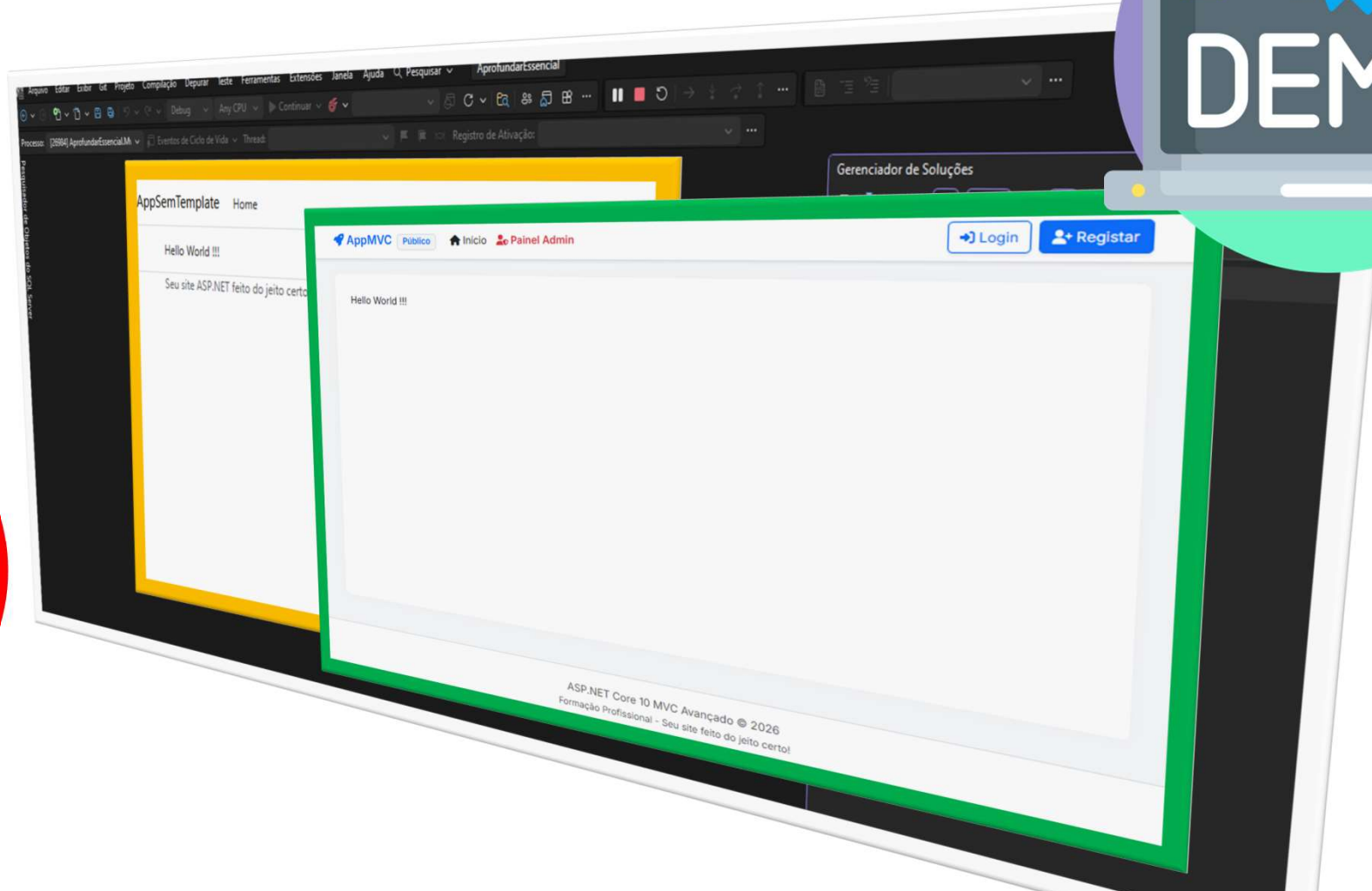
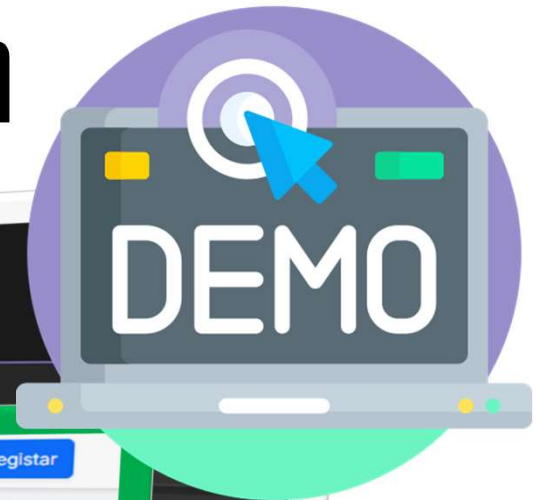
[DEMO] - Bundling & Minification



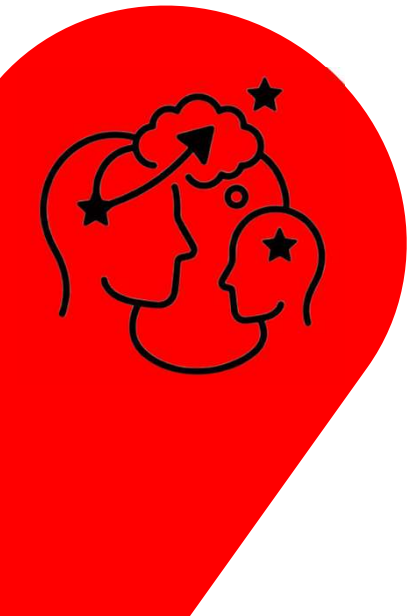
Trabalhar com mais Layouts



[DEMO] - Utilizar Layout Admin



Trabalhar com Tag Helper Customizado



CUSTOM TAG HELPER: MyButton IMPLEMENTATION

STEP 1: DEVELOPING THE TAG HELPER CLASS

```

MyButtonTagHelper.cs
[HtmlTargetElement("*", Attributes = "button-type, route-id")]
public class MyButtonTagHelper : TagHelper
{
    public MyButtonTagHelper(IHttpContextAccessor contextA
    _contextAccessor = contextAccessor;

    [HtmlAttributeName("button-type")]
    public ButtonType TypeButtonSelection { get; set; }
    [HtmlAttributeName("route-id")]
    public int RouteId { get; set; }

    public override void Process(int rtem, age, scot, Cott
    {
        switch (nameAction = 'Details';
        nameAction = dFontAwesome(fa-search);
        else: nameClass = 'Edit';
        nameClass = AdFontAwesome(fa-pencil-alt);
        else: nameClass = 'Delete';
        spanIcon = AddFontAwesome(fa-trash);
    }

    output.TagName = "a";
    output.Attributes.SetAttribute("href", ...);
    output.Attributes.SetAttribute("class", nameClass);
    var iconSpan = new TagBuilder("span");
    iconSpan.AddCssClass(spanIcon);
    output.Content.AppendHtml(iconSpan);
}

public enum ButtonType
{
    Details = 1,
    Edit,
    Delete
}
    
```

STEP 2: USING THE TAG HELPER IN RAZOR VIEW

```

Products/index.cshtml
<table>
<tr>
<td>
<my-button button-type="Details"
route-id="@item.Id"></my-button>
<my-button button-type="Edit" route-
id="@item.Id"></my-button>
<my-button button-type="Delete"
route-id="@item.Id"></my-button>
</td>
<td>
@* <a asp-action="Details" class="btn btn-info"
asp-route-id="@item.Id"> <span class="fa
fa-search"></a>
<a asp-action="Edit" class="btn btn-warning"
asp-route-id="@item.Id"> <span class="fa fa-pencil-
alt"></span></a>
<a asp-action="Delete" class="btn btn-danger"
asp-route-id="@item.Id"> <span class="fa fa-
trash"></span></a> *@
</td>
</tr>
</table>
    
```

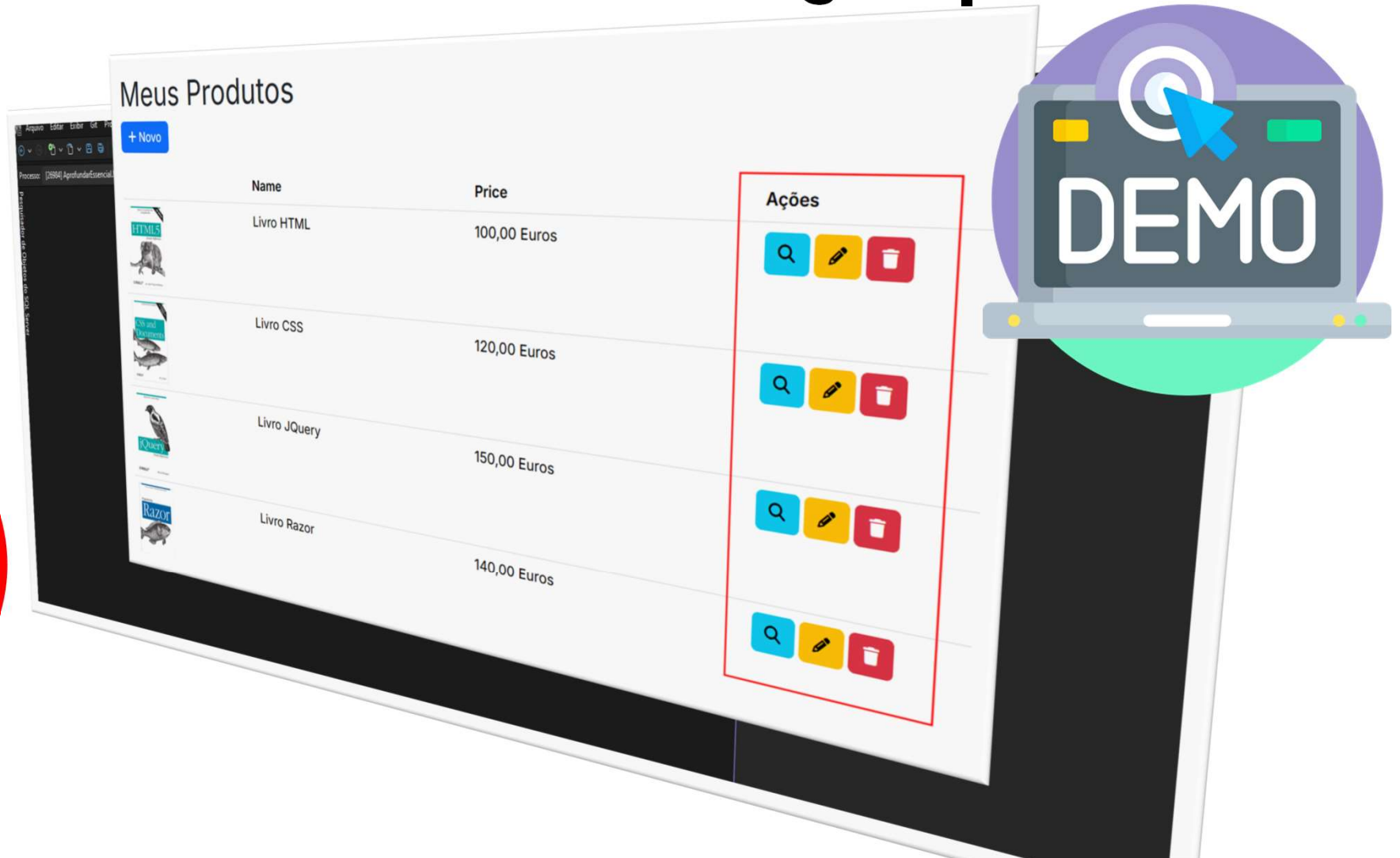
ENCAPSULATION: UI LOGIC IS CENTRALIZED
REUSABILITY: EASY TO USE IN ANY CRUD
MAINTAINABILITY: CHANGE ONE CLASS, UPDATE ALL VIEWS

For reference: <https://learn.microsoft.com/en-us/aspnet/core/mvc/views/razor#tag-helpers>

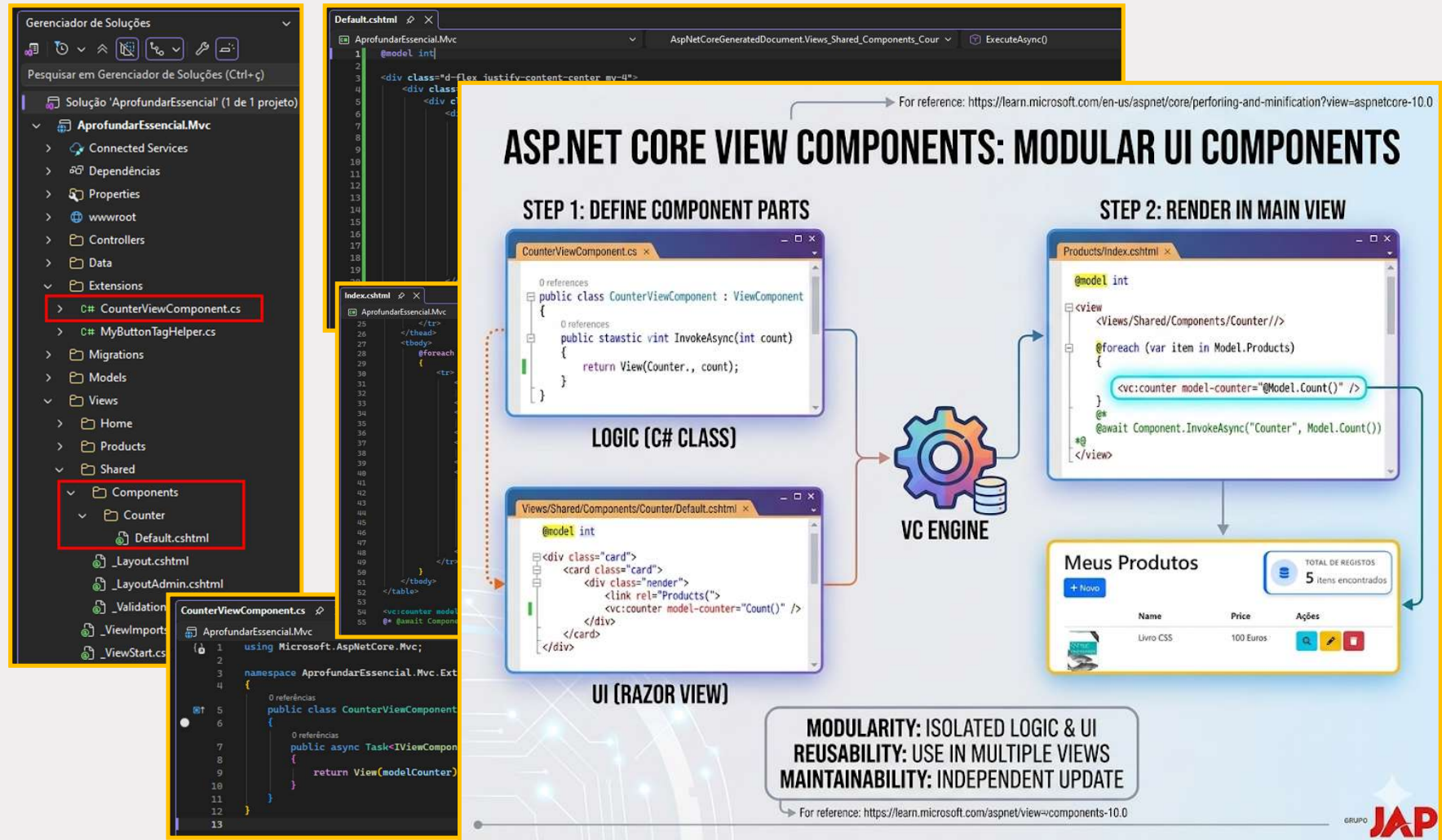
13 references
 public class Product
 {
 [Key]
 11 references
 public int Id { get; set; }
 1 references
 public string? Name { get; set; }
 1 references
 public string? Image { get; set; }
 1 references
 public decimal Price { get; set; }
 }

ardingDotNetDb
 Tabelas
 Tabelas do Sistema
 Tabelas Externas
 Tabelas do Razo Descartadas
 dbo.__EFMigrationsHistory
 dbo.Products
 Colunas
 Id (PK, int, não nulo)
 Name (nvarchar(max), nulo)
 Image (nvarchar(max), nulo)
 Price (nvarchar(max), nulo)

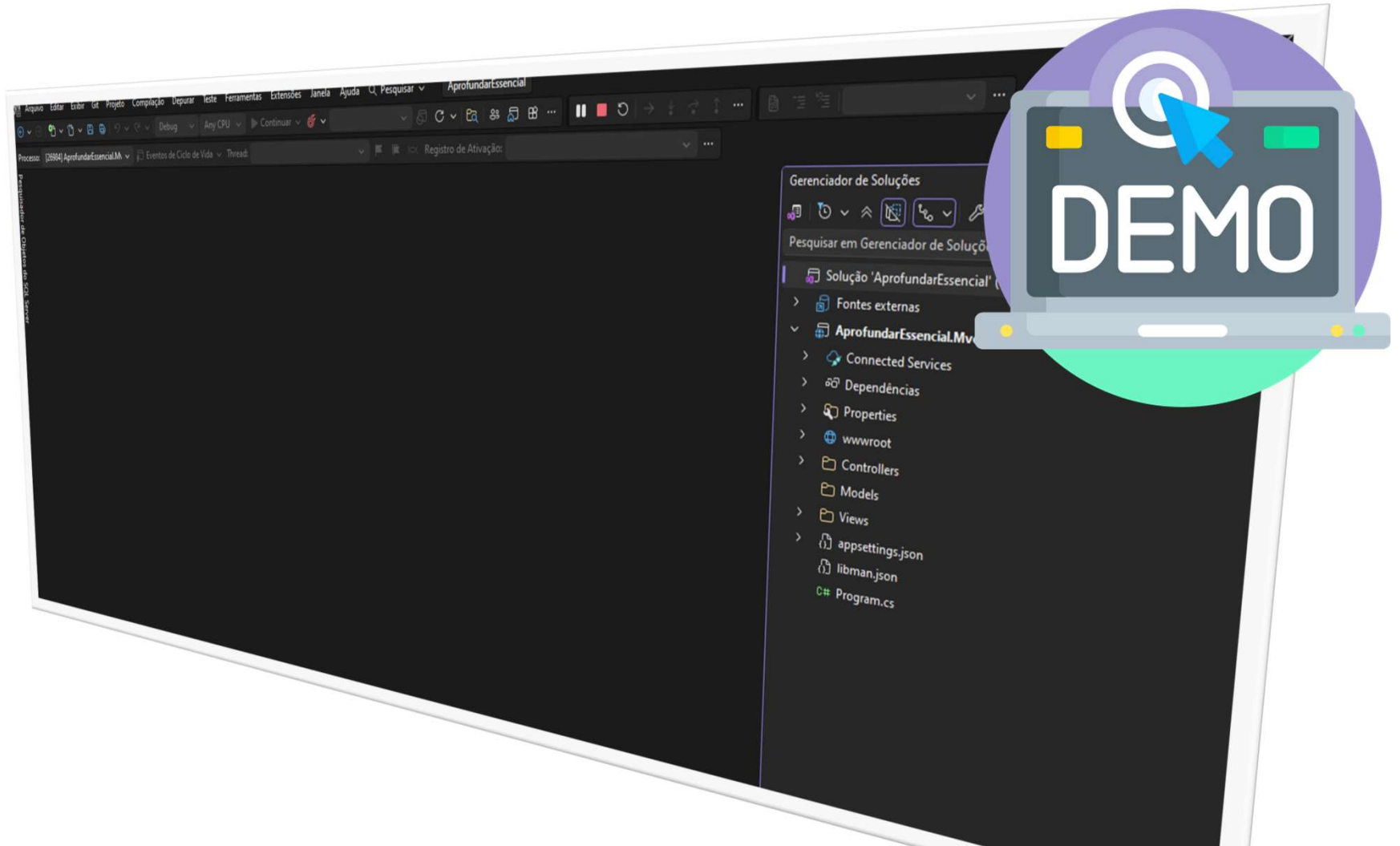
[DEMO] - Criar CRUD e Tag Helper



Trabalhar com View Component

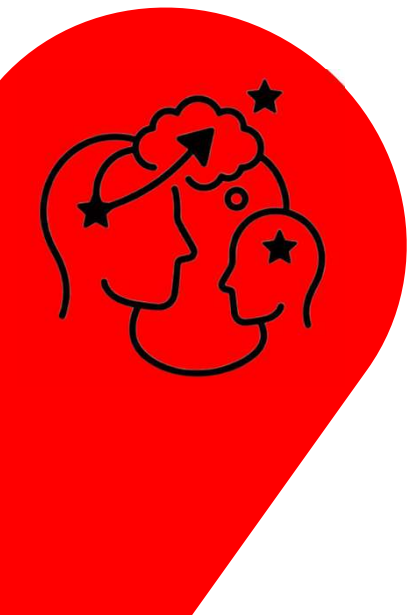
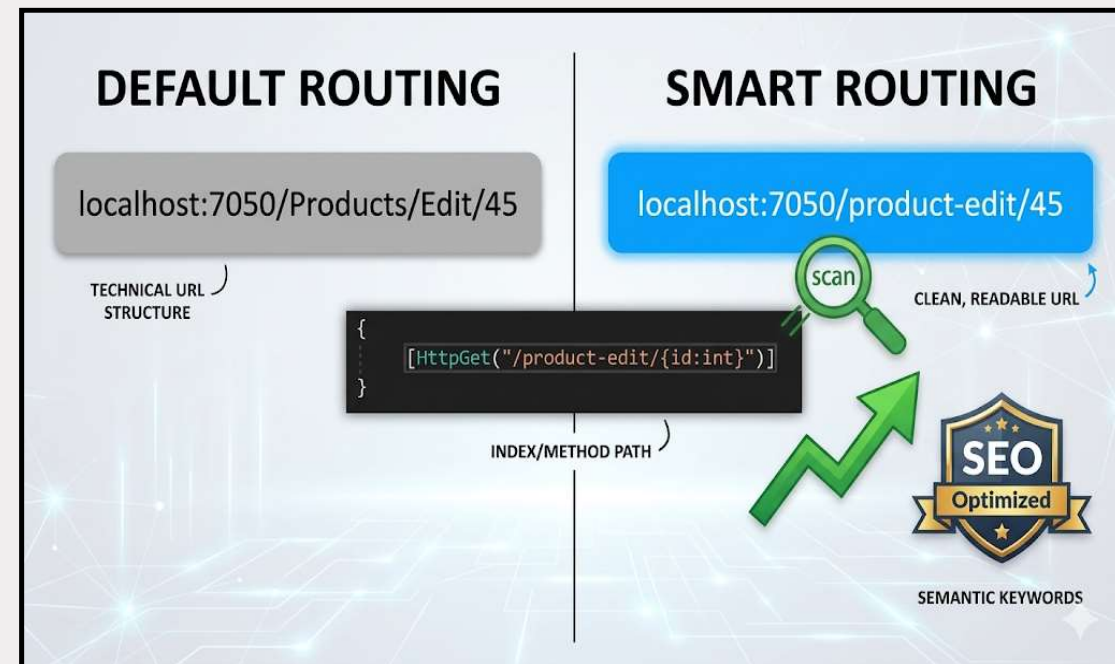
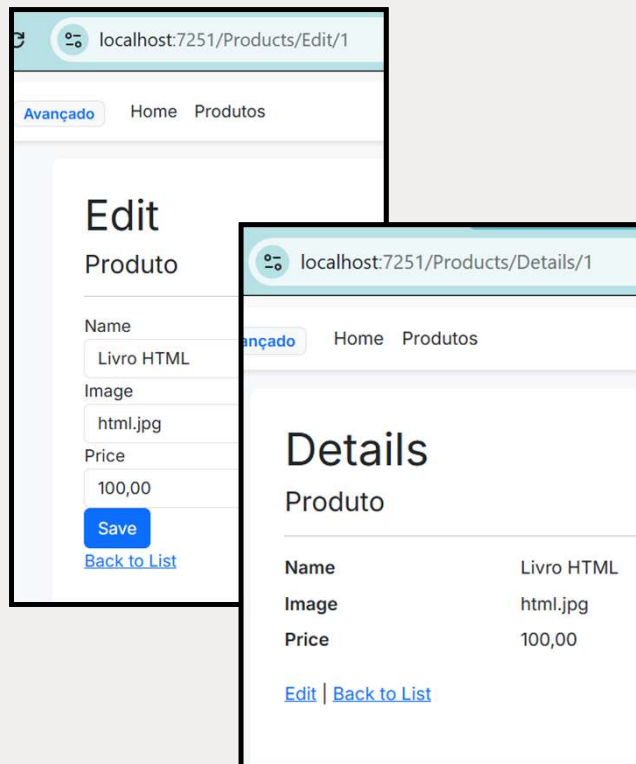


[DEMO] - Usar Tag Help e View Component

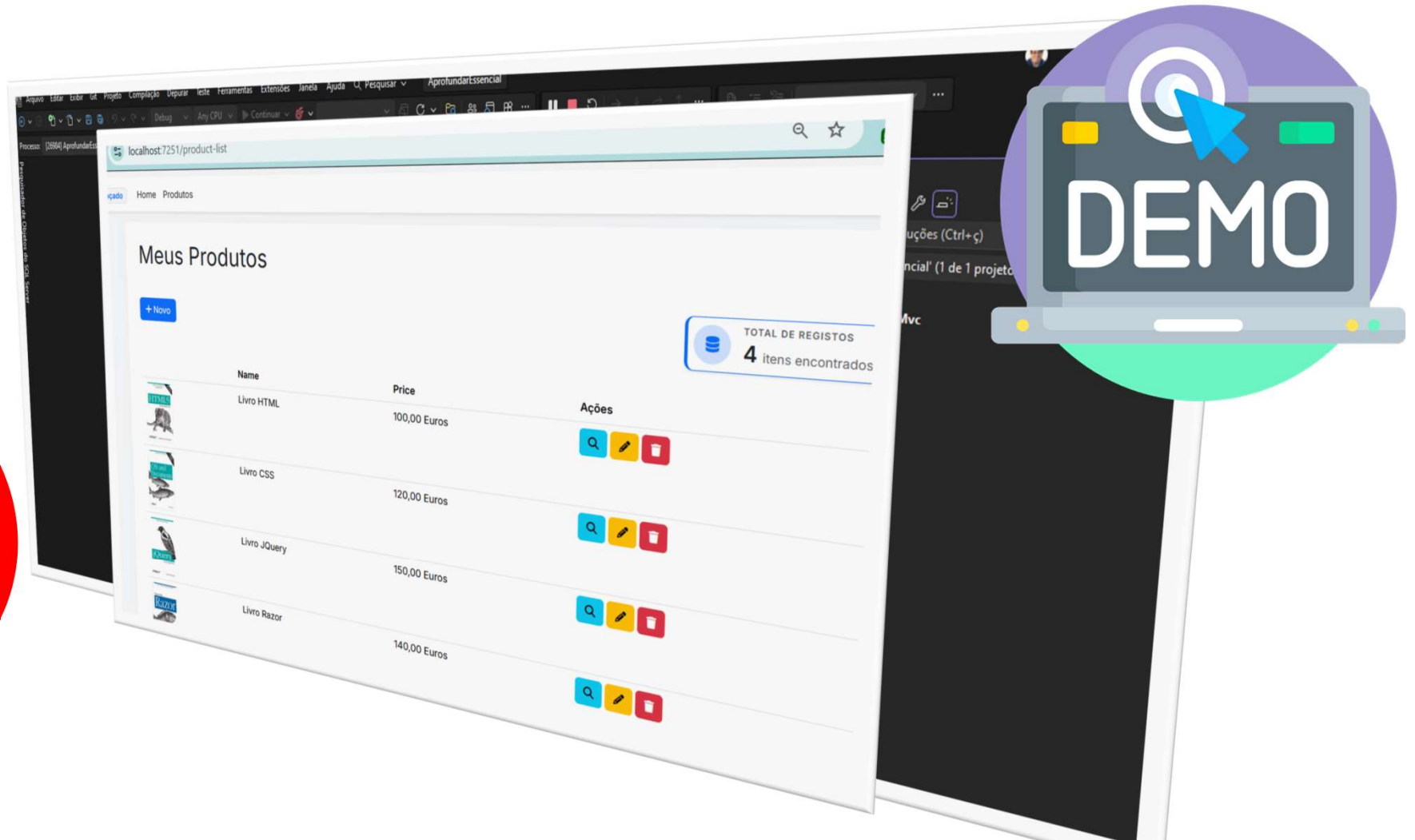


Roteamento Inteligente

SEO, sigla para *Search Engine Optimization* (Otimização para Mecanismos de Busca), é um conjunto de estratégias de marketing digital que visa melhorar o posicionamento de um site nos resultados orgânicos (gratuitos) de buscadores como Google e Bing. O objetivo é aumentar o tráfego qualificado e a visibilidade online, tornando o site mais relevante para os usuários.  Google for Developers +3



[DEMO] - Rotas Inteligentes





Módulo: Injeção de Dependência (DI)

O CORAÇÃO DO ASP.NET CORE
DESACOPLANDO CÓDIGO E GERENCIANDO CICLOS DE VIDA COM EFICIÊNCIA



Como funciona o DI no ASP.NET Core

A Injeção de Dependência (DI) é um **padrão de projeto** que ajuda a reduzir o **acoplamento** de código em sua aplicação.

No contexto do ASP.NET Core a injeção de dependência é usada para **adicionar** serviços e **gerenciar** a vida útil dos objetos.

O ASP.NET Core possui um contêiner de injeção de dependência **integrado**.

Este contêiner é responsável por **construir** os objetos necessários e **fornecê-los** quando necessário.

Para que um serviço seja injetado no código é necessário **registrá-lo** ao contêiner de DI

```
var builder = WebApplication.CreateBuilder(args);  
  
builder.Services.AddScoped<ApplicationDbContext>();  
  
builder.Services.AddControllersWithViews();  
  
var app = builder.Build();
```

Para fazer uso do serviço registrado é de costume **injetar** este serviço via **construtor**.

```
public class AlunosController : Controller  
{  
    private readonly ApplicationDbContext _context;  
  
    public AlunosController(ApplicationDbContext context)  
    {  
        _context = context;  
    }  
}
```



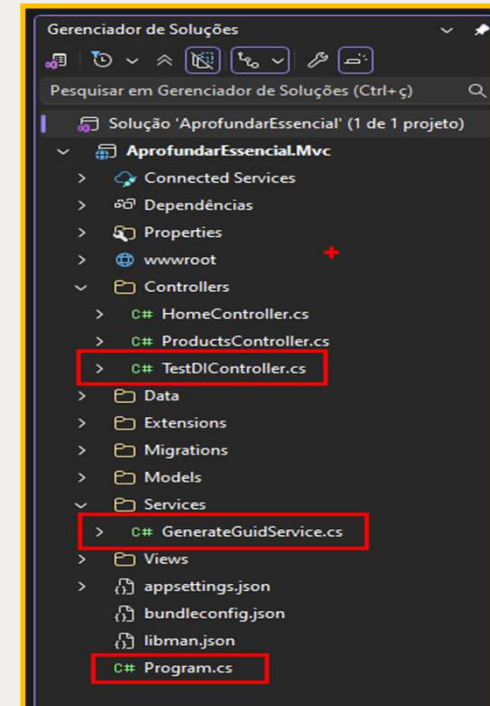
Configurar a DI no ASP.NET Core

```

GenerateGuidService.cs*
AprofundarEssencial.Mvc
namespace AprofundarEssencial.Mvc.Services
{
    2 referências
    public class GenerateGuidService : IGenerateGuidService
    {
        3 referências
        public Guid OperationId { get; private set; }

        0 referências
        public GenerateGuidService()
        {
            OperationId = Guid.NewGuid();
        }

        4 referências
        public interface IGenerateGuidService
        {
            3 referências
            Guid OperationId { get; }
        }
    }
}
    
```



```

Program.cs
AprofundarEssencial.Mvc
1 using AppSemTemplate.Data;
2 using AprofundarEssencial.Mvc.Extensions;
3 using AprofundarEssencial.Mvc.Services;
4 using Microsoft.EntityFrameworkCore;
5
6 var builder = WebApplication.CreateBuilder(args);
7
8 builder.Services.AddControllersWithViews();
9
10 //builder.Services.AddRouting(options =>
11 //    options.ConstraintMap["slugify"] = typeof(RouteSlugifyParameterTransformer));
12
13 builder.Services.AddSingleton<HttpContextAccessor>, HttpContextAccessor>();
14
15 builder.Services.AddScoped<IGenerateGuidService, GenerateGuidService>();
16
17 builder.Services.AddDbContext<AppDbContext>(o =>
18     o.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
19
    
```

```

1 referência
public class TestDIController : Controller
{
    private readonly IGenerateGuidService _generateGuidService;

    0 referências
    public TestDIController(IGenerateGuidService generateGuidService)
    {
        _generateGuidService = generateGuidService;
    }

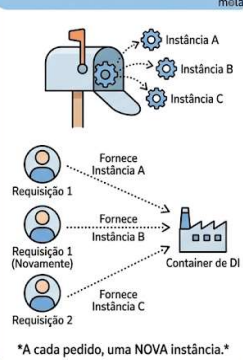
    0 referências
    public IActionResult Index()
    {
        var result = $"Operation ID: {_generateGuidService.OperationId}";
        return View(result);
    }
}
    
```



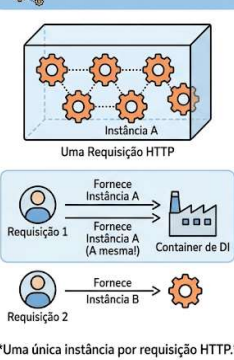
Tipos de Ciclos de Vida do DI #1

CICLOS DE VIDA DA INJEÇÃO DE DEPENDÊNCIA (DI) NO ASP.NET CORE

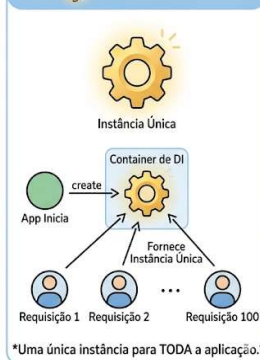
TRANSIENT (TRANSITÓRIO)



SCOPED (ESCOPADO)



SINGLETON



1. Transient (Transitório)

Use quando o serviço é leve, não guarda estado e você quer evitar qualquer efeito colateral entre diferentes partes da aplicação.

- **Serviços de Cálculo ou Utilitários:** Um serviço que apenas formata strings, calcula impostos ou valida o **Código Postal de Portugal** que criamos. Não há razão para reaproveitar a instância.
- **Envio de E-mail/SMS:** Cada vez que um componente precisa disparar uma notificação, ele recebe uma instância nova e limpa para configurar o destinatário e a mensagem.
- **Mapeadores (AutoMapper):** Transformar um Model em ViewModel. É uma tarefa rápida e isolada que não depende de dados anteriores.

2. Scoped (Escopado)

Use quando você precisa que a mesma instância seja compartilhada por todos os componentes envolvidos em **uma única requisição HTTP** (como o Controller e um ViewComponent).

- **Contexto de Banco de Dados (Entity Framework):** O exemplo clássico do `AppDbContext`. Você quer que todas as alterações feitas no Controller e nos serviços durante aquele "clique" do usuário usem a mesma conexão e transação.
- **Informações do Utilizador Logado:** Um serviço que carrega os dados do perfil do usuário do banco de dados uma única vez no início da requisição e os mantém disponíveis para qualquer View ou Componente que precise deles.
- **Carrinho de Compras (durante o checkout):** Para garantir que o cálculo total e a validação de estoque usem exatamente os mesmos dados de memória enquanto o pedido está sendo processado.

3. Singleton

Use quando o serviço deve ser compartilhado por **todos os usuários** e durar enquanto o site estiver no ar. Cuidado: o serviço deve ser "Thread-Safe".

- **Cache de Configurações Globais:** Se o seu site lê configurações de um arquivo JSON ou de uma API externa (como taxas de câmbio que mudam pouco), você carrega uma vez no Singleton e todos os usuários lêem da memória.
- **Logs de Auditoria em Memória:** Um serviço que conta quantos usuários visitaram certas páginas ou armazena logs rápidos antes de enviá-los para um arquivo físico.
- **Clientes de API Externas (HttpClient):** Manter uma única instância do `HttpClient` (geralmente via `IHttpClientFactory`) evita o problema de esgotamento de sockets no servidor, sendo reaproveitado por toda a aplicação.



Tipos de Ciclos de Vida do DI #2

```

GenerateGuidService.cs
AprofundarEssencial.Mvc
AprofundarEssencial.Mvc.Services.GenerateGuidService
GenerateGuidService

2
3
4
5 public class OperacaoService
6 {
7     0 referências
8     public OperacaoService(IGenerateGuidServiceTransient transient,
9                             IGenerateGuidServiceScoped scoped,
10                            IGenerateGuidServiceSingleton singleton)
11     {
12         Transient = transient;
13         Scoped = scoped;
14         Singleton = singleton;
15     }
16
17     4 referências
18     public IGenerateGuidServiceTransient Transient { get; }
19
20     4 referências
21     public IGenerateGuidServiceScoped Scoped { get; }
22
23     4 referências
24     public IGenerateGuidServiceSingleton Singleton { get; }
25 }
26
27 4 referências
28 public class GenerateGuidService : IGenerateGuidServiceTransient, IGenerateGuidServiceScoped, IGenerateGuidServiceSingleton
29 {
30     0 referências
31     public GenerateGuidService()
32     {
33         OperationId = Guid.NewGuid();
34     }
35
36     11 referências
37     public Guid OperationId { get; private set; }
38 }
39
40 3 referências
41 public interface IGenerateGuidService
42 {
43     11 referências
44     Guid OperationId { get; }
45 }
46
47 4 referências
48 public interface IGenerateGuidServiceTransient : IGenerateGuidService
49 {
50 }
51
52 4 referências
53 public interface IGenerateGuidServiceScoped : IGenerateGuidService
54 {
55 }
56
57 4 referências
58 public interface IGenerateGuidServiceSingleton : IGenerateGuidService
59 {
60 }
61

```

// Registos específicos para as interfaces específicas
 builder.Services.AddTransient<IGenerateGuidServiceTransient, GenerateGuidService>();
 builder.Services.AddScoped<IGenerateGuidServiceScoped, GenerateGuidService>();
 builder.Services.AddSingleton<IGenerateGuidServiceSingleton, GenerateGuidService>();
 // Registo do serviço que consome as três dependências
 builder.Services.AddTransient<OperacaoService>();

```

TestDIController.cs
AprofundarEssencial.Mvc
AprofundarEssencial.Mvc.Controllers.TestDIController
TestDIController

1
2 using AprofundarEssencial.Mvc.Services;
3 using Microsoft.AspNetCore.Mvc;
4
5 namespace AprofundarEssencial.Mvc.Controllers
6 {
7     [Route("test-di")]
8     1 referências
9     public class TestDIController : Controller
10     {
11         4 referências
12         public OperacaoService OperacaoService { get; }
13         public OperacaoService OperacaoService2 { get; }
14
15         0 referências
16         public TestDIController(OperacaoService operacaoService, OperacaoService operacaoService2)
17         {
18             OperacaoService = operacaoService;
19             OperacaoService2 = operacaoService2;
20         }
21
22         0 referências
23         public string Index()
24         {
25             return
26                 "Primeira instância: " + Environment.NewLine +
27                 OperacaoService.Transient.OperationId + Environment.NewLine +
28                 OperacaoService.Scoped.OperationId + Environment.NewLine +
29                 OperacaoService.Singleton.OperationId + Environment.NewLine +
30                 Environment.NewLine +
31                 "Segunda instância: " + Environment.NewLine +
32                 OperacaoService2.Transient.OperationId + Environment.NewLine +
33                 OperacaoService2.Scoped.OperationId + Environment.NewLine +
34                 OperacaoService2.Singleton.OperationId + Environment.NewLine;
35         }
36     }
37 }
38

```

localhost:7110/test-di

Primeira instância:
 cb8f886c-4039-4a16-9c77-fd7b39eccc28
 c83dcc31-7d7c-4962-85ee-6b991036aa04
 a6dfc048-4aef-4063-b924-f4fcc9e01a3e

Segunda instância:
 f94a1db0-6879-4b4c-ad1f-19d7c8968fd7
 c83dcc31-7d7c-4962-85ee-6b991036aa04
 a6dfc048-4aef-4063-b924-f4fcc9e01a3e

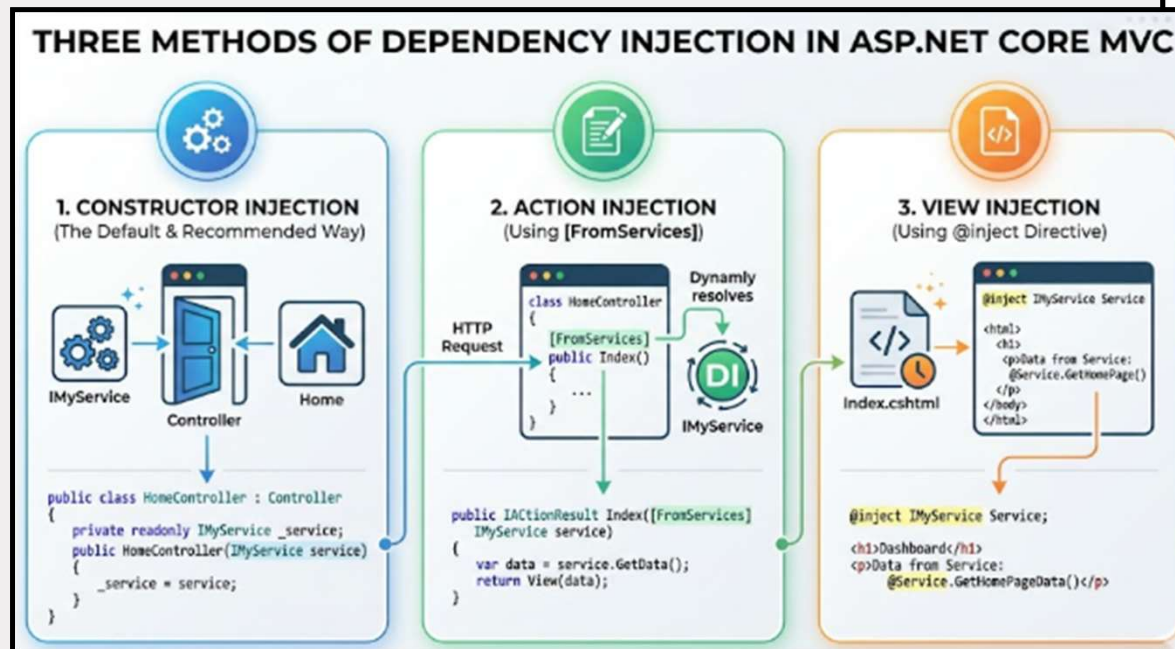
localhost:7110/test-di

Primeira instância:
 b8e9a1dd-8efd-4351-af22-565f626f842f
 dcf5f32-08db-4990-bda2-74436981d6d1
 a6dfc048-4aef-4063-b924-f4fcc9e01a3e

Segunda instância:
 5ba20c78-032a-44bd-8c29-6f0fa99ef2b1
 dcf5f32-08db-4990-bda2-74436981d6d1
 a6dfc048-4aef-4063-b924-f4fcc9e01a3e



Outras formas de se fazer DI



1. Injeção via Construtor (Constructor Injection)

É a abordagem padrão e mais recomendada. As dependências são declaradas nos parâmetros do construtor da classe e o framework resolve todas elas automaticamente. Garante que a classe não seja criada sem suas dependências. [@codewithmukesh #2](#)

csharp

```
public class HomeController : Controller
{
    private readonly IMyService _service;

    public HomeController(IMyService service)
    {
        _service = service;
    }
}
```

Use o código com cuidado.

2. Injeção via Action / Método (Method Injection)

Usada quando você precisa de um serviço específico apenas para um método pontual (como uma action de Controller ou um endpoint de Minimal API), evitando inflar o construtor da classe inteira. Utiliza o atributo `[FromServices]`. [YouTube - Canal do NET #1](#)

csharp

```
public IActionResult MinhaAction([FromServices] IMyService service)
{
    var dados = service.ObterDados();
    return View(dados);
}
```

Use o código com cuidado.

3. Injeção na View (View Injection)

Ideal para quando você precisa expor um serviço de domínio ou utilitário diretamente no arquivo `.cshtml`, sem precisar passar a informação via Controller. Utiliza a diretiva `@inject`. [Macronet .net #1](#)

razor

```
@inject IMyService Service

<div>
    <p>@Service.ObterMensagemDoApi()</p>
</div>
```



Aceder diretamente o Container DI

★ Visão geral criada por IA

Para aceder diretamente ao container de Injeção de Dependência (DI) do ASP.NET Core usando o `IServiceProvider`, você pode obter a instância do provedor e utilizar o método `GetService` OU `GetRequiredService`. [Microsoft Learn +3](#)

Embora o acesso direto seja uma alternativa viável, ele é geralmente evitado (conhecido como *Service Locator anti-pattern*) em favor da injeção no construtor. [LinkedIn · Guilherme Zordan +1](#)

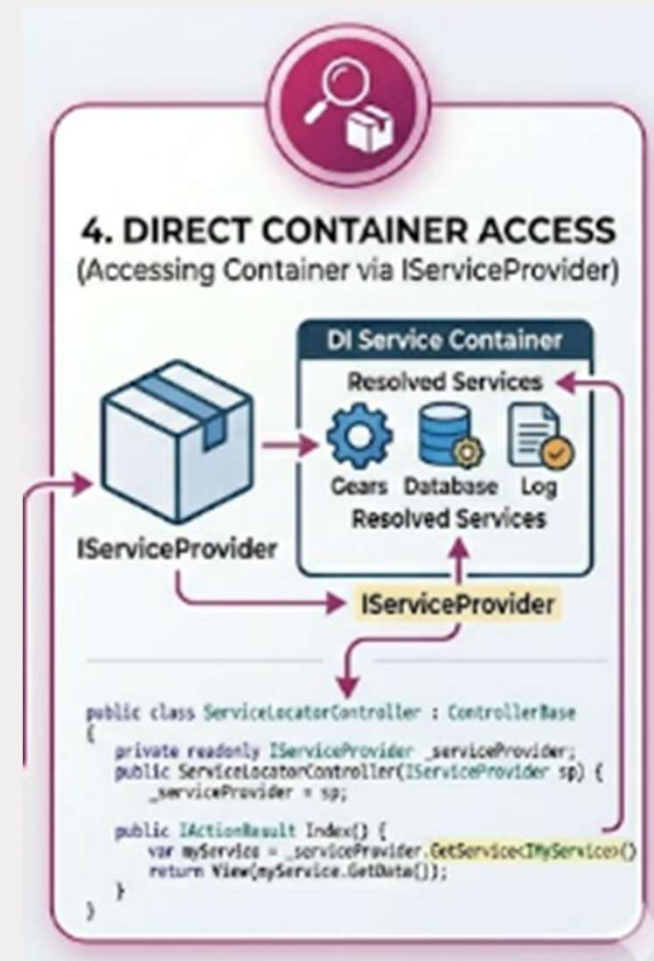
```

C:\Formacoes\04-ASP.NET-Core-MVC-Avancado\Base\ASP.NET Core MVC Avancado Main\src\WebAppMain.Mvc\bin\Debug\net10.0\W...
Instancia Singleton direto da program.cs: 8a1a3608-d90c-4beb-87fa-f1eee21dde09
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:7251
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5211
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Formacoes\04-ASP.NET-Core-MVC-Avancado\Base\ASP.NET Core MVC Avancado Main\src\WebAppMain.Mv
  
```

localhost:7251/test-di/test-direct-di

Google Google Tradutor PERSONAL

Instancia Singleton:
ae35a04a-807a-4fb5-9b86-ad5d77d8c0a0



[DEMO] - O Maravilhoso mundo das DIs



CICLOS DE VIDA DA INJEÇÃO DE DEPENDÊNCIA (DI) NO ASP.NET CORE

TRANSIENT (TRANSITÓRIO)

Instância A, Instância B, Instância C

Requisição 1: Fornece Instância A

Requisição 1 (Novamente): Fornece Instância B

Requisição 2: Fornece Instância C

A cada pedido, uma NOVA instância.

SCOPED (ESCOPODO)

Instância A

Uma Requisição HTTP

Requisição 1: Fornece Instância A

Requisição 2: Fornece Instância A (A mesma)

Uma única instância por requisição HTTP.

SINGLETON

Instância Única

App Inicia: create

Requisição 1: Fornece Instância Única

Requisição 2: Fornece Instância Única

Uma única instância para TODA a aplicação.

localhost:7110/test-di/

Primeira instância:
b7a7c74a-5028-4de5-80d7-83c0f904e1ea
c6310260-a19c-4187-9c0d-dc7364088507
d97e64b2-6074-4acc-be71-393efe03a93c

Segunda instância:
84f5adf3-5b86-47f8-99e9-4b15c024b0c8
c6310260-a19c-4187-9c0d-dc7364088507
d97e64b2-6074-4acc-be71-393efe03a93c





Módulo: Segurança

A FORTALEZA DO ASP.NET CORE

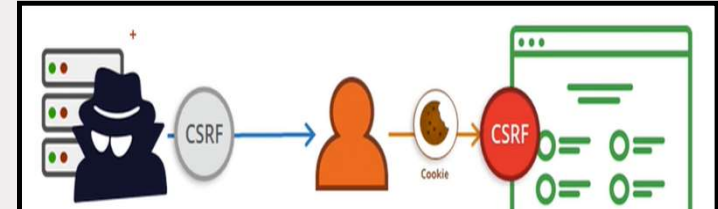
BLINDANDO A APLICAÇÃO CONTRA AMEAÇAS MODERNAS E GERENCIANDO ACESSO



CSRF (Cross-Site Request Forgery)

Visão geral criada por IA

CSRF (**Cross-Site Request Forgery** ou Falsificação de Solicitação entre Sites) é um tipo de ataque cibernético que engana o navegador de um usuário autenticado para executar ações indesejadas em outro site (como alterar senhas, e-mails ou transferir fundos) sem consentimento. Explora a confiança que um site tem nas credenciais salvas do usuário. Fortinet +3



```

1 <!DOCTYPE html>
2 <html lang="pt-br">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Veja Fotos de Lindos Cachorros!</title>
7 </head>
39 </body>
40 <div class="container">
41   <h1>Olá Amante de Pets!</h1>
42   <p>Preparamos uma coleção de fotos de lindos cães e gatos só para você. Clique no botão</p>
43   <form action="https://localhost:7110/my-products/create-new-item" method="post">
44     <input type="hidden" name="Nome" value="Você foi hackeado" />
45     <input type="hidden" name="Valor" value="50000" />
46     <input type="hidden" name="Imagem" value="Hacker-PNG-Photo-1" />
47     <input type="submit" class="button" value="Ver Fotos" />
48   </form>
49   <p>Obrigado por fazer parte da nossa comunidade pet!</p>
50 </div>
51 </body>
52 </html>

```

Olá Amante de Pets!

Preparamos uma coleção de fotos de lindos cães e gatos só para você. Clique no botão abaixo para ver estas belezinhas!

Ver Fotos

Obrigado por fazer parte da nossa

```

[HttpPost("create-new-item")]
[ValidateAntiForgeryToken]
public async Task<ActionResult> Create(Product product)
{
    if (ModelState.IsValid)
    {
        _context.Add(product);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }
    return View(product);
}

```

```

[HttpGet("create-new-item")]
0 referências
public IActionResult Create()
{
    return View();
}

[HttpPost("create-new-item")]
//[ValidateAntiForgeryToken]
0 referências
public async Task<ActionResult> Create(Product product)
{
    if (ModelState.IsValid)
    {
        _context.Add(product);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }
    return View(product);
}

```

AppSemTemplate Home Produtos Testes de DI

Meus Produtos

+ Novo

	Name	Price	Ações
	Livro CSS	100 Euros	
	Livro JQuery	120 Euros	
	Livro HTML	80 Euros	
	Livro Razor	115 Euros	
	Você foi hackeado	50000 Euros	

TOTAL DE REGISTROS: 5
Itens encontrados

Esta página não está a funcionar

Se o problema persistir, contacte o pro...

HTTP ERROR 400

```

Program.cs
AprofundandoEssencialMvc
7
8 var builder = WebApplication.CreateBuilder(args);
9
10 builder.Services.AddControllersWithViews(options =>
11 {
12     options.Filters.Add(new AutoValidateAntiforgeryTokenAttribute());
13 });
14

```



XSS (Cross-Site Scripting)

Visão geral criada por IA

Cross-site scripting (XSS) é uma vulnerabilidade de segurança web onde criminosos injetam scripts maliciosos (geralmente JavaScript) em sites legítimos. Quando usuários visitam essas páginas, o navegador executa o código, permitindo o roubo de cookies, tokens de sessão, credenciais ou o redirecionamento da vítima para sites maliciosos. Kaspersky +3

```
@{
    var inseguro = "<img src='https://t3.ftcdn.net/jpg/05/56/29/10/360_F_556291020_556291020_q2ieMiOCKYbtoLITrmt7qcSL1LJYyWrU.jpg' /> Olá!";
}

@inseguro
Olá!
```



 Olá!

Seu site ASP.NET feito do jeito certo!!



```
@{
    var inseguro = "<img src='https://t3.ftcdn.net/jpg/05/56/29/10/360_F_556291020_556291020_q2ieMiOCKYbtoLITrmt7qcSL1LJYyWrU.jpg' /> Olá!";
}

<br />
<br />
@inseguro
<br />
<br />
@Html.Raw(inseguro)

Olá!
```




Olá!

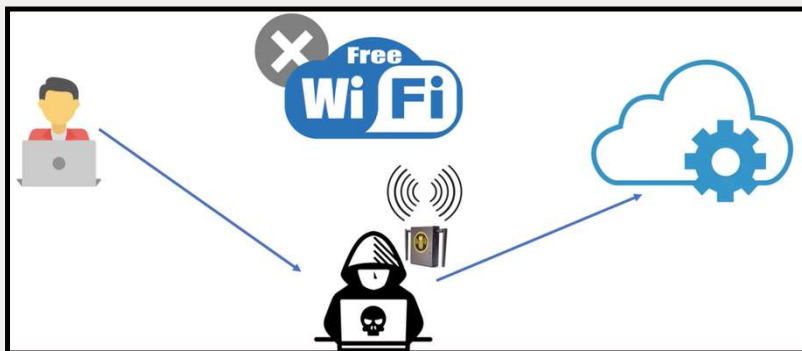
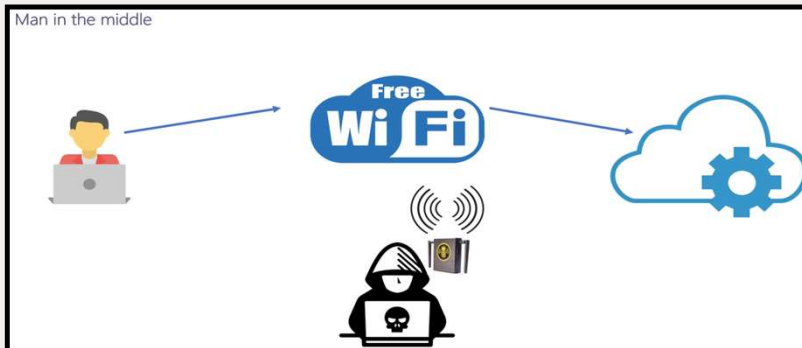
Seu site ASP.NET feito do jeito certo!!



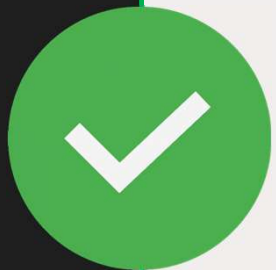
HSTS (HTTPS Strict Transport Security)

Visão geral criada por IA

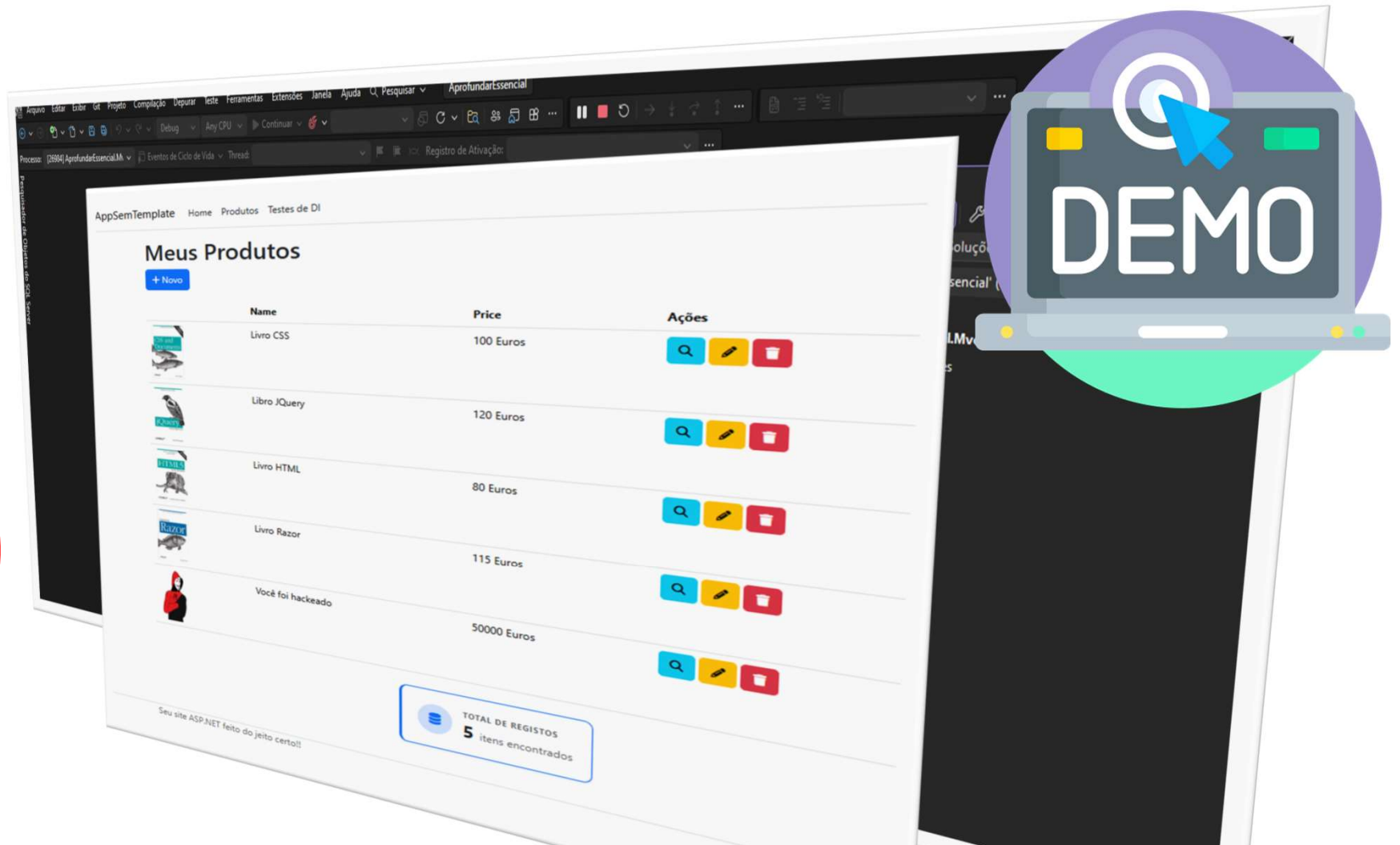
HSTS (HTTP Strict Transport Security) é uma política de segurança web que obriga os navegadores a se conectarem a um site apenas via HTTPS, eliminando conexões HTTP inseguras. Ele impede ataques de "homem no meio" (man-in-the-middle), como o *SSL stripping*, garantindo que o navegador converta automaticamente solicitações HTTP em HTTPS. [MDN Web Docs +3](#)



```
1 using AppSemTemplate.Data;
2 using Microsoft.AspNetCore.Mvc.Services;
3 using Microsoft.EntityFrameworkCore;
4
5 var builder = WebApplication.CreateBuilder(args);
6
7 builder.Services.AddControllersWithViews();
8
9 //builder.Services.AddRouting(options =>
10 //    options.ConstraintMap["slugify"] = typeof(RouteSlugifyParameterTransformer));
11
12 builder.Services.AddSingleton<IHttpContextAccessor, HttpContextAccessor>();
13
14 // Registos específicos para as interfaces específicas
15 builder.Services.AddTransient<IGenerateGuidServiceTransient, GenerateGuidService>();
16 builder.Services.AddScoped<IGenerateGuidServiceScoped, GenerateGuidService>();
17 builder.Services.AddSingleton<IGenerateGuidServiceSingleton, GenerateGuidService>();
18
19 // Registo do serviço que consome as três dependências
20 builder.Services.AddTransient<OperacaoService>();
21
22 builder.Services.AddDbContext<AppDbContext>(o =>
23     o.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
24
25 builder.Services.AddHsts(options =>
26 {
27     options.Preload = true;
28     options.IncludeSubDomains = true;
29     options.MaxAge = TimeSpan.FromDays(60);
30     options.ExcludedHosts.Add("example.com");
31     options.ExcludedHosts.Add("www.example.com");
32 });
33
34 var app = builder.Build();
35
36 if (app.Environment.IsDevelopment())
37 {
38 }
39 else
40 {
41     app.UseHsts();
42 }
43
44 app.UseStaticFiles();
45
46 app.UseHttpsRedirection();
47
```



[DEMO] - CSRF + XSS + HSTS = Segurança



Explorar ASP.NET Identity (Authentication)

```
PS > dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore
PS > dotnet add package Microsoft.AspNetCore.Identity.UI
PS > dotnet add package Microsoft.AspNetCore.Identity.UI
ou
PMC > Install-Package Microsoft.AspNetCore.Identity.EntityFrameworkCore
PMC > Install-Package Microsoft.AspNetCore.Identity.UI
PMC > Install-Package Microsoft.AspNetCore.Identity.UI
```

```
AppDbContext.cs
1 using AprofundarEssencial.Mvc.Models;
2 using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
3 using Microsoft.EntityFrameworkCore;
4
5 namespace AppSemTemplate.Data
6
7 public class AppDbContext : IdentityDbContext<DbContext> //DbContext
8 {
9
10     public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
11     {
12     }
13
14     public DbSet<DbContext> DbContext { get; set; }
15 }
16
```

```
Program.cs
34
35 builder.Services.AddDefaultIdentity<IdentityUser>(options =>
36 {
37     options.SignIn.RequireConfirmedAccount = true;
38     options.AddEntityFrameworkStores<AppDbContext>();
39 })
40
41 builder.Build();
42
43 if (app.Environment.IsDevelopment())
44 {
45     app.UseDeveloperExceptionPage();
46 }
47
48 app.UseHttpsRedirection();
49
50 app.UseStaticFiles();
51
52 app.UseRouting();
53
54 app.UseAuthorization();
55
56 //app.MapControllerRoute(
57 //    name: "default",
58 //    pattern: "{controller:slugify=Home}/{action:slugify=Index}/{id?}"
59 //);
60
61 app.MapControllerRoute(
62     name: "default",
63     pattern: "{controller=Home}/{action=Index}/{id?}"
64 );
65
66 app.MapRazorPages();
67
68 app.Run();
69
```

```
PS > dotnet ef migrations add Initial
PMC > Add-Migrations Initial
```

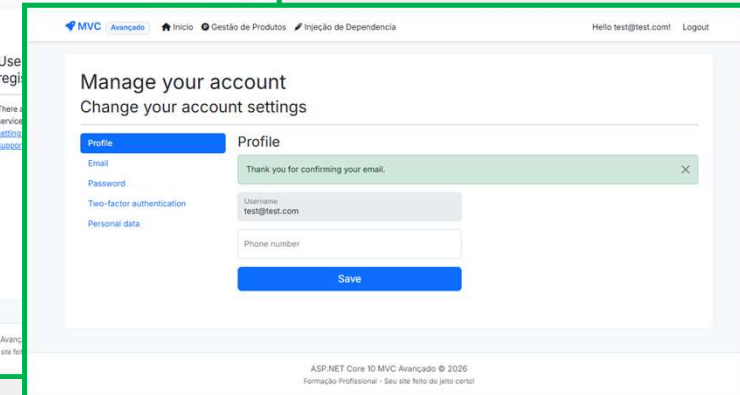
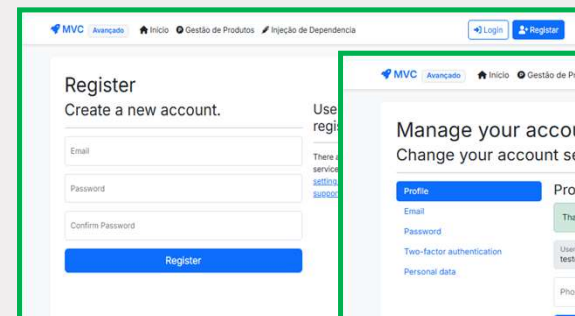
```
PS > dotnet ef database update
PMC > Update-Database
```



```
_LoginPartial.cshtml
1 @using Microsoft.AspNetCore.Identity
2 @inject SignInManager<IdentityUser> SignInManager
3 @inject UserManager<IdentityUser> UserManager
4
5 <ul class="navbar-nav">
6     @if (SignInManager.IsSignedIn(User))
7     {
8         <li class="nav-item">
9             <a class="nav-link text-dark" asp-area="Identity" asp-page="/Account/Manage/Index" title="Manage">Manage</a>
10         </li>
11     }
12     else
13     {
14         <li class="nav-item">
15             <a class="nav-link text-dark" asp-area="Identity" asp-page="/Account/Login" title="Login">Login</a>
16         </li>
17     }
18 </ul>
19
```

```
_Layout.cshtml
16 </head>
17 <body>
18     <div class="navbar navbar-expand-sm navbar-toggleable-sm" style="background-color: #f8f9fa; padding: 5px 0 0 0;">
19         <div class="container-fluid">
20             <div class="navbar-brand" asp-area="" asp-page="/Index">AprofundarEssencial.Mvc</div>
21             <div class="navbar-collapse collapse d-sm-inline-block">
22                 <ul class="navbar-nav flex-grow-1">
23                     <li class="nav-item">
24                         <a class="nav-link text-dark" href="/Index">Home</a>
25                     </li>
26                     <li class="nav-item">
27                         <a class="nav-link text-dark" href="/Account/Login">Login</a>
28                     </li>
29                     <li class="nav-item">
30                         <a class="nav-link text-dark" href="/Account/Manage/Index">Manage</a>
31                     </li>
32                     <li class="nav-item">
33                         <a class="nav-link text-dark" href="/Account/Logout">Logout</a>
34                     </li>
35                 </ul>
36             </div>
37         </div>
38     </div>
39
```

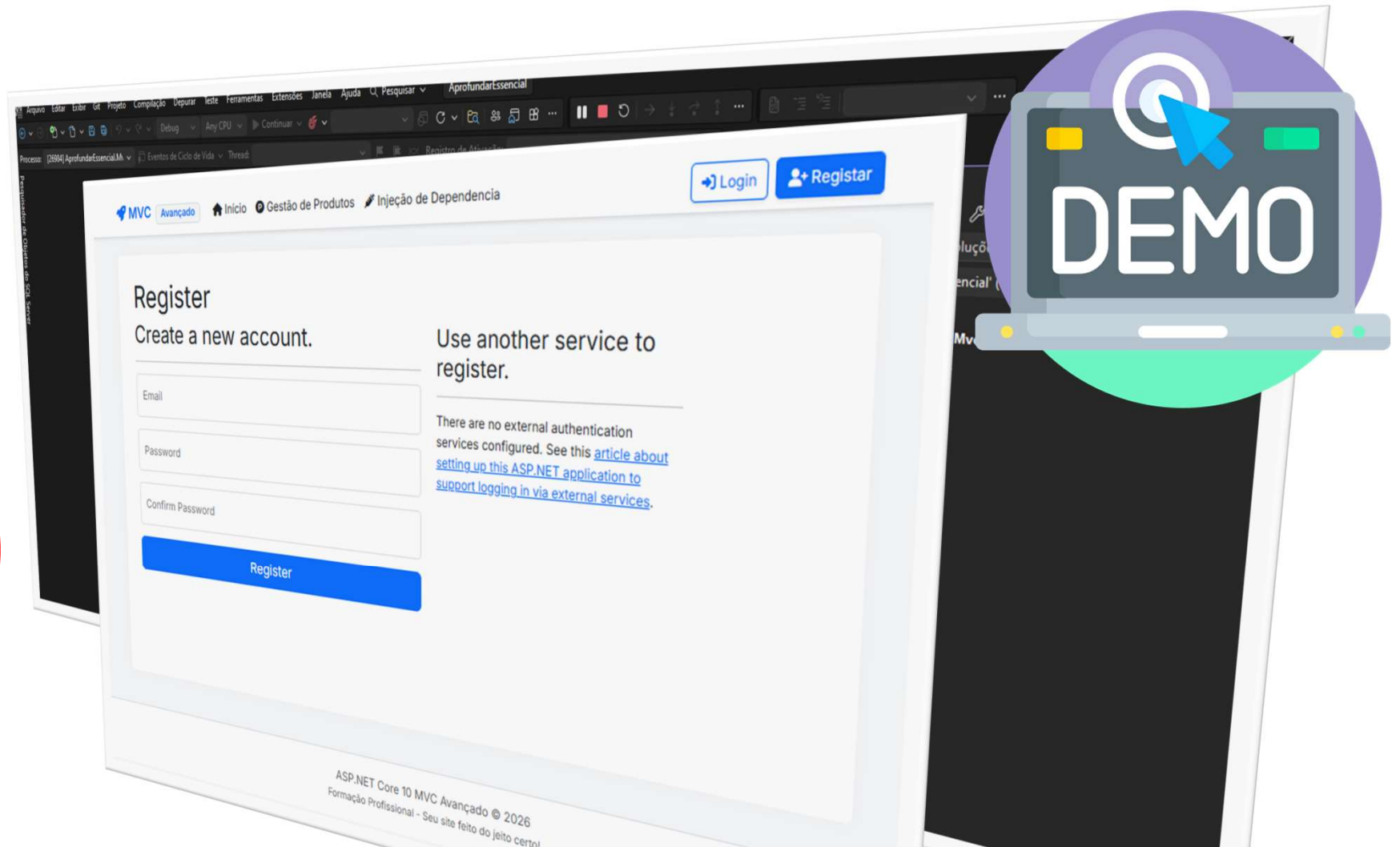
```
_ViewStart.cshtml
1
2 @{
3     Layout = "/Views/Shared/_Layout.cshtml";
4 }
5
```



```
ProductsController.cs
17
18 public async Task<ActionResult> Index()
19 {
20     var user = HttpContext.User.Identity.IsAuthenticated ? HttpContext.User.Identity.Name : null;
21     return View(user);
22 }
23
24 public async Task<ActionResult> Details(int id)
25 {
26     var user = HttpContext.User.Identity.IsAuthenticated ? HttpContext.User.Identity.Name : null;
27     return View(user);
28 }
29
```



[DEMO] - Identity | Authentication



Explorar o Identity | Authorization | Roles

Visão geral criada por IA

Roles (papéis) no ASP.NET Identity são conjuntos de permissões atribuídas a usuários para controlar o acesso a recursos específicos em uma aplicação web. Elas funcionam como agrupamentos, permitindo definir "quem" pode fazer "o quê" (ex: "Admin", "Usuário", "Gerente"), utilizando o atributo `[Authorize(Roles = "Admin")]` para proteger controladores ou actions. [YouTube +3](#)

```
ProductsController.cs
namespace AprofundarEssencial.Mvc.Controllers
{
    [Authorize]
    [Route("my-products")]
    public class ProductsController : Controller
    {
        private readonly ApplicationDbContext _context;

        public ProductsController(ApplicationDbContext context)
        {
            _context = context;
        }

        [AllowAnonymous]
        public async Task<IActionResult> Index()
        {
            var user = HttpContext.User;

            return _context.Products
                .Where(p => p.UserId == user.Id)
                .ToListAsync()
                .Problem();
        }

        [Route("details/{id}")]
        public async Task<IActionResult> Details(int id)
        {
            if (id == null || _context.Products.FirstOrDefault(p => p.Id == id) == null)
            {
                return NotFound();
            }

            var produto = await _context.Products
                .FirstOrDefaultAsync(p => p.Id == id);
            if (produto == null)
            {
                return NotFound();
            }

            return View(produto);
        }
    }
}
```

```
ProductsController.cs
using AppSemTemplate.Data;
using AprofundarEssencial.Mvc.Models;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace AprofundarEssencial.Mvc.Controllers
{
    [Authorize(Roles = "Admin")]
    [Route("my-products")]
    public class ProductsController : Controller
    {
        private readonly ApplicationDbContext _context;

        public ProductsController(ApplicationDbContext context)
        {
            _context = context;
        }

        [AllowAnonymous]
        public async Task<IActionResult> Index()
        {
            var user = HttpContext.User.Identity;

            return _context.Products != null ?
                View(await _context.Products.ToListAsync()) :
                Problem("Entity set 'AppDbContext.Products' is null.");
        }
    }
}
```

```
Program.cs
builder.Services.AddDbContext<AppDbContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
builder.Services.AddDefaultIdentity<IdentityUser>(o =>
    o.SignIn.RequireConfirmedAccount = true)
    .AddRoles<IdentityRole>()
    .AddEntityFrameworkStores<AppDbContext>();
```

Log in

Use a local account to log in.

Use another service to log in.

Email: test@test.com

Password:

Remember me!

Log in

Forgot your password?

Register as a new user

Reset email confirmation

Acesso Proibido

Você não tem acesso a este recurso.

Id	UserName	NormalizedUserName	Email
10bb90bb-7b9c-4da8-8854-eb5f288cddfb	test@test.com	TEST@TEST.COM	test@test.com
NULL			

Id	Name	NormalizedName	ConcurrencyStamp
1	Admin	ADMIN	NULL
NULL			

UserId	RoleId
10bb90bb-7b9c-4da8-8854-eb5f288cddfb	1
NULL	NULL

Edit

Produto

Name: Livro CSS

Image: CSS.jpg

Price: 100

Save

Back to List



Authorization | Policies Required Roles

Visão geral criada por IA

No ASP.NET Core Identity, **policies que requerem roles** são regras de autorização baseadas em políticas (Policy-Based Authorization) que exigem que o usuário autenticado pertença a uma ou mais **Roles** (perfis/papéis) específicas para acessar um recurso. Elas oferecem mais flexibilidade e desacoplamento do que verificar roles diretamente na Controller. [YouTube +2](#)

```
ProductsController.cs*
AprofundarEssencial.Mvc
115
116 [Authorize(Policy = "CanDelete")]
117 [HttpPost("delete-item/{id}"), ActionName("Delete")]
118 [ValidateAntiForgeryToken]
119 public async Task<IActionResult> DeleteConfirmed(int id)
120 {
121     if (_context.Products == null)
122     {
123         return Problem("Entity set 'AppDbContext.Products' is null.");
124     }
125     var product = await _context.Products.FindAsync(id);
126     if (product != null)
127     {
128         _context.Products.Remove(product);
129     }
130
131     await _context.SaveChangesAsync();
132     return RedirectToAction(nameof(Index));
133 }
```

```
Program.cs*
AprofundarEssencial.Mvc
41
42 builder.Services.AddAuthorization(options =>
43 {
44     options.AddPolicy("CanDelete", policy =>
45         policy.RequireRole("Admin"));
46 });
47
48 var app = builder.Build();
```

MVC Avançado Início Gestão de Produtos Injeção de Dependência Hello t

Meus Produtos

+ Novo

Name	Price	Ações
Livro CSS	100 Euros	🔍 ✎ 🗑️
Livro JQuery	120 Euros	🔍 ✎ 🗑️

MVC Avançado Início Gestão de Produtos Injeção de Dependência Hello test@test.com! Logout

Acesso Proibido

Você não tem acesso a este recurso.

MVC Avançado Início Gestão de Produtos Injeção de Dependência

Delete

Are you sure you want to delete this?

Produto

Name	Você foi hackeado
Image	Hacker-PNG-Photo-Image.png
Price	50000

[Delete](#) | [Back to List](#)



Authorization | Policies Required Claims

Visão geral criada por IA

No ASP.NET Core Identity, **políticas que requerem claims** (declarações) são regras de autorização baseadas em atributos específicos do usuário, indo além de simples funções (roles). Elas verificam se o usuário possui uma declaração (par chave-valor, ex: Idade: 21 OU Departamento: TI) antes de liberar o acesso a um recurso. [Microsoft Learn +3](#)

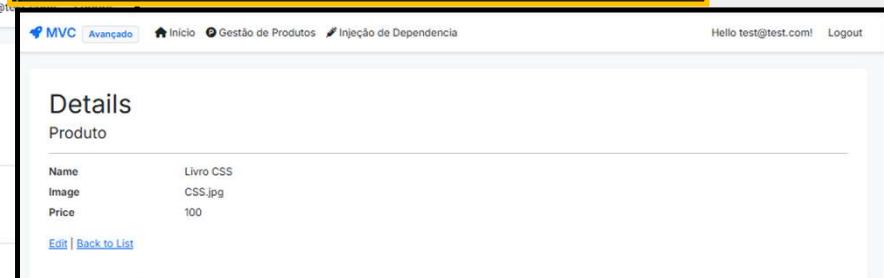
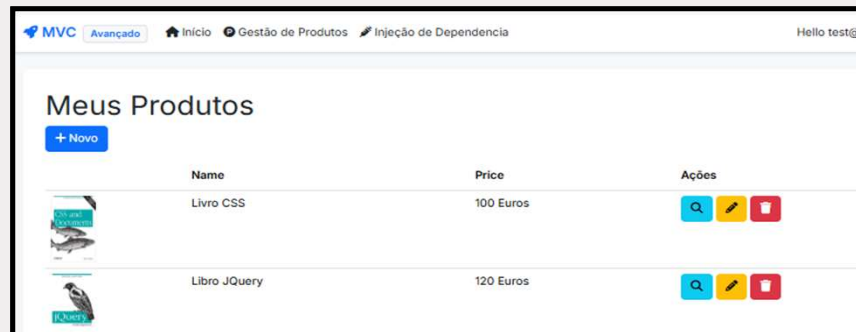
```
Program.cs
AprofundarEssencial.Mvc
41
42 builder.Services.AddAuthorization(options =>
43 {
44     options.AddPolicy("CanDeleteProduct", policy =>
45         policy.RequireRole("Admin", "Manager", "Seller"));
46
47     options.AddPolicy("ViewProduct", policy =>
48         policy.RequireClaim("Product", "View"));
49 });
50
51 var app = builder.Build();
52
```

```
ProductsController.cs
AprofundarEssencial.Mvc
8
9 [Authorize(Roles = "Admin")]
10 [Route("my-products")]
11 public class ProductsController : Controller
12 {
13     private readonly ApplicationDbContext _context;
14
15     0 referências
16     public ProductsController(ApplicationDbContext context)
17     {
18         _context = context;
19     }
20
21     //[[AllowAnonymous]]
22     [Authorize(Policy = "ViewProduct")]
23     public async Task<IActionResult> Index()
24     {
25         var user = HttpContext.User.Identity;
26
27         return _context.Products != null ?
28             View(await _context.Products.ToListAsync()) :
29             Problem("Entity set 'AppDbContext.Products' is null");
30     }
31
32     [Authorize(Policy = "ViewProduct")]
33     [Route("details/{id}")]
34     public async Task<IActionResult> Details(int? id)
35     {
36         if (id == null || _context.Products == null) return NotFound();
37     }
38 }
39
```

dbo.AspNetU...aims [Dados]

Máx. de Linhas: 1000

	Id	UserId	ClaimType	ClaimValue
	1	10bb90bb-7b9c-4da8-8854-eb5f288cddfb	Product	View
	NULL	NULL	NULL	NULL



Custom Authorization

```

ProductsController.cs
AprofundarEssencial.Mvc

43
44 [ClaimsAuthorize(ClaimName: "Product", claimValue: "Create")]
45 [HttpPost("create-new-item")]
46 public IActionResult Create()
47 {
48     return View();
49 }
50
51 [ClaimsAuthorize(ClaimName: "Product", claimValue: "Create")]
52 [HttpPost("create-new-item")]
53 [ValidateAntiForgeryToken]
54 public async Task<IActionResult> Create(Product product)
55 {
56     if (ModelState.IsValid)
57     {
58         _context.Add(product);
59         await _context.SaveChangesAsync();
60         return RedirectToAction(nameof(Index));
61     }
62     return View(product);
63 }
64
65 [ClaimsAuthorize(ClaimName: "Product", claimValue: "Edit")]
66 [Route("edit-item/{id}")]
67 public async Task<IActionResult> Edit(int? id)
68 {
69     if (id == null || _context.Products == null) return NotFound();
70     var product = await _context.Products.FindAsync(id);
71     if (product == null) return NotFound();
72     return View(product);
73 }
74
75 [ClaimsAuthorize(ClaimName: "Product", claimValue: "Edit")]
76 [HttpPost("edit-item/{id}")]
77 [ValidateAntiForgeryToken]
78 public async Task<IActionResult> Edit(int id, Product product)
79 {
80     if (id != product.Id) return NotFound();
81     if (ModelState.IsValid)
82     {
83         try
84         {
85             _context.Update(product);
86             await _context.SaveChangesAsync();
87         }
88         catch (DbUpdateConcurrencyException)
89         {
90             if (!ProductExists(product.Id))
91                 return NotFound();
92             else
93                 throw;
94         }
95     }
96     return RedirectToAction(nameof(Index));
97 }
98
99
100

```

```

ClaimsAuthorizationExtensions.cs
AprofundarEssencial.Mvc

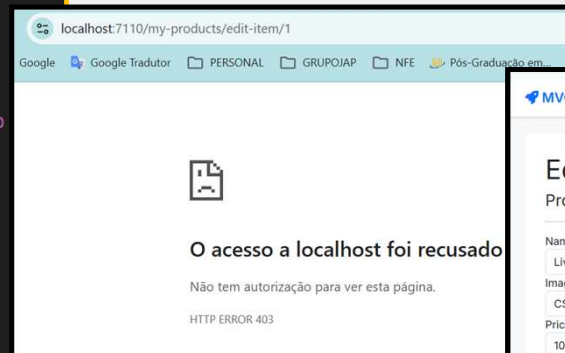
1 namespace AprofundarEssencial.Mvc.Extensions
2 {
3     public class CustomAuthorization
4     {
5         public static bool ValidateUserClaims(HttpContext context, string claimName, string claimValue)
6         {
7             if (context.User.Identity == null) throw new InvalidOperationException();
8             return context.User.Identity.IsAuthenticated &&
9                 context.User.Claims.Any(c => c.Type == claimName && c.Value.Split(',').Contains(claimValue));
10        }
11    }
12
13    public class RequirementClaimFilter : IAuthorizationFilter
14    {
15        private readonly Claim _claim;
16
17        public RequirementClaimFilter(Claim claim)
18        {
19            _claim = claim;
20        }
21
22        public void OnAuthorization(AuthorizationFilterContext context)
23        {
24            if (context.HttpContext.User.Identity == null) throw new InvalidOperationException();
25            if (!context.HttpContext.User.Identity.IsAuthenticated)
26            {
27                context.Result = new RedirectToRouteResult(new RouteValueDictionary { { "area", "Identity", "page", "Account/Login" } });
28                return;
29            }
30            if (!CustomAuthorization.ValidateUserClaims(context.HttpContext, _claim.Type, _claim.Value))
31            {
32                context.Result = new StatusCodeResult(403);
33            }
34        }
35    }
36
37    public class ClaimsAuthorizeAttribute : TypeFilterAttribute
38    {
39        public ClaimsAuthorizeAttribute(string claimName, string claimValue) : base(typeof(RequirementClaimFilter))
40        {
41            Arguments = new object[] { new Claim(claimName, claimValue) };
42        }
43    }
44 }

```

dbo.AspNetUsers [Dados]

Máx. de Linhas: 1000

	Id	UserId	ClaimType	ClaimValue
	1	10bb90bb-7b9...	Product	View
	2	10bb90bb-7b9...	Product	Create
	3	10bb90bb-7b9...	Product	Edit
	NULL	NULL	NULL	NULL



MVC Avançado

Inicio Gestão de Produtos Injeção de Dependência

Edit

Produto

Name

Livro CSS

Image

CSS.jpg

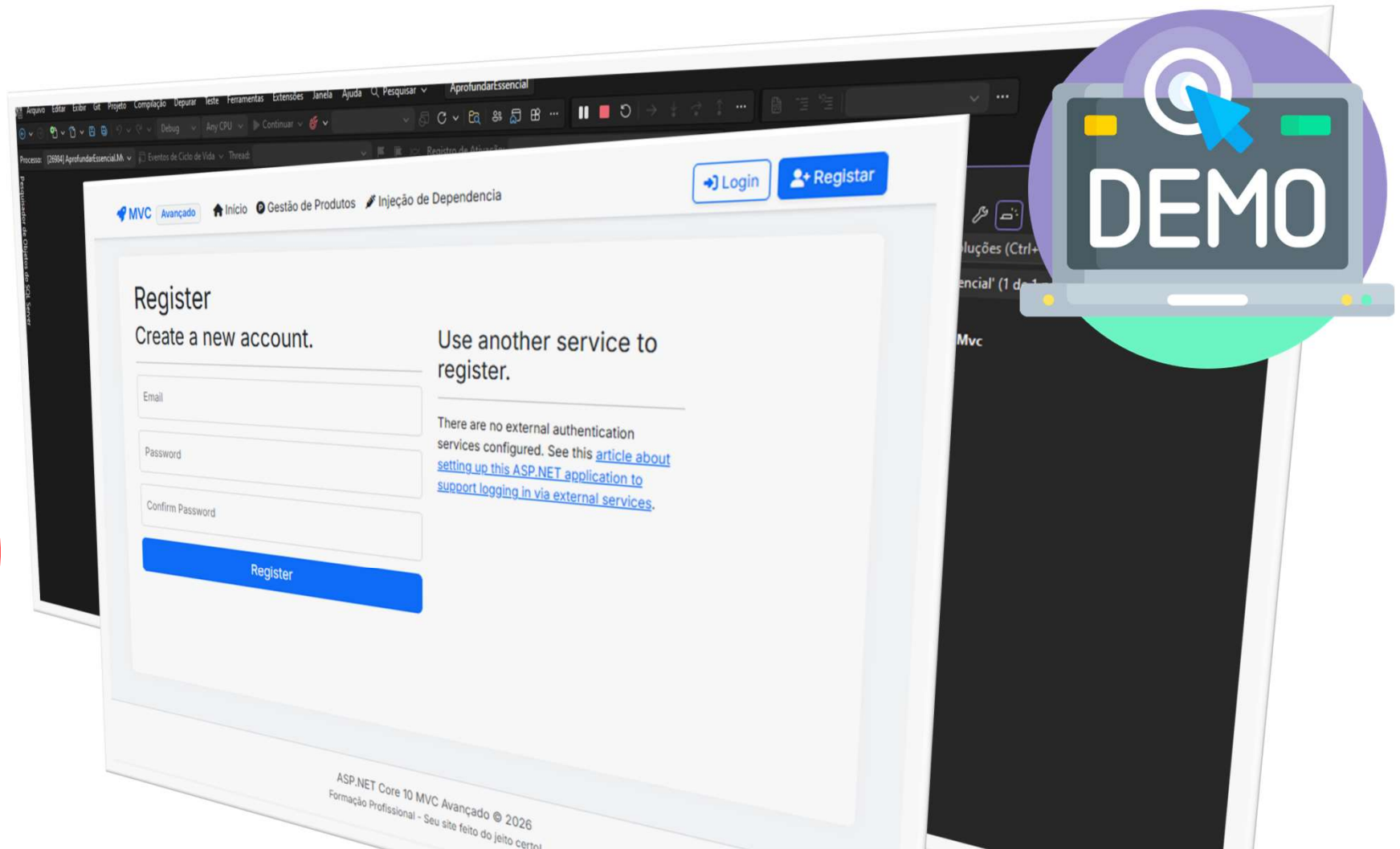
Price

100

Save

Back to List

[DEMO] - Identity | Authorization





Módulo: Configurações Avançadas

DOMINANDO O PIPELINE E O AMBIENTE DO ASP.NET CORE



Organizar a classe program.cs

Visão geral criada por IA

Extension Methods (Métodos de Extensão) em C# permitem adicionar novos métodos a tipos existentes (classes, structs ou interfaces) sem modificar o código-fonte original, herdar ou recompilar o tipo. Eles são definidos como métodos estáticos dentro de uma classe estática, usando a palavra-chave `this` no primeiro parâmetro para indicar qual tipo está sendo estendido. [Microsoft Learn +3](#)

```
Program.cs
1 using AprofundarEssencial.Mvc.Configurations;
2
3 var builder = WebApplication.CreateBuilder(args);
4
5 builder
6     .AddMvcConfiguration()
7     .AddIdentityConfiguration()
8     .AddDependencyInjectionConfiguration();
9
10 var app = builder.Build();
11
12 app.UseMvcConfiguration();
13
14 app.Run();
15
```

```
MvcConfig.cs
1 using Microsoft.AspNetCore.Mvc;
2
3 namespace AprofundarEssencial.Mvc.Configurations
4 {
5     public static class MvcConfig
6     {
7         // Referência
8         public static WebApplicationBuilder AddMvcConfiguration(this WebApplicationBuilder builder)
9         {
10             builder.Services.AddControllersWithViews(options =>
11             {
12                 options.Filters.Add(new AutoValidateAntiforgeryTokenAttribute());
13             });
14
15             // Builder.Services.AddRouting(options =>
16             // {
17             //     options.ConstraintMap["slugify"] = typeof(RouteSlugifyParameterTransformer);
18             // });
19
20             builder.Services.AddHttpsOptions(options =>
21             {
22                 options.Preload = true;
23                 options.IncludeSubDomains = true;
24                 options.MaxAge = TimeSpan.FromDays(60);
25                 options.ExcludedHosts.Add("example.com");
26                 options.ExcludedHosts.Add("www.example.com");
27             });
28
29             return builder;
30         }
31
32         // Referência
33         public static WebApplication UseMvcConfiguration(this WebApplication app)
34         {
35             if (app.Environment.IsDevelopment())
36             {
37                 app.UseDeveloperExceptionPage();
38             }
39             else
40             {
41                 app.UseExceptionHandler("/Error");
42                 app.UseHsts();
43             }
44
45             app.UseHttpsRedirection();
46             app.UseStaticFiles();
47             app.UseRouting();
48             app.UseAuthorization();
49
50             // app.MapControllerRoute(
51             //     name: "default",
52             //     pattern: "{controller:slugify}/{action:slugify}/{id?}");
53
54             app.MapControllerRoute(
55                 name: "default",
56                 pattern: "{controller:home}/{action:index}/{id?}"
57                 WithStaticAssets());
58
59             app.MapRazorPages();
60
61             return app;
62         }
63     }
64 }
```

```
DIConfig.cs
1 using AppSemTemplate.Data;
2 using AprofundarEssencial.Mvc.Services;
3 using Microsoft.EntityFrameworkCore;
4
5 namespace AprofundarEssencial.Mvc.Configurations
6 {
7     // 0 referências
8     public static class DIConfig
9     {
10         // 1 referência
11         public static WebApplicationBuilder AddDependencyInjectionConfiguration(this WebApplicationBuilder builder)
12         {
13             builder.Services.AddSingleton<HttpContextAccessor, HttpContextAccessor>();
14
15             // Registos específicos para as interfaces específicas
16             builder.Services.AddTransient<IGenerador>();
17             builder.Services.AddScoped<IGerador>();
18             builder.Services.AddSingleton<IGerador>();
19
20             // Registo do serviço que consome as
21             builder.Services.AddDbContext<AppDbContext>
22                 .UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"));
23
24             return builder;
25         }
26     }
27 }
```

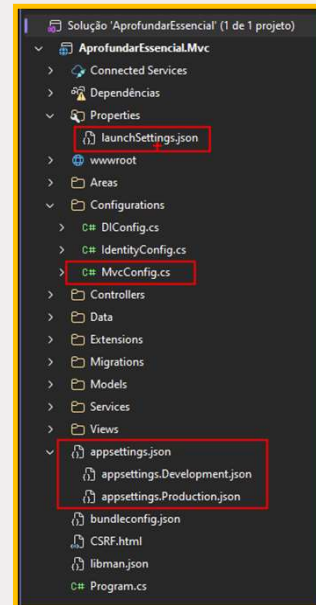
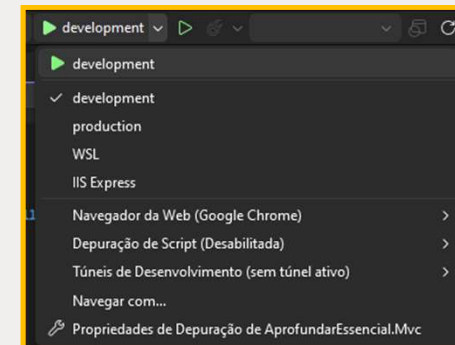
```
IdentityConfig.cs
1 using AppSemTemplate.Data;
2 using Microsoft.AspNetCore.Identity;
3
4 namespace AprofundarEssencial.Mvc.Configurations
5 {
6     // 0 referências
7     public static class IdentityConfig
8     {
9         // 1 referência
10         public static WebApplicationBuilder AddIdentityConfiguration(this WebApplicationBuilder builder)
11         {
12             builder.Services.AddDefaultIdentity<IdentityUser>(options =>
13             {
14                 options.SignIn.RequireConfirmedAccount = true;
15                 options.AddRoles<IdentityRole>();
16                 options.AddEntityFrameworkStores<AppDbContext>();
17             });
18
19             builder.Services.AddAuthorization(options =>
20             {
21                 options.AddPolicy("CanDeleteProduct", policy =>
22                     policy.RequireRole("Admin", "Manager", "Seller"));
23
24                 options.AddPolicy("ViewProduct", policy =>
25                     policy.RequireClaim("Product", "View"));
26             });
27
28             return builder;
29         }
30     }
31 }
```



Configurar Ambientes

```

MvcConfig.cs
1 using Microsoft.AspNetCore.Mvc;
2
3 namespace AprofundarEssencial.Mvc.Configurations
4 {
5     public static class MvcConfig
6     {
7         public static void AddMvcConfiguration(this WebApplicationBuilder builder)
8         {
9             builder.Configuration
10                .SetBasePath(builder.Environment.ContentRootPath)
11                .AddJsonFile("appsettings.json", true, true)
12                .AddJsonFile($"appsettings.{builder.Environment.EnvironmentName}.json", true, true)
13                .AddEnvironmentVariables();
14
15             builder.Services.AddControllersWithViews(options =>
16             {
17                 options.Filters.Add(new AutoValidateAntiforgeryTokenAttribute());
18             });
19         }
20     }
21 }
    
```



```

appsettings.json
Esquema: https://www.schemastore.org/appsettings.json
1 {
2   "Logging": {
3     "LogLevel": {
4       "Default": "Information",
5       "Microsoft.AspNetCore": "Warning"
6     }
7   },
8   "AllowedHosts": "*"
9 }
    
```

```

appsettings...loment.json
Esquema: https://www.schemastore.org/appsettings.json
1 {
2   "ConnectionStrings": {
3     "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=OnboardingDotNetDb;Trusted_Connection=True;MultipleActiveResultSets=true"
4   }
5 }
    
```

```

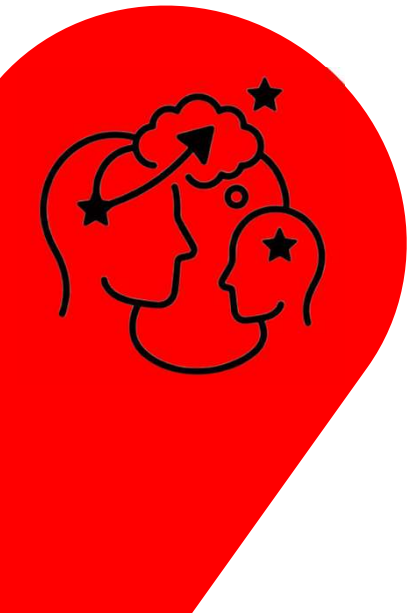
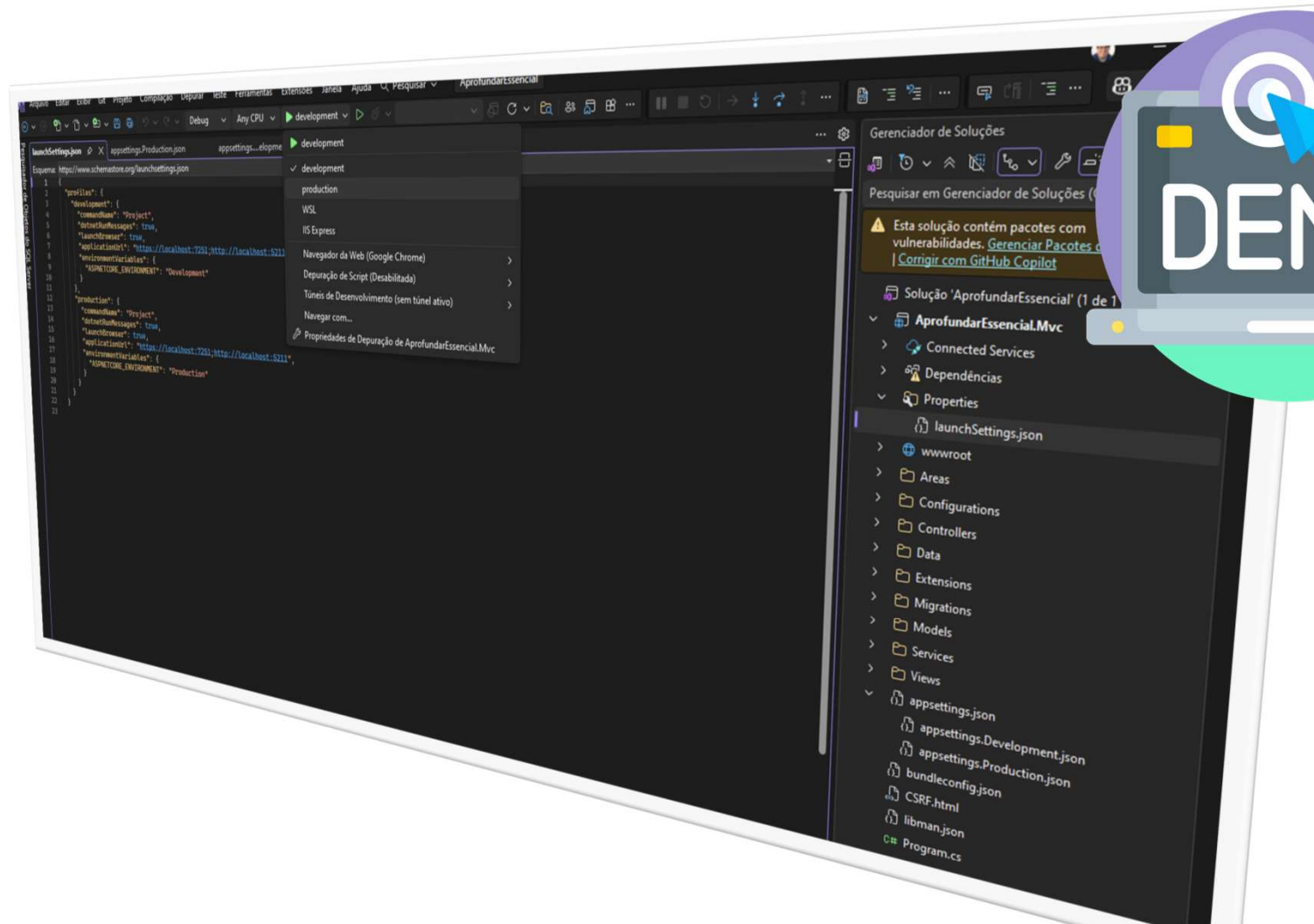
appsettings.Production.json
Esquema: https://www.schemastore.org/appsettings.json
1 {
2   "ConnectionStrings": {
3     "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=PROD_OnboardingDotNetDb;Trusted_Connection=True;MultipleActiveResultSets=true"
4   }
5 }
    
```

```

launchSettings.json
Esquema: https://www.schemastore.org/launchsettings.json
1 {
2   "profiles": {
3     "development": {
4       "commandName": "Project",
5       "dotnetRunMessages": true,
6       "launchBrowser": true,
7       "applicationUrl": "https://localhost:7251;http://localhost:5211",
8       "environmentVariables": {
9         "ASPNETCORE_ENVIRONMENT": "Development"
10      }
11    },
12    "production": {
13      "commandName": "Project",
14      "dotnetRunMessages": true,
15      "launchBrowser": true,
16      "applicationUrl": "https://localhost:7251;http://localhost:5211",
17      "environmentVariables": {
18        "ASPNETCORE_ENVIRONMENT": "Production"
19      }
20    }
21  }
22 }
    
```



[DEMO] - Organizar e Configurar



Ler ficheiro de configuração

```

HomeController.cs
AprofundarEssencial.Mvc
2 using Microsoft.AspNetCore.Mvc;
3 using Microsoft.Extensions.Options;
4 namespace AprofundarEssencial.Mvc.Controllers
5 {
6     1 referência
7     public class HomeController : Controller
8     {
9         private readonly IConfiguration Configuration;
10        private readonly ApiConfiguration ApiConfig;
11
12        0 referências
13        public HomeController(IConfiguration configuration,
14                             IOption<ApiConfiguration> apiConfiguration)
15        {
16            Configuration = configuration;
17            ApiConfig = apiConfiguration.Value;
18        }
19
20        0 referências
21        public IActionResult Index()
22        {
23            var env = Environment.GetEnvironmentVariable("ASPNETCORE_ENVIRONMENT");
24
25            // Access the configuration values directly from IConfiguration
26            var myDomain_directly = Configuration[$"{ApiConfiguration.ConfigName}:Domain"] ?? "DefaultDomain";
27            var userKey2_directly = Configuration[$"{ApiConfiguration.ConfigName}:UserKey"] ?? "DefaultUserKey";
28            var userSecret2_directly = Configuration[$"{ApiConfiguration.ConfigName}:UserSecret"] ?? "DefaultUserSecret";
29
30            // Access the configuration values
31            var apiConfig = new ApiConfiguration();
32            Configuration.GetSection(ApiConfiguration.ConfigName).Bind(apiConfig);
33
34            var myDomain = apiConfig.Domain ?? "DefaultDomain";
35            var userKey = apiConfig.UserKey ?? "DefaultUserKey";
36            var userSecret = apiConfig.UserSecret ?? "DefaultUserSecret";
37
38            return View();
39        }
40    }
41

```

```

ApiConfiguration.cs
AprofundarEssencial.Mvc
1 namespace AprofundarEssencial.Mvc.Configurations
2 {
3     7 referências
4     public class ApiConfiguration
5     {
6         public const string ConfigName = "ApiConfiguration";
7
8         1 referência
9         public string? Domain { get; set; }
10        1 referência
11        public string? UserKey { get; set; }
12        1 referência
13        public string? UserSecret { get; set; }
14    }
15

```

```

appsettings...lopmment.json
Esquema: https://www.schemastore.org/appsettings.json
1 {
2     "ConnectionStrings": {
3         "DefaultConnection": "Server=(localdb)\\mssqllocal
4     },
5     "ApiConfiguration": {
6         "Domain": "https://dominio.com",
7         "UserKey": "MINHACHAVESEGREDO-DEV",
8         "UserSecret": "MEUSEGREDOAAPI-DEV"
9     }
10 }

```

```

0 referências
public IActionResult Index()
{
    var env = Environment.GetEnvironmentVariable("ASPNETCORE_ENVIRONMENT");

    // Access the configuration values directly from IConfiguration
    var myDomain_directly = Configuration[$"{ApiConfiguration.ConfigName}:Domain"] ?? "DefaultDomain";
    var userKey2_directly = Configuration[$"{ApiConfiguration.ConfigName}:UserKey"] ?? "DefaultUserKey";
    var userSecret2_directly = Configuration[$"{ApiConfiguration.ConfigName}:UserSecret"] ?? "DefaultUserSecret";

    // Access the configuration values
    var apiConfig = new ApiConfiguration();
    Configuration.Bind(apiConfig, (AprofundarEssencial.Mvc.Configurations.ApiConfiguration));

    var myDomain = apiConfig.Domain;
    var userKey = apiConfig.UserKey;
    var userSecret = apiConfig.UserSecret;

    return View();
}

```



```

0 referencias
public ActionResult Index()
{
    var env = Environment.GetEnvironmentVariable("ASPNETCORE_ENVIRONMENT");

    // Access the configuration values directly from IConfiguration
    var myDomain_directly = Configuration[($"{ApiConfiguration.ConfigName}:Domain")} ?? "DefaultDomain";
    var userKey2_directly = Configuration[($"{ApiConfiguration.ConfigName}:UserKey")} ?? "DefaultUserKey";
    var userSecret2_directly = Configuration[($"{ApiConfiguration.ConfigName}:UserSecret")} ?? "DefaultUserSecret";

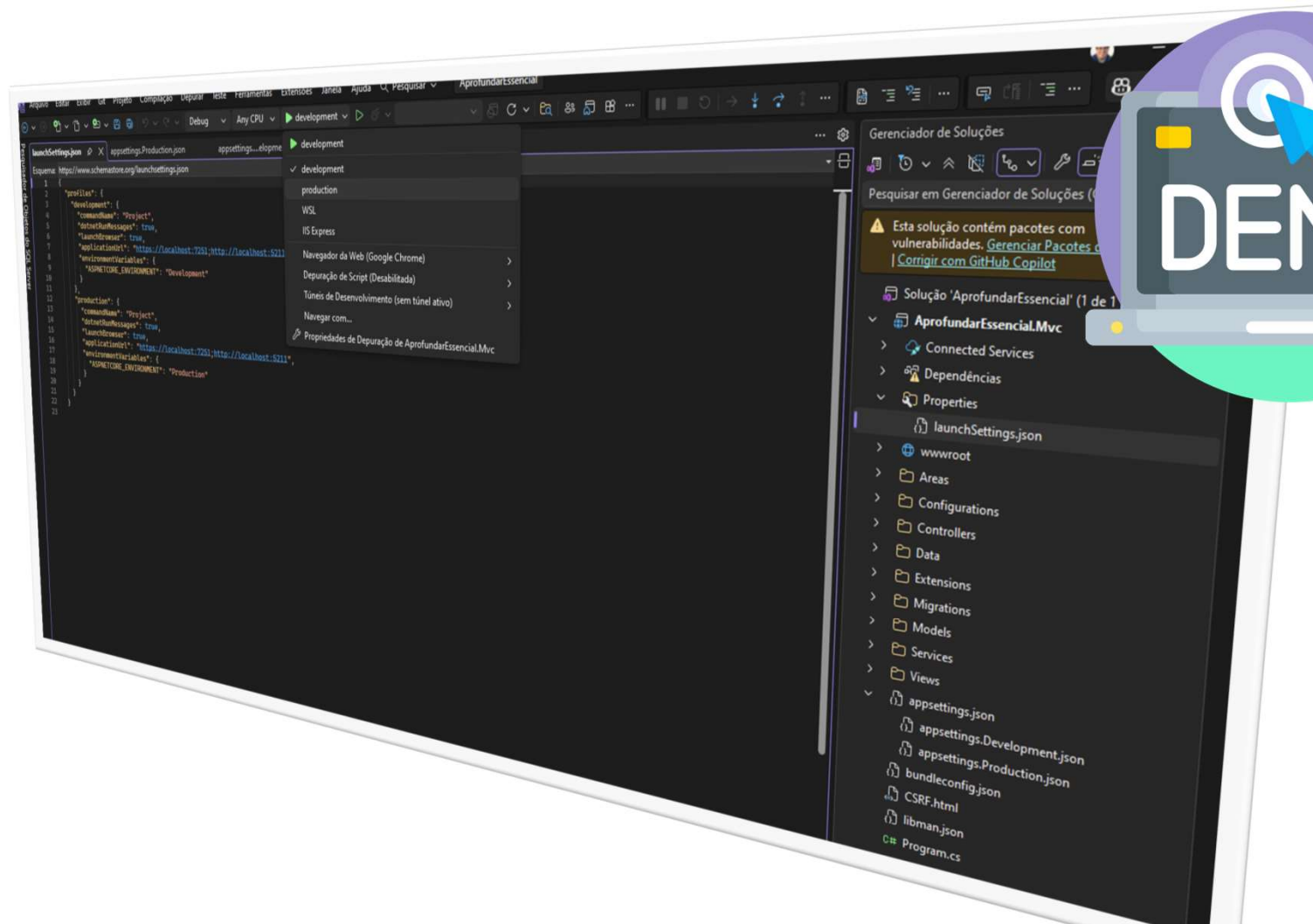
    // Access the configuration values
    var apiConfig = new ApiConfiguration();
    Configuration.GetSection(ApiConfiguration.ConfigName).Bind(apiConfig);

    var myDomain = apiConfig.Domain ?? "DefaultDomain";
    var userKey = apiConfig.UserKey ?? "DefaultUserKey";
    var userSecret = apiConfig.UserSecret ?? "DefaultUserSecret";

    return View();
}

```

[DEMO] - Ler ficheiro e User Secrets



Tratamento de Erros e Middlewares

```

MvcConfig.cs
AprofundarEssencial.Mvc

1 referência
public static WebApplication UseMvcConfiguration(this WebApplication
{
    if (app.Environment.IsDevelopment())
    {
        app.UseDeveloperExceptionPage();
    }
    else
    {
        app.UseExceptionHandler("/error/500");
        app.UseStatusCodePagesWithRedirects("/error/{0}");
        app.UseHsts();
    }

    app.UseHttpsRedirection();

    app.UseStaticFiles();
}
    
```

```

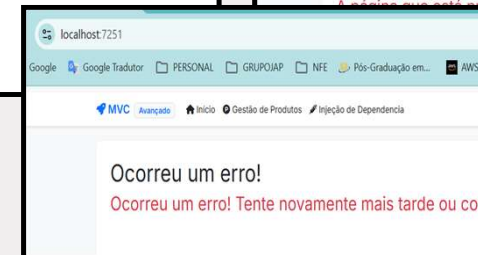
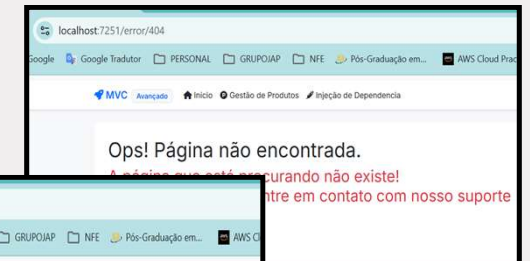
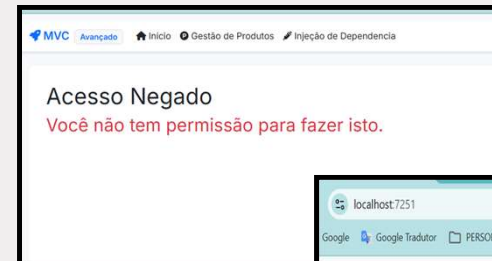
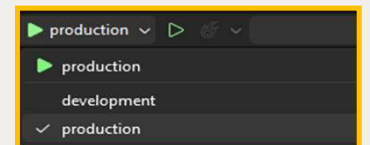
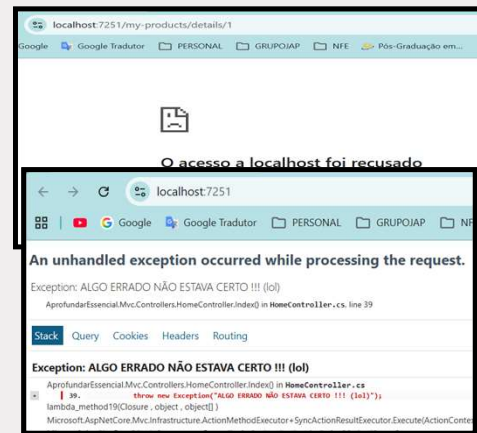
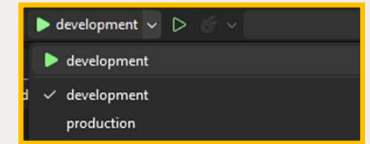
HomeController.cs
AprofundarEssencial.Mvc

return View();
}

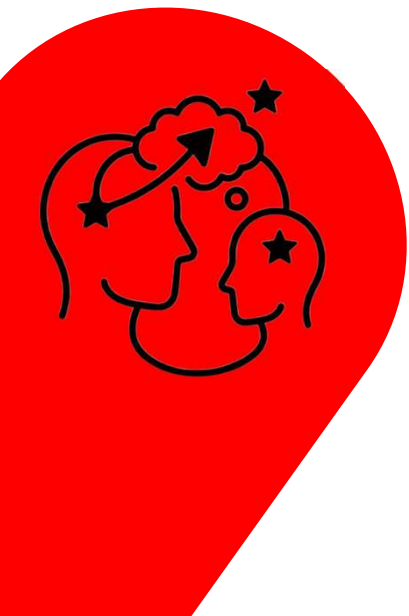
[Route("error/{id:length(3,3)}")]
public IActionResult Errors(int id)
{
    var modelErro = new ErrorViewModel();

    if (id == 500)
    {
        modelErro.Message = "Ocorreu um erro! Tente novamente mais tarde o";
        modelErro.Title = "Ocorreu um erro!";
        modelErro.ErrorCode = id;
    }
    else if (id == 404)
    {
        modelErro.Message = "A página que está procurando não existe! <br>";
        modelErro.Title = "Ops! Página não encontrada.";
        modelErro.ErrorCode = id;
    }
    else if (id == 403)
    {
        modelErro.Message = "Você não tem permissão para fazer isto.";
        modelErro.Title = "Acesso Negado";
        modelErro.ErrorCode = id;
    }
    else
    {
        return StatusCode(500);
    }

    return View("Error", modelErro);
}
    
```



[DEMO] - Tratamento de Erros



Logs para Observabilidade e Monitoria #1

Logs no ASP.NET Core são registros automáticos e estruturados de eventos, erros e atividades que ocorrem durante a execução de uma aplicação, fundamentais para monitoramento, depuração e diagnóstico de problemas. Eles utilizam a API ILogger e podem ser enviados para diversos destinos (console, arquivos, nuvem) usando provedores (*providers*). [Microsoft Learn +2](#)

Configure ASP.NET Core SDK

☒ Error Monitoring ☐ Logs ☐ Metrics ☐ Tracing

Install

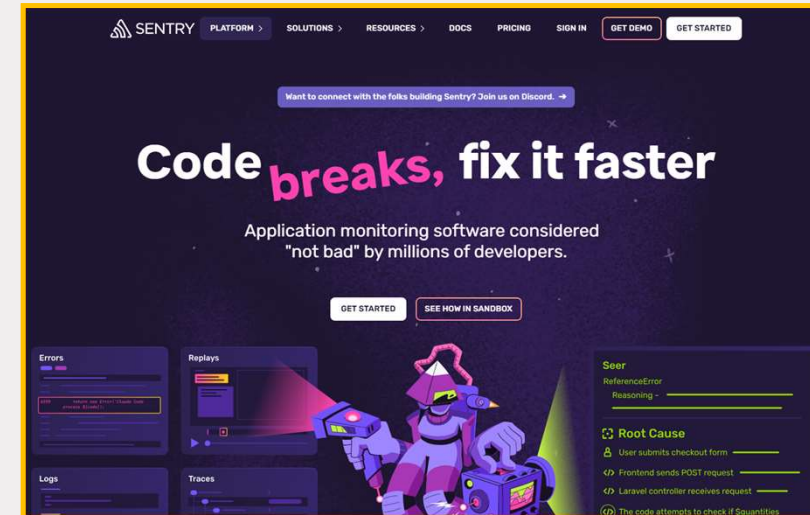
Install the NuGet package:

```
Package Manager .NET Core CLI
Install-Package Sentry.AspNetCore -Version 6.4.1
```

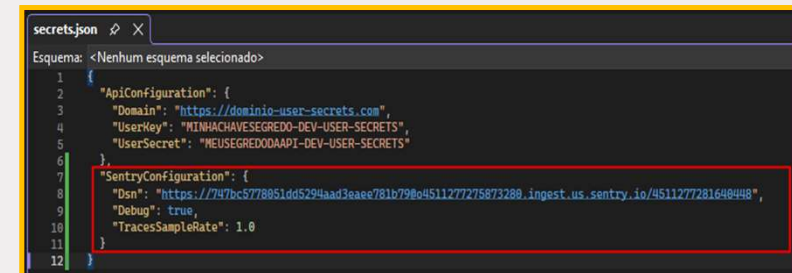
Configure SDK

Add Sentry to `Program.cs` through the `WebHostBuilder`:

```
public static IHostBuilder CreateHostBuilder(string[] args) =>
{
    Host.CreateDefaultBuilder(args)
        .ConfigureWebHostDefaults(webBuilder =>
        {
            // Add the following line:
            webBuilder.UseSentry(o =>
            {
                o.Dsn = "https://747bc5778851dd5294aad3eae781b79@o4511277275873280.ingest.us.sentry.io/4511277281648448";
                // When configuring for the first time, to see what the SDK is doing:
                o.Debug = true;
            });
        });
}
```



<https://sentry.io>



Logs para Observabilidade e Monitoria #2

```

LoggingConfig.cs
AprofundarEssencial.Mvc
1 using Sentry.Extensibility;
2
3 namespace AprofundarEssencial.Mvc.Configurations
4 {
5     0 referências
6     public static class LoggingConfig
7     {
8         1 referência
9         public static WebApplicationBuilder AddLoggingConfiguration(this WebApplicationBuilder builder)
10         {
11             // Aqui integramos o Sentry diretamente no WebHost
12             builder.WebHost.UseSentry(options =>
13             {
14                 // Puxamos a DSN do appsettings.json para segurança
15                 builder.Configuration.GetSection("SentryConfiguration").Bind(options);
16
17                 // Configurações para demonstração em aula
18                 options.TracesSampleRate = 1.0;
19                 options.IncludeActivityData = true;
20                 options.MaxRequestBodySize = RequestSize.Always;
21                 options.SendDefaultPii = true; // Útil para ver o usuário do Identity no log
22             });
23
24             return builder;
25         }
26     }
    
```

```

Program.cs
AprofundarEssencial.Mvc
1 using AprofundarEssencial.Mvc.Configurations;
2
3 var builder = WebApplication.CreateBuilder(args);
4
5 builder
6     .AddMvcConfiguration()
7     .AddIdentityConfiguration()
8     .AddDependencyInjectionConfiguration()
9     .AddLoggingConfiguration();
10
11 var app = builder.Build();
12
13 app.UseMvcConfiguration();
14
15 app.Run();
16
    
```

```

HomeController.cs
AprofundarEssencial.Mvc
74 [Route("home-error")]
75
76 0 referências
77 public IActionResult TestError()
78 {
79     throw new Exception("Na Controller Home gerar erro de demonstração para o Módulo Avançado!");
80 }
81
82
    
```

```

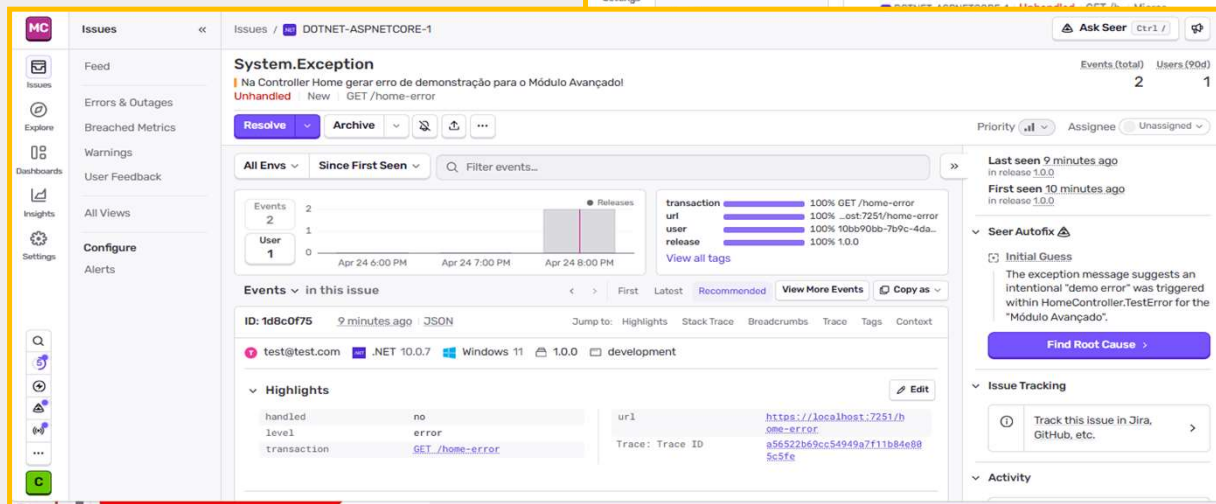
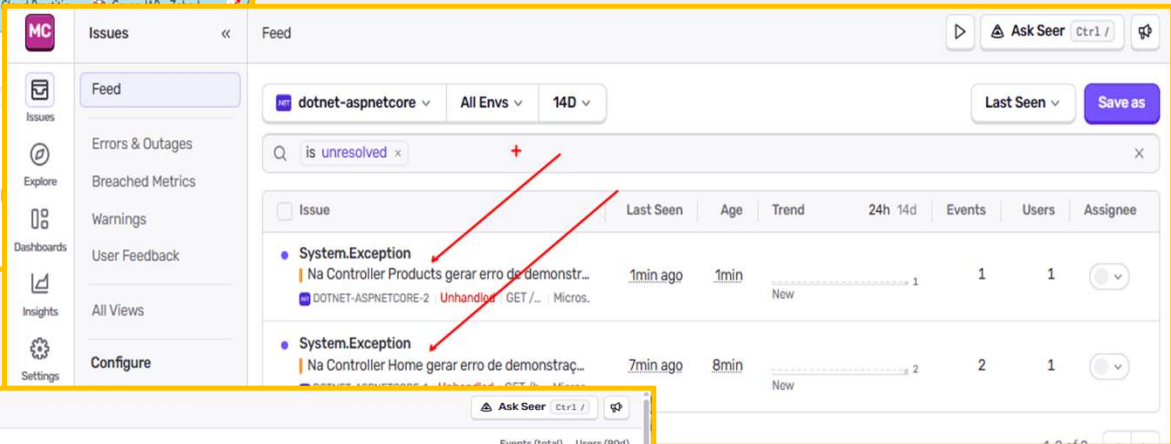
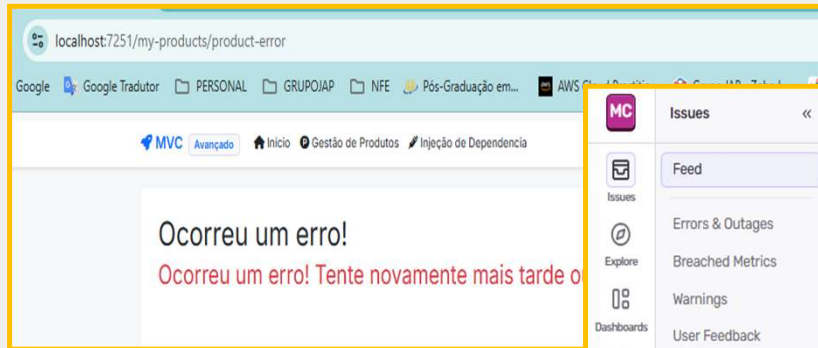
ProductsController.cs
AprofundarEssencial.Mvc
132 [AllowAnonymous]
133 [Route("product-error")]
134
135 0 referências
136 public IActionResult TestError()
137 {
138     throw new Exception("Na Controller Products gerar erro de demonstração para o Módulo Avançado!");
139 }
140
    
```

```

HomeController.cs
AprofundarEssencial.Mvc
44 [Route("error/{id:length(3,3)}")]
45
46 0 referências
47 public IActionResult Errors(int id)
48 {
49     var modeloErro = new ErrorViewModel();
50
51     if (id == 500)
52     {
53         modeloErro.Message = "Ocorreu um erro! Tente novamente mais tarde ou contate nosso suporte.";
54         modeloErro.Title = "Ocorreu um erro!";
55         modeloErro.ErrorCode = id;
56         SentrySdk.CaptureMessage($"Erro HTTP detectado: {id}", SentryLevel.Fatal);
57     }
58     else if (id == 404)
59     {
60         modeloErro.Message = "A página que está procurando não existe! <br />Em caso de dúvidas entre em contato conosco.";
61         modeloErro.Title = "Ops! Página não encontrada.";
62         modeloErro.ErrorCode = id;
63         SentrySdk.CaptureMessage($"Erro HTTP detectado: {id}", SentryLevel.Warning);
64     }
65     else if (id == 403)
66     {
67         modeloErro.Message = "Você não tem permissão para fazer isto.";
68         modeloErro.Title = "Acesso Negado";
69         modeloErro.ErrorCode = id;
70         SentrySdk.CaptureMessage($"Erro HTTP detectado: {id}", SentryLevel.Warning);
71     }
72     else
73     {
74         SentrySdk.CaptureMessage($"Erro HTTP detectado: {id}", SentryLevel.Fatal);
75     }
76     return StatusCode(500);
77 }
78
79 return View("Error", modeloErro);
80
81
82
83
    
```



Logs para Observabilidade e Monitoria #3



<https://my-own-company-em.sentry.io/issues/errors-outages/?project=4511277281640448>



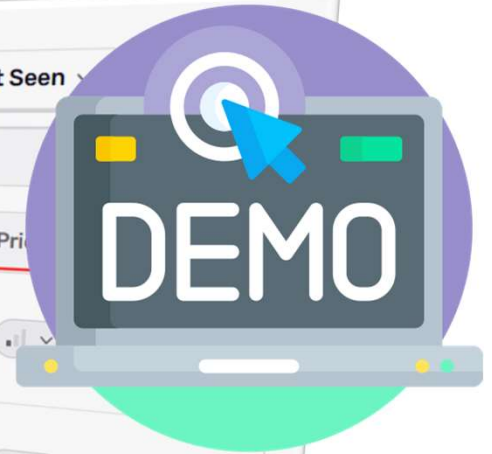
[DEMO] - Observabilidade e Monitoria



dotnet-aspnetcore All Envs 14D Last Seen

is unresolved issue.category is error or outage

Issue	Last Seen	Age	Trend	24h	14d	Events	Users	Pri
[email] Acessou: https://localhost:7251/my-products... POST /my-products/edit-item/{id} DOTNET-ASPNETCORE-8	13s ago	13s	New	1	1	1	1	
[email] Acessou: https://localhost:7251/my-products... GET /my-products DOTNET-ASPNETCORE-5	13s ago	1min	New	5	5	1	1	
[email] Acessou: https://localhost:7251/my-products... GET /my-products/edit-item/{id} DOTNET-ASPNETCORE-7	23s ago	23s	New	1	1	1	1	
[email] Acessou: https://localhost:7251/error/404 GET /error/{id:length(3,3)} DOTNET-ASPNETCORE-4	23s ago	2min	New	5	5	1	1	
Erro HTTP detectado: 404 GET /error/{id:length(3,3)}	23s ago	59min	New	7	11	2	2	



Auditoria/Log através de Filtro

```

AuditFilter.cs
using Microsoft.AspNetCore.Http.Extensions;
using Microsoft.AspNetCore.Mvc.Filters;
using Microsoft.AspNetCore.Mvc;

namespace AprofundarEssencial.Mvc.Extensions
{
    public class AuditFilter : IActionFilter
    {
        public void OnActionExecuted(ActionExecutedContext context)
        {
            if (context.HttpContext.User.Identity.IsAuthenticated)
            {
                var message = context.HttpContext.User.Identity.Name + " Acesso: " + context.HttpContext.Request.GetDisplayUrl();
                // NOTE: The observability service is used to store audit logs, but due to cost considerations it may not be the appropriate service.
                SentrySdk.CaptureMessage(message, SentryLevel.Info);
            }
        }

        public void OnActionExecuting(ActionExecutingContext context)
        {
            //throw new NotImplementedException();
        }
    }
}
    
```

```

MvcConfig.cs
using AprofundarEssencial.Mvc.Extensions;

public void ConfigureServices(IServiceCollection services)
{
    //...
    builder.Services.AddControllersWithViews(options =>
    {
        options.Filters.Add(new AutoValidateAntiforgeryTokenAttribute());
        options.Filters.Add(typeof(AuditFilter));
    });
}
    
```

```

public void OnActionExecuted(ActionExecutedContext context)
{
    if (context.HttpContext.User.Identity.IsAuthenticated)
    {
        var message = context.HttpContext.User.Identity.Name + " Acesso: " + context.HttpContext.Request.GetDisplayUrl();
        // NOTE: The observability service is used to store audit logs, but due to cost considerations it may not be the appropriate service.
        SentrySdk.CaptureMessage(message, SentryLevel.Info);
    }
}
    
```

Issue	Last Seen	Age	Trend	24h	14d	Events	Users	Priority
[email] Acessoou: https://localhost:7251/my-products... POST /my-products/edit-item/{id} DOTNET-ASPNETCORE-8	13s ago	13s	New	1		1	1	
[email] Acessoou: https://localhost:7251/my-products GET /my-products DOTNET-ASPNETCORE-5	13s ago	1min	New	5		5	1	
[email] Acessoou: https://localhost:7251/my-products... GET /my-products/edit-item/{id} DOTNET-ASPNETCORE-7	23s ago	23s	New	1		1	1	
[email] Acessoou: https://localhost:7251/error/404 GET /error/{id:length(3,3)} DOTNET-ASPNETCORE-4	23s ago	2min	New	5		5	1	
Erro HTTP detectado: 404 GET /error/{id:length(3,3)}	23s ago	58min		7		11	2	

[email] Acessoou: https://localhost:7251/my-products

GET /my-products

New

Resolve Archive Filter events...

Events 6 Since First Seen

transaction 100% GET /my-products
url 100% ...st:7251/my-products
user 100% 10bb90bb-7b9c-4da...
release 100% 1.0.0

Events in this issue

ID: 418e9b72 a minute ago JSON Jump to: Highlights Stack Trace Breadcrumbs Trace Tags Context

test@test.com .NET 10.0.7 Windows 11 1.0.0 production

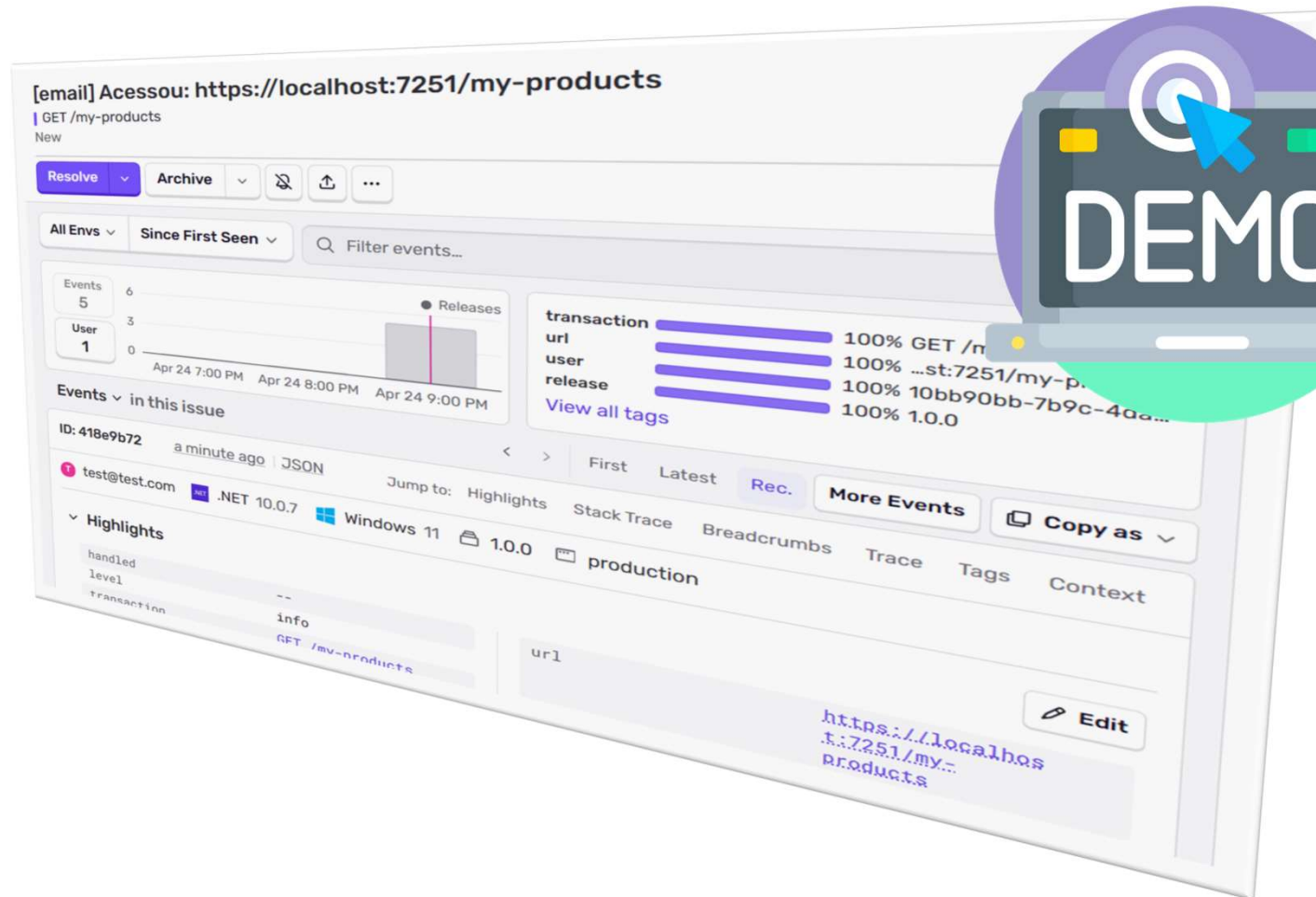
Highlights

handled -- url https://localhost:7251/my-products

<https://my-own-company-em.sentry.io/issues/errors-outages/?project=4511277281640448>



[DEMO] - Monitoria com Filtro





Trabalhar com Culturas

<https://learn.microsoft.com/en-us/aspnet/core/fundamentals/localization?view=aspnetcore-10.0>

Globalização (G11N) e localização (L10N) no ASP.NET Core permitem que aplicativos suportem múltiplos idiomas, formatos de data/moeda e regiões sem refazer o código. A globalização prepara o app para várias culturas, enquanto a localização traduz e ajusta o conteúdo (textos, datas) para locais específicos.  Microsoft Learn +2

```
GlobalizationConfig.cs
AprofundarEssencial.Mvc
1 using Microsoft.AspNetCore.Localization;
2 using System.Globalization;
3
4 namespace AprofundarEssencial.Mvc.Configurations
5 {
6     public static class GlobalizationConfig
7     {
8         public static WebApplication UseGlobalizationConfig(this WebApplication app)
9         {
10             var defaultCulture = new CultureInfo("pt-PT");
11
12             var localizationOptions = new RequestLocalizationOptions
13             {
14                 DefaultRequestCulture = new RequestCulture(defaultCulture),
15                 SupportedCultures = new List<CultureInfo> { defaultCulture },
16                 SupportedUICultures = new List<CultureInfo> { defaultCulture }
17             };
18
19             app.UseRequestLocalization(localizationOptions);
20
21             return app;
22         }
23     }
24 }
```

```
MvcConfig.cs
AprofundarEssencial.Mvc
41 public static WebApplication UseMvcConfiguration(this WebApplication app)
42 {
43     if (app.Environment.IsDevelopment())
44     {
45         app.UseDeveloperExceptionPage();
46     }
47     else
48     {
49         app.UseExceptionHandler("/error/500");
50         app.UseStatusCodePagesWithRedirects("/error/{0}");
51         app.UseHsts();
52     }
53
54     app.UseGlobalizationConfig();
55
56     app.UseHttpsRedirection();
57 }
```

```
_LayoutAdmin.cshtml
AprofundarEssencial.Mvc
74
75 <footer class="border-top footer text-muted mt-5">
76     <div class="container text-center">
77         <p class="mb-0">ASP.NET Core 10 MVC Avançado &copy; @DateTime.Now.Year</p>
78         <small>Formação Profissional - Seu site feito do jeito certo!</small>
79         <p class="mb-0">Idioma selecionado: @System.Globalization.CultureInfo.CurrentCulture.Name</p>
80     </div>
81 </footer>
```

ASP.NET Core 10 MVC Avançado © 2026
Formação Profissional - Seu site feito do jeito certo!
Idioma selecionado: pt-PT



Ajustar validações do front-end

```
Product.cs
AprofundarEssencial.Mvc
1 using System.ComponentModel.DataAnnotations;
2
3 namespace AprofundarEssencial.Mvc.Models
4 {
5
6     13 referências
7     public class Product
8     {
9         [Key]
10         11 referências
11         public int Id { get; set; }
12
13         [Required(ErrorMessage = "0 campo (0) é obrigatório")]
14         12 referências
15         public string? Name { get; set; }
16
17         [Required(ErrorMessage = "0 campo (0) é obrigatório")]
18         11 referências
19         public string? Image { get; set; }
20
21         [Required(ErrorMessage = "0 campo (0) é obrigatório")]
22         12 referências
23         public decimal Price { get; set; }
24
25     }
26 }
```

PS > dotnet ef migrations add AlterProducts
PMC > Add-Migrations AlterProducts
e
PS > dotnet ef database update
PMC > Update-Database

Create Produto

Name

O campo Name é obrigatório

Image

O campo Image é obrigatório

Price

The field Price must be a number.

Create

```
MvcConfig.cs
AprofundarEssencial.Mvc
17
18
19
20 builder.Services.AddControllersWithViews(options =>
21 {
22     options.Filters.Add(new AutoValidateAntiforgeryTokenAttribute());
23     options.Filters.Add(typeof(AuditFilter));
24
25     MvcOptionsConfig.ConfigureModelBindingMessages(options.ModelBindingMessage
26 });
```

```
ValidationScriptsPartial.cshtml
AprofundarEssencial.Mvc
1 <environment name="Development">
2 <script src="/lib/jquery-validation/jquery.validate.js"></script>
3 <script src="/lib/jquery-validation-unobtrusive/jquery.validate.unobtrusive.js"></script>
4 </environment>
5 <environment name="Staging,Production">
6 <script src="https://ajax.aspnetcdn.com/ajax/jquery.validate/1.17.8/jquery.validate.min.js"
7 asp-fallback-src="/lib/jquery-validation/jquery.validate.min.js"
8 asp-fallback-test="window.jQuery && window.jQuery.validator"
9 crossorigin="anonymous"
10 integrity="sha384-r2Fzjg8640n3679bb1q2v49Wf1cc5s833d15s8b741114f0108d878854d"></script>
11 </environment>
12 <script src="https://ajax.aspnetcdn.com/ajax/jquery.validation.unobtrusive/3.2.9/jquery.validate.unobtrusive.min.js"
13 asp-fallback-src="/lib/jquery-validation-unobtrusive/jquery.validate.unobtrusive.min.js"
14 asp-fallback-test="window.jQuery && window.jQuery.validator && window.jQuery.validator.unobtrusive"
15 crossorigin="anonymous"
16 integrity="sha384-ifv8TYDMwBHzvAk228nBR434FL1R1v/Av18DXE43M/1rWdy0G4iZKt8F21SLdds"></script>
17 </environment>
18
19
20 <script>
21 $.validator.methods.range = function (value, element, param) {
22     var globalizedValue = value.replace(",", ".");
23     return this.optional(element) || (globalizedValue >= param[0] && globalizedValue <= param[1]);
24 };
25
26 $.validator.methods.number = function (value, element) {
27     return this.optional(element) || (/^-?\d+(?:\.\d+)?$/).test(value);
28 };
29
30 $.validator.methods.date = function (value, element) {
31     var date = value.split("/");
32     return this.optional(element) || (/Invalid|NaN/.test(new Date(date[2], date[1], date[0]).toString());
33 };
34 </script>
```

```
MvcOptions.cs
AprofundarEssencial.Mvc
1 using Microsoft.AspNetCore.Mvc.ModelBinding.Metadata;
2
3 namespace AprofundarEssencial.Mvc.Configurations
4 {
5     1 referência
6     public class MvcOptionsConfig
7     {
8         1 referência
9         public static void ConfigureModelBindingMessages(DefaultModelBindingMessageProvider messageProvider)
10         {
11             messageProvider.SetAttemptedValueIsInvalidAccessor((x, y) => "0 valor preenchido é inválido para este campo.");
12             messageProvider.SetMissingBindRequiredValueAccessor(x => "Este campo precisa ser preenchido.");
13             messageProvider.SetMissingModelValueAccessor(x => "Este campo precisa ser preenchido.");
14             messageProvider.SetMissingRequestValueAccessor(x => "É necessário que o body na requisição tenha o campo 'Content-Type' definido para 'application/json'");
15             messageProvider.SetNonPropertyAttemptedValueIsInvalidAccessor(x => "0 valor preenchido é inválido para este campo.");
16             messageProvider.SetNonPropertyUnknownValueIsInvalidAccessor(x => "0 valor preenchido é inválido para este campo.");
17             messageProvider.SetNonPropertyUnknownValueIsInvalidAccessor(x => "0 valor preenchido é inválido para este campo.");
18             messageProvider.SetNonPropertyUnknownValueIsInvalidAccessor(x => "0 valor preenchido é inválido para este campo.");
19             messageProvider.SetNonPropertyUnknownValueIsInvalidAccessor(x => "0 valor preenchido é inválido para este campo.");
20             messageProvider.SetNonPropertyUnknownValueIsInvalidAccessor(x => "Este campo precisa ser preenchido.");
21         }
22     }
23 }
```

Create Produto

Name

O campo Name é obrigatório

Image

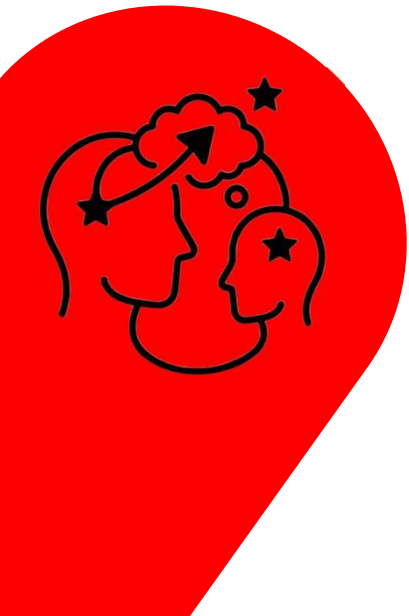
O campo Image é obrigatório

Price

O campo deve ser numérico.

Create

[Back to List](#)



Criar Data Annotation Customizado

```
CoinAttribute.cs
AprofundarEssencial.Mvc
1 using Microsoft.AspNetCore.Mvc.DataAnnotations;
2 using Microsoft.AspNetCore.Mvc.ModelBinding.Validation;
3 using Microsoft.Extensions.Localization;
4 using System.ComponentModel.DataAnnotations;
5 using System.Globalization;
6
7 namespace AprofundarEssencial.Mvc.Extensions
8 {
9     4 referências
10     public class CoinAttribute : ValidationAttribute
11     {
12         0 referências
13         protected override ValidationResult IsValid(object value, ValidationContext validationContext)
14         {
15             try
16             {
17                 var coin = Convert.ToDecimal(value, new CultureInfo("pt-PT"));
18             }
19             catch (Exception)
20             {
21                 return new ValidationResult("Moeda em formato inválido");
22             }
23             return ValidationResult.Success;
24         }
25     }
26
27     2 referências
28     public class CoinAttributeAdapter : AttributeAdapterBase<CoinAttribute>
29     {
30         1 referência
31         public CoinAttributeAdapter(CoinAttribute attribute, IStringLocalizer stringLocalizer) : base(attribute, stringLocalizer)
32         {
33         }
34
35         0 referências
36         public override void AddValidation(ClientModelValidationContext context)
37         {
38             if (context == null)
39             {
40                 throw new ArgumentNullException(nameof(context));
41             }
42
43             MergeAttribute(context.Attributes, "data-val", "true");
44             MergeAttribute(context.Attributes, "data-val-coin", GetErrorMessage(context));
45             MergeAttribute(context.Attributes, "data-val-number", GetErrorMessage(context));
46         }
47
48         2 referências
49         public override string GetErrorMessage(ModelValidationContextBase validationContext)
50         {
51             return "Moeda em formato inválido";
52         }
53
54         1 referência
55         public class CoinValidationAttributeAdapterProvider : IValidationAttributeAdapterProvider
56         {
57             private readonly IValidationAttributeAdapterProvider _baseProvider = new ValidationAttributeAdapterProvider();
58
59             0 referências
60             public IAttributeAdapter GetAttributeAdapter(ValidationAttribute attribute, IStringLocalizer stringLocalizer)
61             {
62                 if (attribute is CoinAttribute coinAttribute)
63                 {
64                     return new CoinAttributeAdapter(coinAttribute, stringLocalizer);
65                 }
66                 return _baseProvider.GetAttributeAdapter(attribute, stringLocalizer);
67             }
68         }
69     }
70 }
```

```
DIConfig.cs
AprofundarEssencial.Mvc
7 namespace AprofundarEssencial.Mvc.Configurations
8 {
9     0 referências
10     public static class DIConfig
11     {
12         1 referência
13         public static WebApplicationBuilder AddDependencyInjectionConfiguration(this WebApplicationBuilder builder)
14         {
15             builder.Services.AddSingleton<IHttpContextAccessor, HttpContextAccessor>();
16
17             // Registos específicos para as interfaces específicas
18             builder.Services.AddTransient<IGenerateGuidServiceTransient, GenerateGuidService>();
19             builder.Services.AddScoped<IGenerateGuidServiceScoped, GenerateGuidService>();
20             builder.Services.AddSingleton<IGenerateGuidServiceSingleton, GenerateGuidService>();
21
22             // Registo do serviço que consome as três dependências
23             builder.Services.AddTransient<OperacaoService>();
24
25             builder.Services.AddSingleton<IValidationAttributeAdapterProvider, CoinValidationAttributeAdapterProvider>();
26
27             builder.Services.AddDbContext<AppDbContext>(o =>
28             {
29                 o.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"));
30             });
31
32             return builder;
33         }
34     }
35 }
```

```
Product.cs
AprofundarEssencial.Mvc
1 using AprofundarEssencial.Mvc.Extensions;
2 using System.ComponentModel.DataAnnotations;
3
4 namespace AprofundarEssencial.Mvc.Models
5 {
6     13 referências
7     public class Product
8     {
9         11 referências
10         [Key]
11         public int Id { get; set; }
12
13         12 referências
14         [Required(ErrorMessage = "O campo {0} é obrigatório")]
15         public string? Name { get; set; }
16
17         11 referências
18         [Required(ErrorMessage = "O campo {0} é obrigatório")]
19         public string? Image { get; set; }
20
21         12 referências
22         [Coin] // [Required(ErrorMessage = "O campo {0} é obrigatório")]
23         public decimal Price { get; set; }
24     }
25 }
```

Create

Produto

Name

Image

Price

as

Moeda em formato inválido

Create

[Back to List](#)



Adicionar Suporte para mais Culturas #1

```
SelectLanguagePartial.cshtml
1 using Microsoft.AspNetCore.Mvc.TagHelpers;
2 using Microsoft.AspNetCore.Localization;
3 using Microsoft.AspNetCore.Mvc.Localization;
4 using Microsoft.Extensions.Options;
5
6 [Inject] IViewLocalizer Localizer;
7 [Inject] IOptions<RequestLocalizationOptions> LocOptions;
8
9 @{
10     var requestCulture = Context.Features.Get<IRequestCultureFeature>();
11     var cultureItems = LocOptions.Value.SupportedUICultures
12         .Select(c => new SelectListItem { Value = c.Name, Text = c.DisplayName })
13         .ToList();
14     var returnUrl = string.IsNullOrEmpty(Context.Request.Path) ? "/" : $"{Context.Request.Path.Value}";
15 }
16
17 <div title="@Localizer["Provedor da cultura do request:"] @requestCulture?.Provider?.GetType().Name">
18     <form id="selectlanguage" asp-controller="Home" asp-action="SetLanguage" asp-route-returnurl="@returnUrl"
19         method="post" class="form-horizontal" role="form">
20         <label asp-for="@requestCulture.RequestCulture.UICulture.Name" @Localizer["Idioma:"]></label>
21         <select name="culture" onchange="this.form.submit();"
22             asp-for="@requestCulture.RequestCulture.UICulture.Name" asp-items="cultureItems">
23             </select>
24     </form>
25 </div>
```

```
_LayoutAdmin.cshtml
74
75 <footer class="border-top footer text-muted mt-5">
76     <div class="container text-center">
77         <p class="mb-0">ASP.NET Core 10 MVC Avançado Ecopy; @DateTime.Now.Year</p>
78         <small>Formação Profissional - Seu site feito do jeito certo!</small>
79         @* <p class="mb-0">Idioma selecionado: @System.Globalization.CultureInfo.CurrentCulture.Name</p> *@
80     </div>
81 </footer>
```

```
HomeController.cs
11
12 public class HomeController : Controller
13 {
14     private readonly IConfiguration Configuration;
15     private readonly ApiService ApiConfig;
16     private readonly IStringLocalizer<HomeController> _localizer;
17
18     // Referências
19     public HomeController(IConfiguration configuration,
20         IOptions<ApiSetting> apiConfiguration,
21         IStringLocalizer<HomeController> localizer)
22     {
23         Configuration = configuration;
24         ApiConfig = apiConfiguration.Value;
25         _localizer = localizer;
26     }
27
28     // Referências
29     public IActionResult Index()
30     {
31         var env = Environment.GetEnvironmentVariable("ASPNETCORE_ENVIRONMENT");
32
33         // Access the configuration values directly from IConfiguration
34         var myDomain_directly = Configuration[$"{ApiSetting.ConfigName}:Domain"] ?? "Default";
35         var userKey2_directly = Configuration[$"{ApiSetting.ConfigName}:UserKey"] ?? "Default";
36         var userSecret2_directly = Configuration[$"{ApiSetting.ConfigName}:UserSecret"] ?? "Default";
37
38         // Access the configuration values
39         var apiConfig = new ApiService();
40         Configuration.GetSection(ApiSetting.ConfigName).Bind(apiConfig);
41
42         var myDomain = apiConfig.Domain ?? "DefaultDomain";
43         var userKey = apiConfig.UserKey ?? "DefaultUserKey";
44         var userSecret = apiConfig.UserSecret ?? "DefaultUserSecret";
45
46         //throw new Exception("ALGO ERRADO NÃO ESTAVA CERTO !!! (lol)");
47
48         return View();
49     }
50
51     [HttpPost]
52     public IActionResult SetLanguage(string culture, string returnUrl)
53     {
54         Response.Cookies.Append(
55             CookieRequestCultureProvider.DefaultCookieName,
56             CookieRequestCultureProvider.MakeCookieValue(new RequestCulture(culture)),
57             new CookieOptions { Expires = DateTimeOffset.UtcNow.AddYears(1) }
58         );
59
60         return LocalRedirect(returnUrl);
61     }
62 }
```

```
GlobalizationConfig.cs
1 using Microsoft.Extensions.Options;
2
3 namespace AprofundarEssencial.Mvc.Configurations
4 {
5     // Referências
6     public static class GlobalizationConfig
7     {
8         //public static WebApplication UseGlobalizationConfig(this WebApplication app)
9         // {
10             // var defaultCulture = new CultureInfo("pt-PT");
11             // var localizationOptions = new RequestLocalizationOptions
12             // {
13                 // DefaultRequestCulture = new RequestCulture(defaultCulture),
14                 // SupportedCultures = new List<CultureInfo> { defaultCulture },
15                 // SupportedUICultures = new List<CultureInfo> { defaultCulture };
16             // };
17             // app.UseRequestLocalization(localizationOptions);
18             // return app;
19         // }
20
21         // Referências
22         public static WebApplication UseGlobalizationConfig(this WebApplication app)
23         {
24             var localizationOptions = app.Services.GetService<IOptions<RequestLocalizationOptions>>();
25             app.UseRequestLocalization(localizationOptions.Value);
26
27             return app;
28         }
29
30         // Referências
31         public static WebApplicationBuilder AddGlobalizationConfig(this WebApplicationBuilder builder)
32         {
33             builder.Services.AddLocalization(options => options.ResourcesPath = "Resources");
34
35             builder.Services.Configure<RequestLocalizationOptions>(options =>
36             {
37                 var supportedCultures = new[] { "en-EN", "pt-PT" };
38                 options.SetDefaultCulture(supportedCultures[0]);
39                 .AddSupportedCultures(supportedCultures)
40                 .AddSupportedUICultures(supportedCultures);
41             });
42
43             return builder;
44         }
45     }
46 }
```



Adicionar Suporte para mais Culturas #2

```
MvcConfig.cs
AprofundarEssencial.Mvc
AprofundarEssencial.Mvc.Configurations.MvcConfig

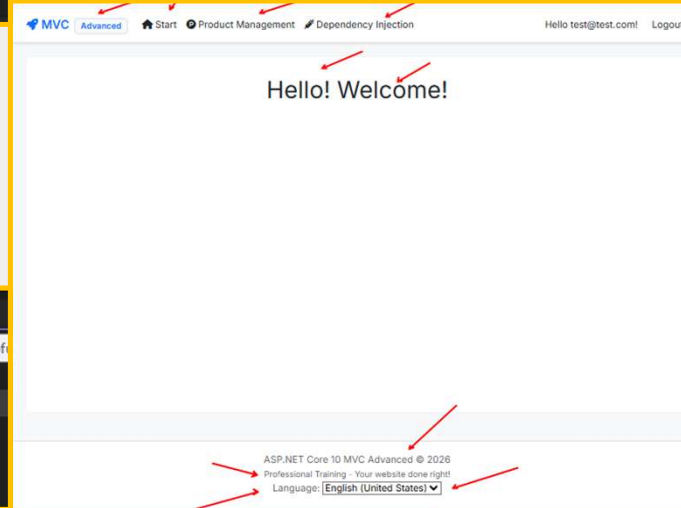
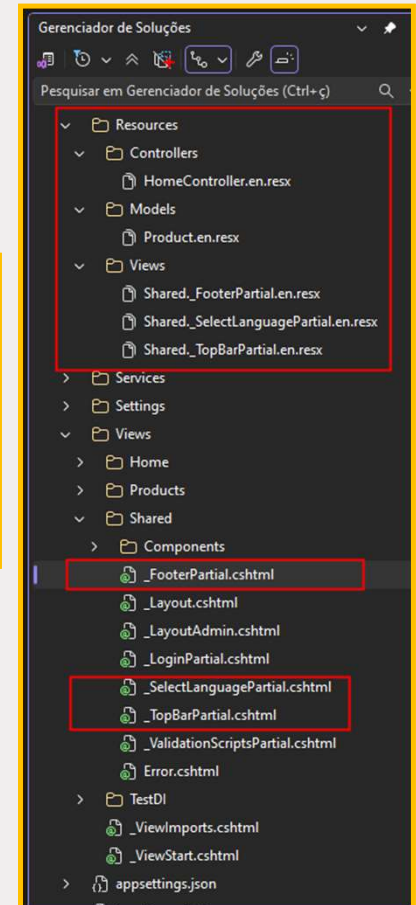
19
20 builder.Services.AddControllersWithViews(options =>
21 {
22     options.Filters.Add(new AutoValidateAntiforgeryTokenAttribute());
23     options.Filters.Add(typeof(AuditFilter));
24
25     MvcOptionsConfig.ConfigureModelBindingMessages(options.ModelBindingMessageProvider);
26
27     .AddViewLocalization(LanguageViewLocationExpanderFormat.Suffix)
28     .AddDataAnnotationsLocalization();
29 }
```

```
Program.cs
AprofundarEssencial.Mvc
AprofundarEssencial.Mvc.Configuration

1 using AprofundarEssencial.Mvc.Configuration;
2
3 var builder = WebApplication.CreateBuilder();
4
5 builder
6     .AddGlobalizationConfig()
7     .AddMvcConfiguration()
8     .AddIdentityConfiguration()
9     .AddDependencyInjectionConfiguration()
10    .AddLoggingConfiguration();
11
12 var app = builder.Build();
13
14 app.UseMvcConfiguration();
15
16 app.Run();
```

```
_FooterPartial.cshtml
AprofundarEssencial.Mvc
AspNetCoreGeneratedDocument.Views_Shared_FooterPartial.en.resx

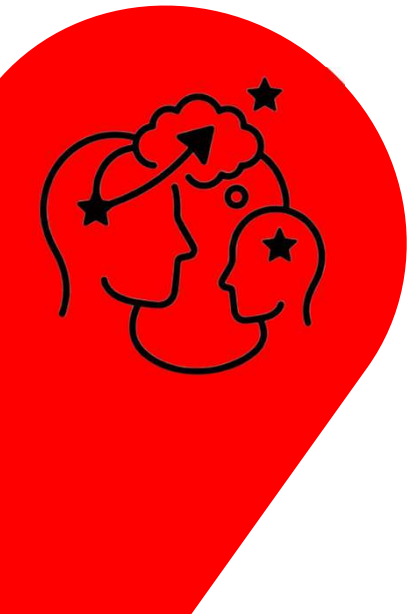
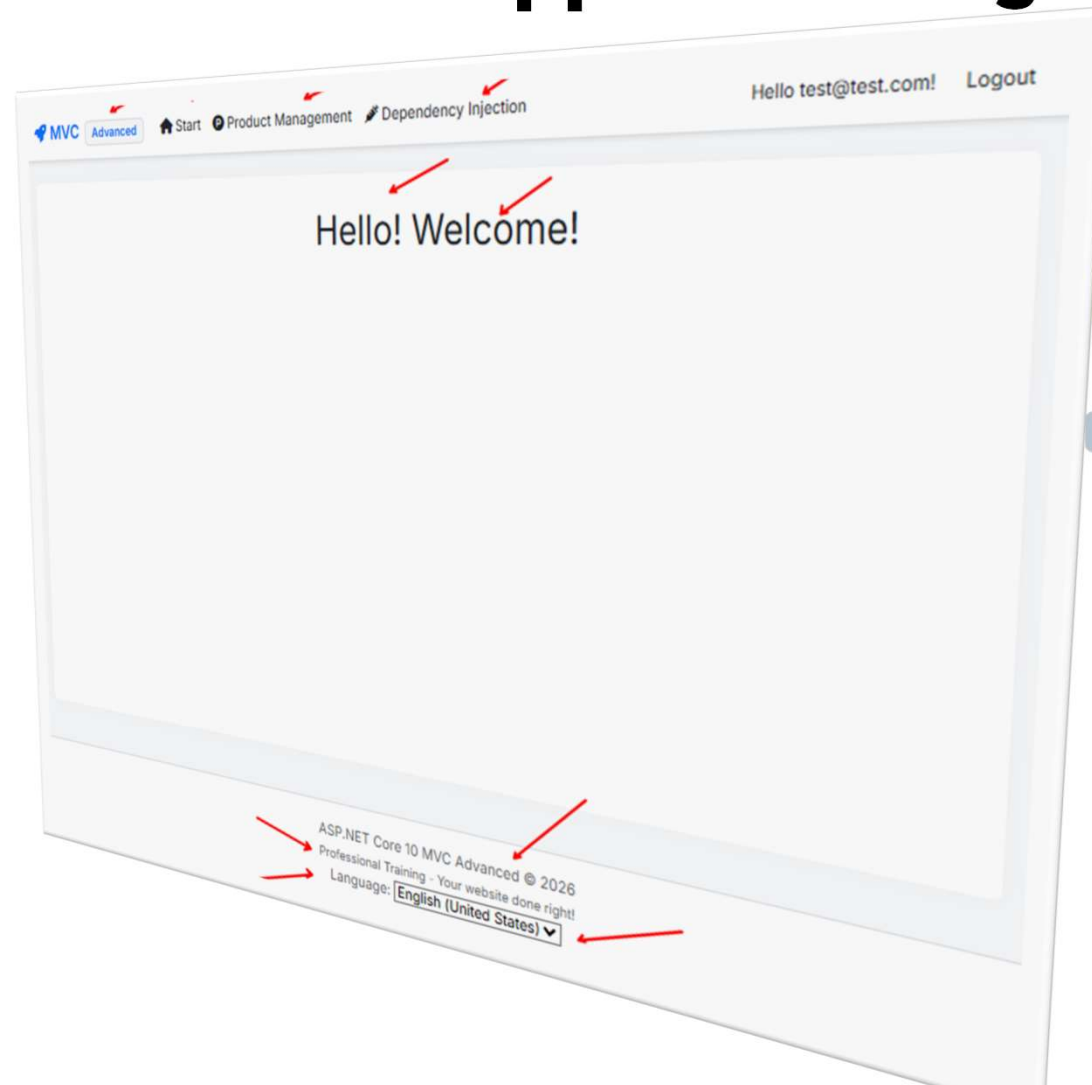
1 @using Microsoft.AspNetCore.Builder
2 @using Microsoft.AspNetCore.Localization
3 @using Microsoft.AspNetCore.Mvc.Localization
4 @using Microsoft.Extensions.Options
5
6 @inject IViewLocalizer Localizer
7 @inject IOptions<RequestLocalizationOptions> LocOptions
8
9 <div class="container text-center">
10     <p class="mb-0">@Localizer["ASP.NET Core 10 MVC Avançado"] &copy;: @DateTime.Now.Year</p>
11     <small>@Localizer["Formação Profissional - Seu site feito do jeito certo"]</small>
12     <partial name="_SelectLanguagePartial" />
13 </div>
```



Nome	en (Inglês)
Avançado	Advanced
Gestão de Produtos	Product Management
Injeção de Dependência	Dependency Injection
Início	Start



[DEMO] - WebApp Multi-Linguagem





<https://getbootstrap.com/docs/5.3/components/modal/#how-it-works>

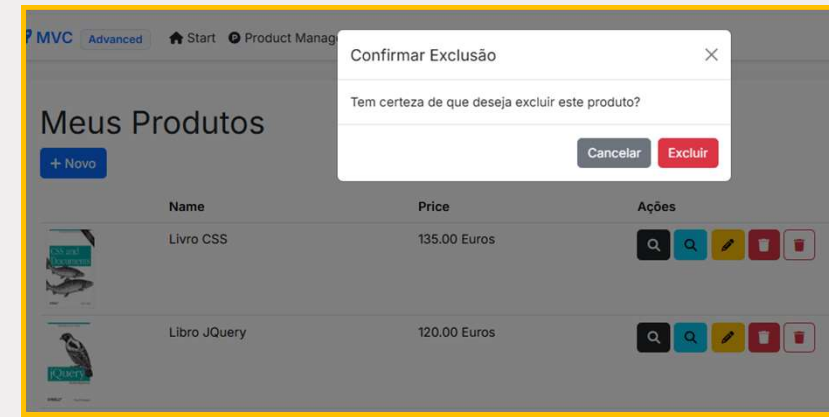
```

55
56
57
58
59
60
61 <div class="modal fade" id="detalhesModal" tabindex="-1" aria-labelledby="detalhesModalLabel" aria-hidden="true">
62   <div class="modal-dialog">
63     <div class="modal-content">
64       <div class="modal-header">
65         <h3 class="modal-title" id="detalhesModalLabel">Detalhes do Produto</h3>
66         <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Fechar"></button>
67       </div>
68       <div class="modal-body">
69         <div class="row">
70           <div class="col-md-4">
71             <img id="produtoImagem" style="height: 180px; width: 61px />
72           </div>
73           <div class="col-md-8">
74             <p><strong>Id:</strong> <span id="produtoId"></span></p>
75             <p><strong>Nome:</strong> <span id="produtoNome"></span></p>
76             <p><strong>Valor:</strong> R$ <span id="produtoValor"></span></p>
77           </div>
78         </div>
79       </div>
80       <div class="modal-footer">
81         <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">Fechar</button>
82       </div>
83     </div>
84   </div>
85 </div>
86
87 <!-- Modal de exclusão -->
88 <div class="modal fade" id="excluirModal" tabindex="-1" role="dialog" aria-labelledby="excluirModalLabel" aria-hidden="true">
89   <div class="modal-dialog" role="document">
90     <div class="modal-content">
91       <div class="modal-header">
92         <h3 class="modal-title" id="excluirModalLabel">Confirmar Exclusão</h3>
93         <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Fechar"></button>
94       </div>
95       <div class="modal-body">
96         <p>Tem certeza de que deseja excluir este produto?</p>
97       </div>
98       <div class="modal-footer">
99         <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">Cancelar</button>
100         <button type="button" class="btn btn-danger" id="confirmarExclusao">Excluir</button>
101       </div>
102     </div>
103   </div>
104 </div>

```

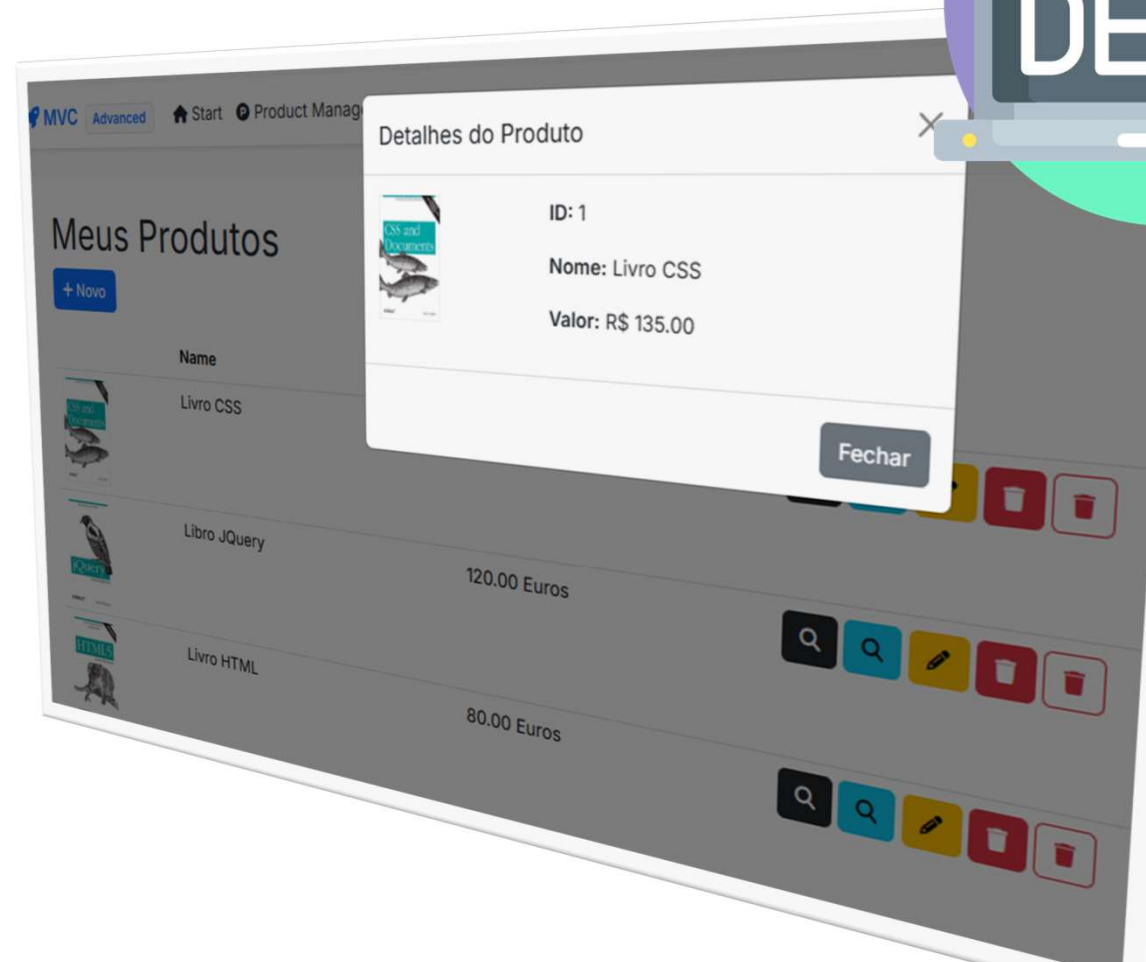
```

100 Index.html
101
102  AprofundarEssencialMvc
103
104  Asp.NetCoreGeneratedContent.Views_Products_In
105
106  @section Scripts {
107  <script>
108
109      $(document).ready(function () {
110
111          $('#modalFormModal').on('show.bs.modal', function (event) {
112              var button = $(event.relatedTarget); // Botão que acionou o modal
113              var id = button.data('id'); // Extrair informações dos atributos data=
114              var nome = button.data('nome');
115              var valor = button.data('valor');
116              var imagem = button.data('imagem');
117
118              // Atualize o conteúdo do modal
119              $('#produtoId').text(id);
120              $('#produtoNome').text(nome);
121              $('#produtoValor').text(valor);
122              $('#produtoImagem').attr('src', '/images/' + imagem);
123          });
124      });
125  </script>
126
127  $(document).ready(function () {
128      var produtoIdParaExcluir; // Variável para armazenar o ID do produto a ser excluído
129
130      var form = $('#antiforgeryform');
131      var token = form[0].value;
132
133      $('#excluirModal').on('show.bs.modal', function (event) {
134          var button = $(event.relatedTarget);
135          produtoIdParaExcluir = button.data('id'); // Armazena o ID do produto a ser excluído
136      });
137
138      $('#confirmarExclusao').click(function () {
139          // Chama o método de exclusão na sua controller
140          $.ajax({
141              url: '/api/products/delete-item/' + produtoIdParaExcluir,
142              type: 'DELETE',
143              headers: { 'RequestVerificationToken': token },
144              success: function () {
145                  // Recarrega a página ou faça outra ação após a exclusão bem-sucedida
146                  location.reload();
147              },
148              error: function () {
149                  alert('Erro ao excluir o produto.');
```



AJAX (Asynchronous JavaScript and XML) é uma técnica de desenvolvimento web que permite atualizar partes de uma página web em segundo plano, sem precisar recarregar o navegador inteiro. Ele combina JavaScript e o objeto XMLHttpRequest para trocar dados com um servidor de forma assíncrona, tornando os sites mais rápidos e responsivos.  MDN Web Docs +3

[DEMO] - Modal Window



Políticas de Cookies

```
MvcConfig.cs
AprofundarEssencial.Mvc
AprofundarEssencial.Mvc.Configurations.MvcConfig

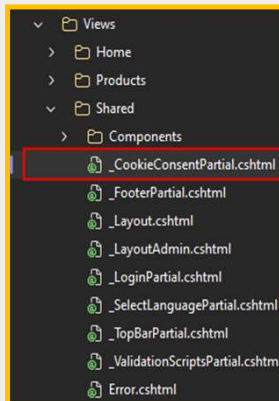
18
19
20
21 builder.Services.AddControllersWithViews(options =>
22 {
23     options.Filters.Add(new AutoValidateAntiforgeryTokenAttribute());
24     options.Filters.Add(typeof(AuditFilter));
25
26     MvcOptionsConfig.ConfigureModelBindingMessages(options.ModelBindingMessageProvider);
27
28     .AddViewLocalization(LanguageViewLocationExpanderFormat.Suffix)
29     .AddDataAnnotationsLocalization();
30
31     builder.Services.Configure<CookiePolicyOptions>(options =>
32     {
33         options.CheckConsentNeeded = context => true;
34         options.MinimumSameSitePolicy = SameSiteMode.None;
35         options.ConsentCookieValue = "true";
36     });
37
```

```
MvcConfig.cs
AprofundarEssencial.Mvc
AprofundarEssencial.Mvc

54 public static WebApplication UseMvcConfiguration(this WebAppl
55 {
56     if (app.Environment.IsDevelopment())
57     {
58         app.UseDeveloperExceptionPage();
59     }
60     else
61     {
62         app.UseExceptionHandler("/error/500");
63         app.UseStatusCodePagesWithRedirects("/error/{0}");
64         app.UseHsts();
65     }
66
67     app.UseGlobalizationConfigC
68
69     app.UseHttpsRedirection();
70
71     app.UseStaticFiles();
72     app.UseCookiePolicy();
73
74     app.UseRouting();
75
76     app.UseAuthorization();
77
```

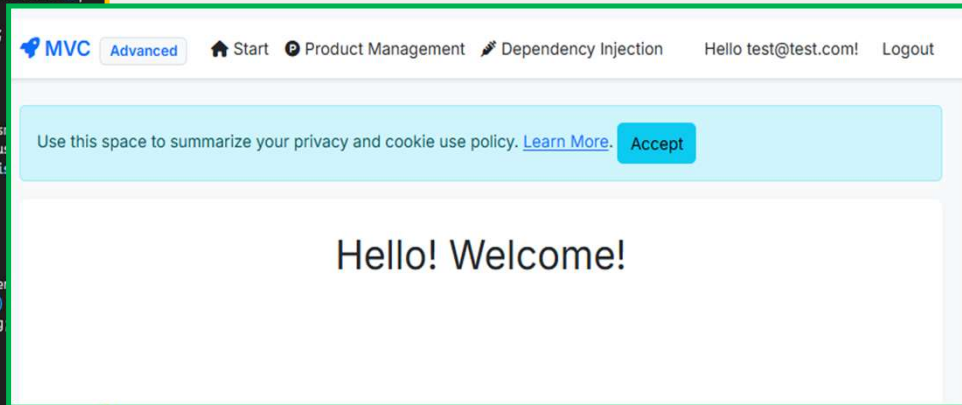
```
_LayoutAdmin.cshtml
AprofundarEssencial.Mvc
Asp

28 <body>
29 <header>
30 <partial name="_TopBarPartial" />
31 </header>
32
33 <div class="container">
34 <partial name="_CookieConsentPartial" />
35
36 <main role="main" class="main-container pb-3">
37 @RenderBody()
38 </main>
39 </div>
40
```

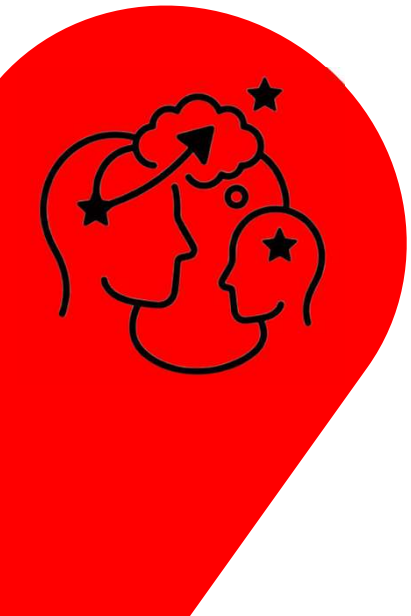
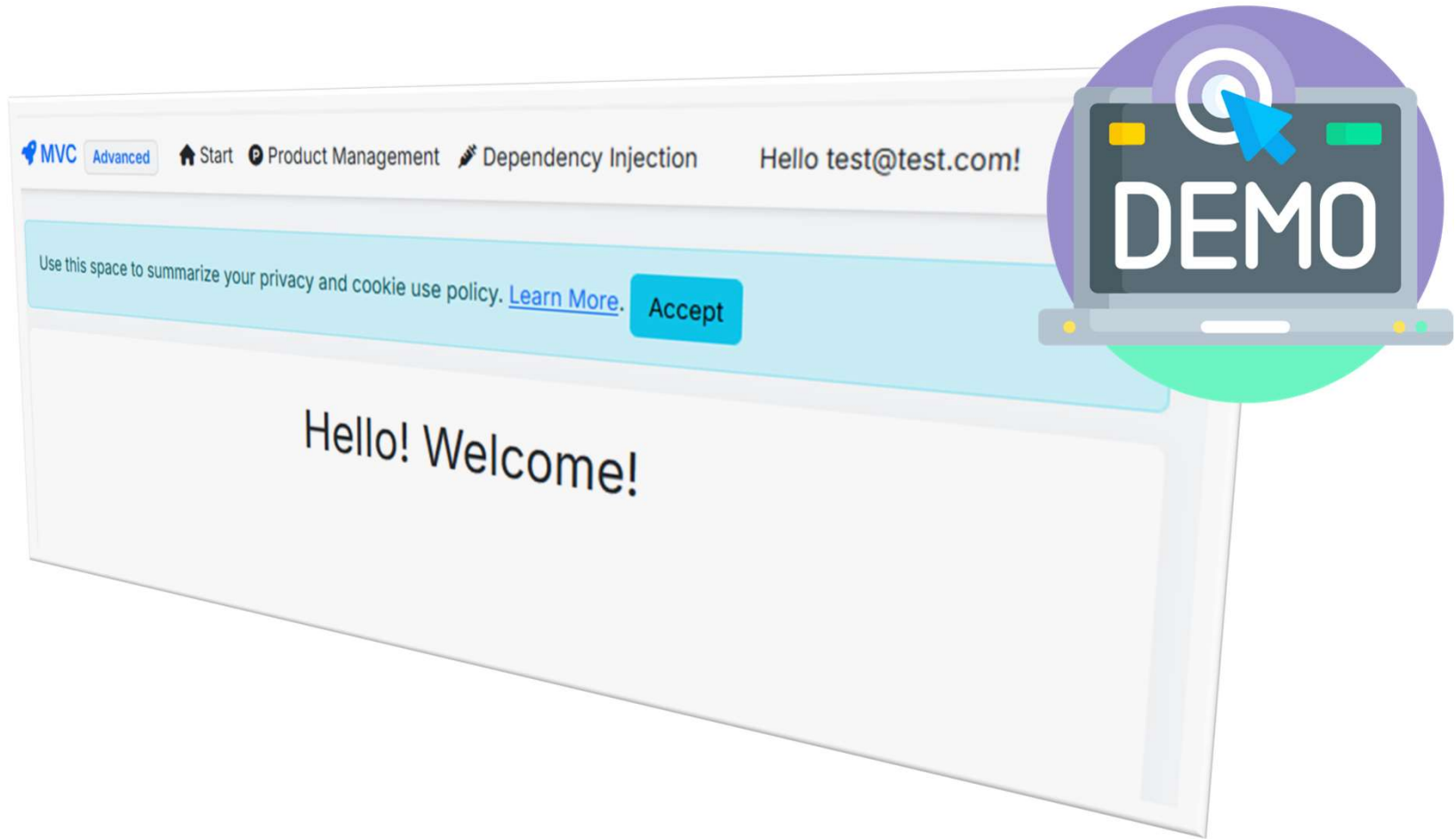


```
_CookieConsentPartial.cshtml
AprofundarEssencial.Mvc
AspNetCoreGeneratedDoc

1 @using Microsoft.AspNetCore.Http.Features
2
3 @if (showBanner)
4 {
5     var consentFeature = Context.Features.Get<ITrackingConsentFeature>();
6     var showBanner = !consentFeature?.CanTrack ?? false;
7     var cookieString = consentFeature?.CreateConsentCookie();
8
9     @if (showBanner)
10     {
11         <div id="cookieConsent" class="alert alert-info alert-dismissible" data-bbox="341 674 554 724">
12             Use this space to summarize your privacy and cookie use policy. <a href="#">Learn More.</a>
13             <button type="button" class="btn btn-info" data-bs-dismiss="alert">Accept</button>
14         </div>
15     }
16
17     <script>
18         (function () {
19             var button = document.querySelector("#cookieConsent button[data-bs-dismiss='alert']");
20             button.addEventListener("click", function (event) {
21                 document.cookie = cookieString;
22             }, false);
23         })();
24     </script>
25 }
```



[DEMO] - Políticas de Cookie



Upload de ficheiro (Create)

```

Products
AprofundarEssencial.Mvc
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
13 referências
public class Product
{
    [Key]
    public int Id { get; set; }
    [Required(ErrorMessage = "O campo (0) é obrigatório")]
    public string? Name { get; set; }
    [NotMapped]
    [Display(Name = "Imagem do Produto")]
    public IFormFile? ImageUpload { get; set; }
    [Required(ErrorMessage = "O campo (0) é obrigatório")]
    public string? Image { get; set; }
    [Coin] [Required(ErrorMessage = "O campo (0) é obrigatório")]
    public decimal Price { get; set; }
}
    
```

```

ProductsController.cs
Product.cs
AprofundarEssencial.Mvc
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
0 referências
private async Task<bool> UploadFile(IFormFile file, string imgPrefix)
{
    if (file.Length <= 0) return false;
    if (System.IO.File.Exists(path))
    {
        ModelState.AddModelError(string.Empty, "Já existe um arquivo com este nome");
        return false;
    }
    using (var stream = new FileStream(path, FileMode.Create))
    {
        await file.CopyToAsync(stream);
    }
    return true;
}
    
```

```

Create.cshtml
AprofundarEssencial.Mvc
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
<div class="row">
    <div class="col-md-4">
        <form enctype="multipart/form-data" asp-action="Create">
            <div class="form-group">
                <input type="text" class="form-control" id="Name" value="" />
                <label class="control-label" for="Name">Nome</label>
            </div>
            <div class="form-group">
                <input type="file" class="form-control" id="ImageUpload" />
                <label class="control-label" for="ImageUpload">Imagem do Produto</label>
            </div>
            <div class="form-group">
                <input type="text" class="form-control" id="Price" value="" />
                <label class="control-label" for="Price">Preço</label>
            </div>
            <div class="form-group">
                <input type="submit" value="Create" class="btn btn-primary" />
            </div>
        </form>
    </div>
    <div class="col-md-8">
        <div class="form-group">
            <input type="text" class="form-control" id="ImageUpload" value="" />
            <label class="control-label" for="ImageUpload">Imagem do Produto</label>
        </div>
        <div class="form-group">
            <input type="text" class="form-control" id="Price" value="" />
            <label class="control-label" for="Price">Preço</label>
        </div>
        <div class="form-group">
            <input type="submit" value="Create" class="btn btn-primary" />
        </div>
    </div>
</div>
<a asp-action="Index" class="btn btn-link">Back to List</a>
</div>
@section Scripts {
    @await Html.RenderPartialAsync("_ValidationScriptsPartial");
}
<script>
    $( "#ImageUpload" ).attr( "data-val", "true" );
    $( "#ImageUpload" ).attr( "data-val-required", "Preencha o campo Imagem." );
</script>
    
```

```

ProductsController.cs
Product.cs
AprofundarEssencial.Mvc
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
[ClaimsAuthorize(claimName: "Product", claimValue: "Create")]
[HttpPost("create-new-item")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create(Product product)
{
    if (ModelState.IsValid)
    {
        var imgPrefix = Guid.NewGuid() + "-";
        if (!await UploadFile(product.ImageUpload, imgPrefix))
        {
            return View(product);
        }
        product.Image = imgPrefix + product.ImageUpload.FileName;
        _context.Add(product);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }
    return View(product);
}
    
```

```

Create.cshtml
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
body {
    padding-top: 10px;
}
.custom-file-button input[type=file] {
    margin-left: -2px;
}
.custom-file-button input[type=file]::webkit-file-upload-button {
    display: none;
}
.custom-file-button input[type=file]::file-selector-button {
    display: none;
}
.custom-file-button:hover label {
    background-color: #ddd;
    cursor: pointer;
}
    
```

MVC
Advanced
Start
Product Management
Dependency Injection
Hello test@tes

Create
Produto

Name

Imagem do Produto
 Nenhum ficheiro selecionado

Price

[Back to List](#)

Abrir

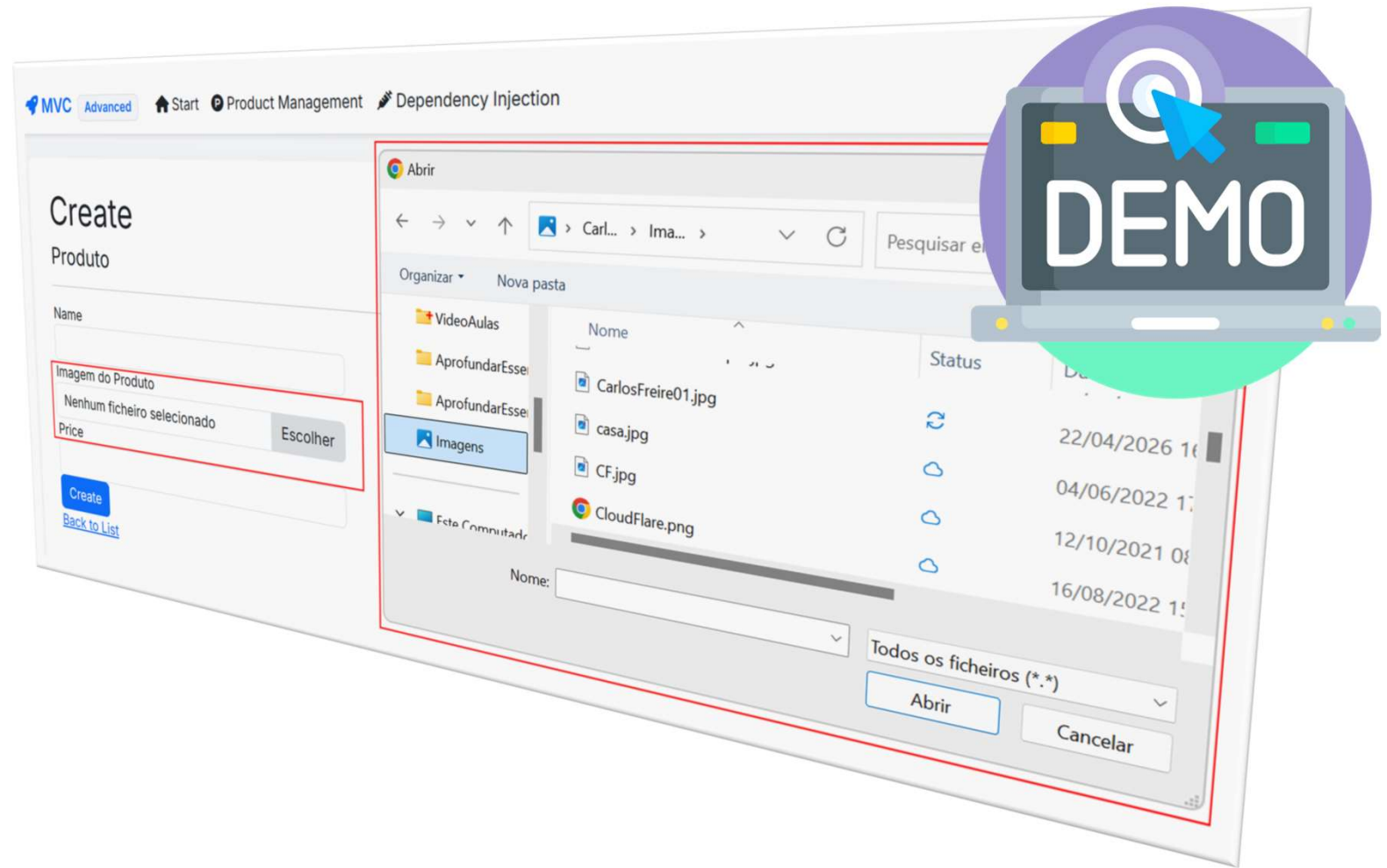
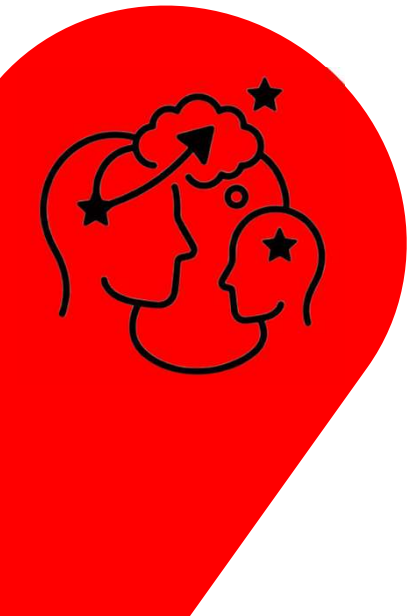
Pesquisar em Imagens

Nome	Status	Data de modif
CarlosFreire01.jpg		22/04/2026 16
casa.jpg		04/06/2022 16
CF.jpg		12/10/2021 00
CloudFlare.png		16/08/2022 16

Nome:
Todos os ficheiros (*.*)



[DEMO] - Upload de ficheiro



Tarefas em Background Services

Background Services no .NET são tarefas executadas de forma assíncrona em segundo plano, essenciais para realizar operações contínuas, periódicas ou de longa duração (como envio de e-mails, processamento de filas ou relatórios) sem bloquear a interface do usuário ou o fluxo principal da aplicação web/API. [Balta.io](#) +1

PS > dotnet add package System.Drawing.Common --version 10.0.7
Ou
PMC > Install-Package System.Drawing.Common -Version 10.0.7

```
MvcConfig.cs | ImageWatermarkService.cs* | Edit.cshtml
AprofundarEssencial.Mvc
41
42
43 builder.Services.AddHsts(options =>
44 {
45     options.Preload = true;
46     options.IncludeSubDomains = true;
47     options.MaxAge = TimeSpan.FromDays(60);
48     options.ExcludedHosts.Add("example.com");
49     options.ExcludedHosts.Add("www.example.com");
50 });
51
52 builder.Services.AddHostedService<ImageWatermarkService>();
53
54
55 return builder;
```

```
Products* | ImageWatermarkService.cs | Edit.cshtml
AprofundarEssencial.Mvc
1 using AprofundarEssencial.Mvc.Extensions;
2 using System.ComponentModel;
3 using System.ComponentModel.DataAnnotations;
4 using System.ComponentModel.DataAnnotations.Schema;
5
6 namespace AprofundarEssencial.Mvc.Models
7 {
8     13 references
9     public class Product
10     {
11         14 references
12         public int Id { get; set; }
13
14         [Required(ErrorMessage = "O campo Nome é obrigatório")]
15         public string? Name { get; set; }
16
17         [NotMapped]
18         [Display(Name = "Imagem do Produto")]
19         public IFormFile? ImageUpload { get; set; }
20
21         [Required(ErrorMessage = "O campo (0) é obrigatório")]
22         public string? Image { get; set; }
23
24         [Column] [Required(ErrorMessage = "O campo (0) é obrigatório")]
25         public decimal Price { get; set; }
26
27         public bool Processed { get; set; }
28     }
29 }
```

PS > dotnet ef migrations add ProcessedInProducts
ou
PMC > Add-Migrations ProcessedInProducts
e
PS > dotnet ef database update
ou
PMC > Update-Database

```
ImageWatermarkService.cs | AprofundarEssencial.Mvc | AprofundarEssencial.Mvc.Services.Image
3 using System.Drawing;
4
5 namespace AprofundarEssencial.Mvc.Services
6 {
7     2 references
8     public class ImageWatermarkService : BackgroundService
9     {
10         private readonly IServiceProvider _serviceProvider;
11         private readonly IWebHostEnvironment _webHostEnvironment;
12         private readonly string _watermarkPath;
13
14         13 references
15         public ImageWatermarkService(IServiceProvider serviceProvider, IWebHostEnvironment webHostEnvironment)
16         {
17             _serviceProvider = serviceProvider;
18             _webHostEnvironment = webHostEnvironment;
19             _watermarkPath = Path.Combine(_webHostEnvironment.WebRootPath, "images/logo_watermark.png");
20         }
21
22         0 references
23         protected override async Task ExecuteAsync(CancellationToken stoppingToken)
24         {
25             while (!stoppingToken.IsCancellationRequested)
26             {
27                 using (var scope = _serviceProvider.CreateScope())
28                 {
29                     var context = scope.ServiceProvider.GetRequiredService<AppDbContext>();
30                     var unprocessedProducts = await context.Products.Where(p => !p.Processed).ToListAsync();
31                     foreach (var product in unprocessedProducts)
32                     {
33                         try
34                         {
35                             var imagePath = Path.Combine(_webHostEnvironment.WebRootPath, "images", product.Image);
36                             var newImagePath = Path.Combine(_webHostEnvironment.WebRootPath, "images", product.Id.ToString() + ".png");
37                             using (Bitmap watermarkImage = new Bitmap(_watermarkPath))
38                             using (Bitmap imageOriginal = new Bitmap(imagePath))
39                             {
40                                 using (var graphic = Graphics.FromImage(newImage))
41                                 {
42                                     graphic.DrawImage(imageOriginal, 0, 0, imageOriginal.Width, imageOriginal.Height);
43                                     var point = new Point(resizedImage.Width - watermarkImage.Width, resizedImage.Height - watermarkImage.Height);
44                                     graphic.DrawImage(watermarkImage, point);
45                                     resizedImage.Save(newImagePath);
46                                 }
47                             }
48                             product.Processed = true;
49                         }
50                         catch (Exception ex)
51                         {
52                             SentrySdk.CaptureException(ex);
53                         }
54                     }
55                     await context.SaveChangesAsync();
56                 }
57                 await Task.Delay(TimeSpan.FromMinutes(1), stoppingToken);
58             }
59         }
60     }
61 }
```

Meus Produtos

+ Novo

	Name	Price
	Livro CSS	135.00 Euros
	Libro JQuery	120.00 Euros



[DEMO] - Adicionar Watermark



Meus Produtos		
+ Novo		
	Name	Price
	Livro CSS	135.00 Euros
	Libro JQuery	120.00 Euros



Encerramento:

Com isso, encerramos a formação **ASP.NET Core MVC Avançado**. Passamos a perceber como funciona de modo avançado o MVC através de conceitos essenciais de front-end, aprofundamos em Injeção de Dependência, Segurança com proteção a CSRF, XSS, HSTS e mais, aprendemos a gerir melhor os ambientes, a tratar melhor os erros e logar de forma inteligente, aprendemos a globalização da aplicação e trabalhar com mais de uma cultura e obtivemos conhecimentos indispensáveis para evoluir mais nosso nível de MVC. Mas entender o MVC é apenas parte necessária para domínio de desenvolvimento de software. Para construir aplicações mais complexas, convido vocês para o próximo nível:

Nossa quinta formação será **POO + SOLID + CLEAN CODE**.

Vamos afinar mais ainda nossos domínios em boas práticas de mercado.

Vejo vocês na próxima formação!

